

An Introduction to MATHCOMP-ANALYSIS



Reynald Affeldt

October 24, 2025

Contents

1	Overview of Rocq and MathComp	9
1.1	A Bit of History	9
1.2	What are Proof Assistants Good for?	10
1.3	Short Presentation of ROCQ	12
1.4	The Rest of this Document	15
2	Introduction to Rocq using SSReflect	17
2.1	The Languages of the ROCQ Proof Assistant	17
2.1.1	GALLINA: the Language of Proofs	17
2.1.2	VERNACULAR: the Language of Commands	18
2.2	Interactive Proof	19
2.3	Discoverability of Definitions and Lemmas	21
2.3.1	Checking Existing Lemmas	21
2.3.2	Searching for Lemmas and Notations	22
2.4	Inductive Types	23
2.4.1	Boolean Numbers	23
2.4.2	Proof by Case Analysis	24
2.4.3	Natural Numbers	25
2.4.4	Recursive Functions	26
2.4.5	Proof by Induction	26
2.5	List Data Structures	27
2.5.1	Lists	27
2.5.2	Vectors	27
2.6	The Leibniz Equality and Rewriting	27
2.7	More Propositional Logic with ROCQ	29
2.8	Predicate Logic: the Existential Quantifier and Sigma-types	29
2.9	Views	30
2.10	Implicit Arguments	31
2.11	Script Management	32
3	Introduction to the MathComp Library	35
3.1	Useful Notation Scopes in MATHCOMP	35
3.2	Generic Definitions and Notations	35
3.3	IMPORTANT Naming Conventions	37

3.3.1	Properties of Operations	37
3.3.2	Properties of Relations	39
3.4	About Mathematical Structures	39
3.5	Building Hierarchies with HIERARCHY-BUILDER	40
3.5.1	ssrbool.v: Boolean Reasoning	42
3.5.2	eqtype.v: Decidable Equality	43
3.5.3	ssrnat.v: Natural Numbers	44
3.5.4	fintype.v: Finite Types	46
3.5.5	seq.v: Lists	47
3.5.6	order.v: Ordered Types	47
3.5.7	IMPORTANT bigop.v: Iterated Operations	48
3.5.8	About Finite Sets	50
3.6	Mathematical Structures in algebra	50
3.6.1	ssralg.v: Algebraic Structures	50
3.6.2	poly.v: Polynomials	51
3.6.3	ssrnum.v: Numeric Types	52
3.6.4	archimedean.v: floor and ceil	52
3.6.5	interval.v: Intervals	53
4	Classical Reasoning using MathComp	55
4.1	Axioms Introduced by MATHCOMP-ANALYSIS	55
4.1.1	Propositional Extensionality	55
4.1.2	Functional Extensionality	56
4.1.3	Constructive Indefinite Description	56
4.1.4	Consequences of Classical Axioms	56
4.2	Naive Set Theory	57
4.2.1	Basic Set-theoretic Operations	57
4.2.2	More Set-theoretic Constructs	58
4.3	Supremum and Infimum	58
4.4	Mathematical Structures in MATHCOMP-ANALYSIS	59
4.4.1	Pointed Types	59
4.4.2	Real Numbers	59
4.5	Convergence	61
4.5.1	Filters	61
4.5.2	Convergence using Filters	62
4.5.3	Filtered Types	62
4.6	Other Structures in MATHCOMP-ANALYSIS	63
4.6.1	Topological Spaces	63
4.6.2	Uniform Spaces	64
4.6.3	Pseudometric Spaces	64
4.6.4	Complete Spaces	65
4.6.5	Normed Modules	66
4.7	near Notations and Tactics	66
4.8	Sequences	70

5	Measure Theory with MathComp-Analysis	73
5.1	Extended Real Numbers	73
5.2	Formalization of σ -algebras	75
5.3	Generated σ -algebra	79
5.4	Formalization of Measures	81
5.4.1	Example: the Dirac Measure	83
5.4.2	Other Instances of Measures	86
5.5	Measurable Functions	86
6	Integration Theory with MathComp-Analysis	87
6.1	Simple Functions	87
6.1.1	Approximation Theorem	88
6.2	Integral of Measurable Functions	89
6.2.1	Integral of a Simple Function	89
6.2.2	Integral of a Non-negative Function	90
6.2.3	Integral of a Measurable Function	90
6.2.4	Properties of the Integral	90
6.3	Monotone Convergence Theorem	91
6.3.1	Monotone Convergence for Simple Functions	91
6.3.2	Monotone Convergence Intermediate Lemma	91
6.3.3	Proof of the Monotone Convergence Theorem	92
6.4	Fubini's Theorem	94
7	Derivation with MathComp-Analysis	97
7.1	Differentiation	97
8	Conclusion	99
	Bibliography	101
A	Cheat Sheets	107
B	Rocq and MathComp Installation	113
	List of Tables	115
	List of Figures	117
	Index	118

Abstract

This document is a memo written for a class of about ten hours held at the Graduate School of Mathematics at Nagoya University from [2022-12-19] to [2022-12-23]. The intent is to provide the necessary background about the ROCQ proof assistant and the MATHCOMP library (for students who already had an exposition to these pieces of software) to be able to understand and get started with the MATHCOMP-ANALYSIS library. This document is meant to be self-contained. Since ROCQ and MATHCOMP are already explained elsewhere, the parts about them are rather cursory, relying on pointers to the appropriate literature such as the ROCQ reference manual [The Coq Development Team, 2024], the original SSREFLECT manual [Gonthier et al., 2016], and the Mathematical Components book [Mahboubi and Tassi, 2021]. And for Japanese readers: [Affeldt, 2017], [Hagiwara and Affeldt, 2018].

Revision history:

- Version 1: [2022-12-23]
- Version 2: [2023-01-27] (typos)
- Version 3: [2023-03-17] (update w.r.t. MATHCOMP-ANALYSIS version 0.6.1)
- Version 4: [2023-04-22] (update w.r.t. MATHCOMP-ANALYSIS version 0.6.2)
- Version 5: [2024-01-03] (update w.r.t. MATHCOMP-ANALYSIS version 0.6.6)
- Version 6: [2024-02-12] (update w.r.t. MATHCOMP-ANALYSIS version 1.0.0)
- Version 7: [2024-06-26] (update w.r.t. MATHCOMP-ANALYSIS version 1.2.0)
- Version 8: [2024-08-06] (update w.r.t. MATHCOMP-ANALYSIS version 1.3.0)
- Version 9: [2025-01-13] (update w.r.t. MATHCOMP-ANALYSIS version 1.8.0)
- Version 10: [2025-10-24] (update w.r.t. MATHCOMP-ANALYSIS version 1.14.0)

Chapter 1

Overview of Rocq and MathComp

Goal of this chapter: This chapter is an overview about proof assistants based on dependent type theory and serves as an introduction to the topic of this document which is more specifically about the MATHCOMP-ANALYSIS library.

1.1 A Bit of History

The creation of proof assistants is the result of research on the foundations of mathematics that has been happening since the last century.

It started with the discovery of contradictions in early set theory, contradictions such as this one explained by Russell in 1901: $a \in a$ is equivalent to $a \notin a$ if one defines $a \stackrel{\text{def}}{=} \{x \mid x \notin x\}$. Set theory has been quickly patched to avoid such contradictions.

The idea to use types to prevent contradictions provides an alternative to set theory to describe the foundations of mathematics. This alternative has been proposed by Russell in 1908:

Whatever contains an apparent variable must not be a possible value
of that variable. (Bertrand Russell, [Russell, 1908])

The theory of types has been developed in the *Principia Mathematica* written in 1910–1913 by Whitehead and Russell [Whitehead and Russell, 1927]. In the 1930s, Curry showed a correspondence between propositional logic and combinators. In 1940, Church used the λ -calculus to propose the simple theory of types [Church, 1940]. It was becoming clear that types could be used to perform proof checking and that there was a relation with algorithms. This ultimately led to the *Curry-Howard correspondence* in 1969 [Howard, 1980].

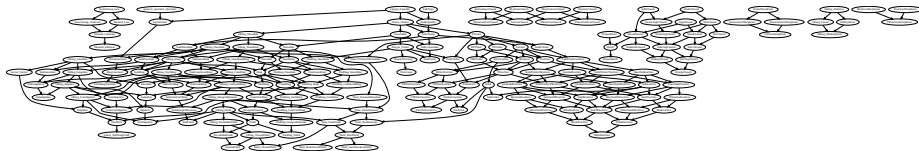
Proof checking using types was soon implemented using a computer by de Bruijn in 1967–1968 as the proof assistant AUTOMATH. Research on type theory and its implementation continued in the 1970s with Martin-Löf and Milner

in particular and led to the implementation of LCF among others. As for the ROCQ proof assistant (ROCQ website), its implementation started in France in 1984 and is still thriving.

It is worth noting that in parallel efforts were also undertaken to put more structure in mathematics. In particular, starting in 1934–1935, Bourbaki has been using set theory for that purpose. On this occasion, Bourbaki put an emphasis on the notion of mathematical structure. However:

Theory of Sets was meant to provide a formally rigorous basis for the whole of the treatise, and the concept of structure represented the ultimate stage of this undertaking. The result, however, was different: Theory of Sets appears as an ad-hoc piece of mathematics imposed upon Bourbaki by his own declared positions about mathematics, rather than as a rich and fruitful source of ideas and mathematical tools.
(Leo Corry, [Corry, 1992])

Retrospectively, it seems that the notion of mathematical structure could not really be used systematically for the lack of a mechanical tool. In some sense, the Mathematical Components project (Mathematical Components website) that started at the Inria-Microsoft Research Joint Center in 2005 can be seen as an attempt at fulfilling this goal. By the way, here is what the hierarchy of mathematical structures in core MATHCOMP looked like in May 2023 (at the time of the release of MATHCOMP 2):



With that many structures, it is no wonder that one cannot manage on paper. See Fig. 3.1, page 41 for a bit more readable hierarchy.

1.2 What are Proof Assistants Good for?

Today formal verification has emerged as a mean to guarantee the correctness of software and mathematics and it is often carried out using proof assistants.

The most obvious application of formal verification is to prevent bugs in computer programs. Formal verification is indeed recommended to provide the highest level of assurance in the international standard Common Criteria for Information Technology Security Evaluation for computer security (ISO/IEC 15408).

Formal verification is also useful to verify large mathematical proofs. The proof of the Kepler conjecture by Hales is a famous example. Though his proof was accepted as a theorem in 1998 by the *Annals of Mathematics*, referees said that they were only “99% certain” of the correctness because of a substantial use of computer programs in the course of the proof. Subsequently, Hales started the Flyspeck project in 2003 to formally verify his proof using the Isabelle/HOL

and the HOL Light proof assistants. It took eleven years. Other famous mathematicians have recognized the need for formal verification:

A technical argument by a trusted author, which is hard to check and looks similar to arguments known to be correct, is hardly ever checked in detail. (Vladimir Voevodsky [Voevodsky, 2014])

The ROCQ proof assistant is a piece of software used to verify computer programs and mathematics. ROCQ has been awarded the ACM SIGPLAN Programming Languages Software Award and the ACM Software System Award in 2013. Xavier Leroy at Collège de France and his colleagues have been awarded the ACM Software System Award in 2021 for their verification of a C compiler in ROCQ. Though there are other proof assistants around (Mizar, Isabelle/HOL, PVS, Lean, etc.), none has received so much academic recognition so far. ROCQ has also been used to formalize mathematics, for example the Four Color Theorem in 2004 [Gonthier, 2008]

Theorem `four_color (m : map R) : simple_map m -> map_colorable 4 m.`

the Odd Order Theorem in 2013 [Gonthier et al., 2013]

Theorem `Feit_Thompson (gT : finGroupType) (G : {group gT}) :
odd #|G| -> solvable G.`

the Abel–Ruffini theorem in 2021 [Bernard et al., 2021], etc. These examples are about algebra, the goal of this class is rather about analysis.

You can find online a list of research papers using MATHCOMP. More generally, research papers on proof assistants can be found in the proceedings of the International Conference on Interactive Theorem Proving (ITP) or the ACM SIGPLAN International Conference on Certified Programs and Proofs (CPP). Yet, the usage of proof assistants has been spreading to other conferences in computer science, in particular programming languages.

The author of this memo has been using ROCQ with colleagues to perform a few experiments, e.g.:

- Formalization of information theory (e.g., Shannon’s source and channel coding theorems [Affeldt et al., 2014]), formalization of error-correcting codes [Affeldt et al., 2020b]
- 3D geometry for robotics [Affeldt and Cohen, 2017]
- Formalization of analysis [Affeldt et al., 2018, Affeldt et al., 2020a], measure and integration theory [Affeldt and Cohen, 2023, Ishiguro and Affeldt, 2024]
- Verification of probabilistic programs [Affeldt et al., 2021, Affeldt et al., 2023, Saito and Affeldt, 2023]

In particular, the formalization of analysis gave rise to an extension of the MATHCOMP library called MATHCOMP-ANALYSIS. It is available online as open source software (MATHCOMP-ANALYSIS) and this will be the main topic of this class.

1.3 Short Presentation of Rocq

It is maybe better to follow this introduction using a proper installation of the ROCQ proof assistant, see Appendix B. Do not worry too much about details, we will come back to them in the next two chapters.

The ROCQ proof assistant is essentially a programming environment in which one can write functions. Let us define the addition of natural numbers (encoded in unary) using the GALLINA language of the ROCQ proof assistant:

```
From mathcomp Require Import ssreflect ssrfun ssrbool eqtype.
From mathcomp Require Import ssrnat.

Fixpoint add n m :=
  if n is n'.+1 then (add n' m).+1
  else m.
```

When writing ROCQ code using the theory **X**, it is better to go to the file **X.v**, copy-paste the header (**Require Imports**, etc.), and add **Require Import X**. This should explain the first two lines above.

We can compute the result of, say, $2 + 3$:

```
Compute add 2 3.
(* = 5 : nat *)
```

What will turn out to be very important is the *type* of expressions. Each expression has a type. The type of 0, 1, 2, etc. is **nat**, the type of natural numbers. The type of **add** is:

```
About add.
(* add : nat -> nat -> nat *)
```

In other words, **add** is a function that takes two natural numbers and returns a natural number. More precisely, **add** is a function that given a natural number returns a function that takes a natural number and returns a natural number, so that **add 2** actually makes sense as an expression:

```
Check add 2.
(* add 2 : nat -> nat *)
```

In the simple case of the addition of natural numbers, the type information should not be surprising since it is what one can find in most typed programming languages.

Instead of natural numbers, let us consider the data structure of lists (notation: $[: a; b; c]$) and their concatenation as implemented by the function **cat** (which is coming from the file ^{MATHCOMP}**seq.v**):

```
From mathcomp Require Import seq.

Compute cat [:: 1; 2; 3] [:: 4].
(* = [:: 1; 2; 3; 4] : seq nat *)

About cat.
(* cat : forall {T : Type}, seq T -> seq T -> seq T *)
```

We can observe that, even though we computed the concatenation of lists of natural numbers, the `cat` function is not restricted to numerical types in particular. It can handle any type: the type of `cat` is parameterized by a type `T` that is generic. `Type` is a type provided by the GALLINA language to mean “any type”. We say that `cat` is *polymorphic*.

We actually brought up the example of lists to explain another, more important feature of the ROCQ proof assistant: *dependent types*, i.e, types that depends on functions’ inputs. The basic example used to illustrate dependent types is the type of “fixed-size lists”; you can think also of the type of vectors in algebra. Let us ignore for now how it is implemented and only look at the construct `[tuple of xyz]` that turns a list `xyz` into a fixed-size list (a `tuple` in MATHCOMP parlance).

```
From mathcomp Require Import tuple.
```

```
Check [tuple of cat [:: 1; 2; 3] [:: 4]].
(* [tuple of [:: 1; 2; 3] ++ [:: 4]] : (3 + 1).-tuple nat *)
```

Thanks to the type of fixed-size lists, the type system of ROCQ displays the size of the list in its type of the form `n.-tuple nat`. This is already a form of program verification in the sense that one can use the type system to verify the output of the `cat` function:

```
Check cat [:: 1; 2; 3] [:: 4] : 4.-tuple _.
Fail Check cat [:: 1; 2; 3] [:: 4] : 3.-tuple _.
(* The command has indeed failed with message:
   The term "[:: 1; 2; 3] ++ [:: 4]" has type "seq nat"
   while it is expected to have type "3.-tuple ?T". *)
```

The fact that the above command succeeds is a proof that the result has the right size.

The property of the `cat` function that we just used can be expressed in general terms:

```
Check (fun n m (x : n.-tuple nat) (y : m.-tuple nat) => [tuple of cat x y]).
(* : forall n m : nat, n.-tuple nat -> m.-tuple nat -> (n + m).-tuple nat *)
```

The `forall` expression indicates that the return type is depending on inputs. This notation suggests that types can be used to represent lemmas.

Indeed, let us work out some proof now. The following expression is a valid type (`[::]` is a notation for the empty list):

```
Check forall l : list nat, cat [::] l = l.
(* forall l : seq nat, [::] ++ l = l : Prop *)
```

Like `Type`, `Prop` is a ROCQ type. `Type` and `Prop` are (essentially) the only types provided by default by ROCQ. `=` is a binary predicate for equality. We will come back to them later.

The following type is also valid:

```
Check forall l : list nat, cat l l = l.
```

The statement is valid, that does not mean that it is true.

We can use ROCQ to discriminate between propositions that hold and propositions that do not hold. It is simply by providing a term that has the appropriate type. For example:

```
Check (fun l => erefl) : forall l : list nat, cat [::] l = l.
```

So, `fun l => erefl` (whatever it is) is a proof that the statement

```
forall l : list nat, cat [::] l = l
```

is true. In contrast, it is not a proof of `forall l : list nat, cat l l = l`:

```
Fail Check (fun l => erefl) : forall l : list nat, cat l l = l.
```

In ROCQ, we can regard types as lemmas and terms as proofs. This is the basic idea of the Curry-Howard correspondence we mentioned in § 1.1.

Let us go back to natural numbers. `fun n => erefl` is also a proof that `add 0 n = n`:

```
Check (fun n => erefl) : forall n, add 0 n = n.
```

But it is not a proof for `add n 0 = n`:

```
Fail Check (fun l => erefl) : forall n, add n 0 = n.
```

What could be a proof then? Here is one:

```
Check (nat_ind (fun n => add n 0 = n) (erefl 0)
  (fun n (ih : add n 0 = n) =>
    eq_trans (f_equal (fun f => f (add n 0)) (erefl S)) (f_equal S ih)))
  : forall n, add n 0 = n.
```

Whatever that term is, it should be clear that it is not going to be practical to require a user to write such programs to serve as proofs.

In the ROCQ proof assistant (actually in most proof assistants), proofs are written incrementally using *tactics*. Tactics are provided to the proof assistant in the form of *scripts*. Here is a script (a one-liner) corresponding to the proof above:

```
Lemma addn0 n : add n 0 = n.
Proof. by elim: n => /= n ->. Qed.

About addn0.
(* addn0 : forall n : nat, add n 0 = n *)
```

We will explain the tactics in the next chapter but you can already guess that, even though it is definitely shorter than providing a term, it is still going to be a bit technical.

Let us conclude with a proof that the addition of natural numbers is commutative. It requires one more intermediate lemma:

```
Lemma addnS m n : add m n.+1 = (add m n).+1.
```

```
Proof. by elim: m => // = m ->. Qed.
```

```
Lemma addC m n : add m n = add n m.
```

```
Proof. by elim: m n => [n|m ih n]; rewrite ?addn0 // = ih -addnS. Qed.
```

This small example shows that to prove the commutativity of the addition of natural numbers `addC`, we needed already two lemmas: `addn0` and `addnS`. When developing a theory of lemmas, which lemma should be proved? How should they be organized in files? How should they be named and documented? There are going to be a lot of them. In fact, when developing a formal theory, many problems are not so much about the proof, because after all its main idea is already known, but rather about problems akin to software engineering.

1.4 The Rest of this Document

Hopefully, we are going to end the week by looking at the proof of Fubini's theorem. In other words, we are going to produce a proof term whose type will be something like:

```
Context d1 d2
```

```
(T1 : measurableType d1) (T2 : measurableType d2) (R : realType).
```

```
Variables (m1 : {sigma_finite_measure set T1 -> \bar R})
```

```
(m2 : {sigma_finite_measure set T2 -> \bar R}).
```

```
Variable f : T1 * T2 -> \bar R.
```

```
Hypothesis imf : (m1 \x m2).-integrable setT f.
```

```
Theorem Fubini :
```

```
\int[m1]_x \int[m2]_y f (x, y) = \int[m2]_y \int[m1]_x f (x, y).
```

For this purpose, we will explain

- the commands and tactics of the ROCQ proof assistant that we will use,
- the mathematical structures provided by MATHCOMP, the mathematical structures provided by MATHCOMP-ANALYSIS (Fig. 4.1), as well as new mathematical structures (for measure theory: semiring of sets, ring of sets, algebra of sets, σ -algebras—Fig. 5.1; etc.), and
- the libraries of definitions, lemmas, notations, etc. that we have used or newly developed.

Chapter 2

Introduction to Rocq using SSReflect

Goal of this chapter: This chapter aims at a technical introduction to the ROCQ proof assistant using its SSREFLECT extension. We try to favor the information that is the most useful to use the MATHCOMP libraries and refer to the ROCQ documentation [The Coq Development Team, 2024] otherwise.

There exist many introductions to the ROCQ proof assistant, most notably the standard textbook [Bertot and Castéran, 2004] (whose French translation is available online [Bertot and Castéran, 2015]). Regarding SSREFLECT tactics, there is [Gonthier et al., 2016, Mahboubi and Tassi, 2021] as well as cheat sheets made available by the developers of SSREFLECT that might be worth printing.

2.1 The Languages of the Rocq Proof Assistant

2.1.1 Gallina: the Language of Proofs

GALLINA is an extensible language with which we will write formal statements and proofs. It is an implementation of a variant of the typed λ -calculus suitable

for constructive reasoning. The core GALLINA terms are (approximate syntax):

<code>t</code>	<code>:=</code>	<code>Prop</code> <code>Set</code> <code>Type</code>	sorts
		<code>x, A</code>	variables
		<code>-</code>	triggers type inference
		<code>forall x : A, B</code>	dependent product
		<code>A -> B</code>	non-dependent product
		<code>fun x => t</code>	function abstraction
		<code>let x := t1 in t2</code>	local definition
		<code>t1 t2</code>	function application
		<code>c</code>	constant
		<code>match t1 with pattern => t2 end</code>	pattern-matching
		<code>fix f x : A := t</code>	anonymous fixpoint

`Type` is actually one identifier hiding a hierarchy (in the sense of subtyping) of types (`Type1`, `Type2`, etc.). We can see the actual indices by using `Set Printing Universes` but that is rarely needed.

`Type` is *predicative*, so that, for example, one can write

```
(forall A : Type, A) : Type
```

but then if the first `Type` is `Type1`, then the second `Type` should be, say, `Type2`. See [Affeldt, 2017, slide 58] for a type derivation.

`Prop` is *impredicative*, so that for example:

```
(forall A : Prop, A -> A) : Prop
```

See [Affeldt, 2017, slide 58] for a type derivation.

`Prop` is intuitively the type of propositions (and predicates). Strictly speaking it is different from the type of boolean numbers (see § 2.4.1). We can however extend the system to blur this difference (see § 3.5.1 and § 4.1) but, in general, the user should be aware that `Prop` and `bool` are different in ROCQ.

`Set` can be understood as `Type0`. There is a ROCQ option to make `Set` impredicative which is occasionally useful (see [Affeldt and Nowak, 2021] for example) but this has not been a concern so far with MATHCOMP and MATHCOMP-ANALYSIS.

There are two useful notations in MATHCOMP for functions. A function that ignores its first argument can be write `fun _ => xyz` or `fun=> xyz`. The notation `f ^~ x` is a replacement for `fun y => f y x`.

The rest of the syntax is similar to programming languages from the ML family.

The user can augment this syntax with new constants using definitions (`Definition`, see § 2.1.2), inductives (`Inductive`, see § 2.4), and notations (see § 2.3.2).

2.1.2 Vernacular: the Language of Commands

The VERNACULAR is a language of commands to organize GALLINA terms. In this document, these commands will appear in **dark magenta** and will be explained along the way.

The **Definition** command binds a term to an identifier. The term is visible and can be used to perform computation. We say it is *transparent*. See **Definition** in the ROCQ manual.

The **Lemma** command binds a type to an identifier. There is a term that is built incrementally by the tactics but it is kept hidden, not available for computation. We say it is *opaque*. **Lemma** is essentially a special case of **Definition**. See **Lemma** in the ROCQ manual.

The language of tactics should also maybe be counted as a third language. It will however be introduced incrementally along this chapter.

2.2 Interactive Proof

In a proof assistant (and in ROCQ in particular), it is maybe better not to think of a proof as something static, that you read, like a book. It is maybe better to think of it as something dynamic, that you interactively execute, and maintain in the long run. Of course, it can also be read, but maybe rather to get the gist of the proof or to retrieve useful information such as the key lemmas or the main technique (proof by contradiction, generalization of the induction principle, etc.).

A proof starts with a statement. At first, the statement is not yet proved, so it appears as a *goal*. Then the user writes a script, and the script shall fulfill the goal upon execution. Under the hood, the script is actually building a monolithic term by incrementally opening and closing subgoals like branches on a tree.

What the interface displays is only the current state which is a *local context* and a subgoal. The goal itself appears as a stack of hypotheses ended by a conclusion. This distinction between local context, stack of hypotheses, and conclusion is important to use tactics efficiently (in particular, SSREFLECT's ones).

Let us use the following notations to describe a goal. In particular, $\mathfrak{T}$ corresponds to the top of the stack of hypotheses:

```

h1 : t1
h2 : t2
...
=====
 $\mathfrak{T} \rightarrow \mathfrak{T}_1 \rightarrow \mathfrak{T}_2 \rightarrow \dots \rightarrow \mathfrak{C}$ 

```

Example: Basic Propositional Logic Proving a lemma means to provide a term whose type reads as the wanted statement. Let us consider the following example: for all propositions A, B, C , $(A \rightarrow B \rightarrow C) \rightarrow (A \rightarrow B) \rightarrow A \rightarrow C$.

This can be proved by providing directly a GALLINA term:

```

Definition axiomS (A B C : Prop) : (A -> B -> C) -> (A -> B) -> A -> C :=
  fun f g a => f a (g a).

```

Obviously, formal proof using **Definition** and GALLINA will not scale, hence the use of tactics.

The **move** Tactic

Used in isolation, **move** does almost nothing.

move can be combined with the *tactical* $\Rightarrow$ (this is not the same arrow as in **fun** $\Rightarrow$...) to pop objects from the stack of hypotheses. From the point of view of the λ -calculus, using the **move** $\Rightarrow$ tactic corresponds to having a λ -abstraction to the proof term. **move** can also be combined with the *tactical* $:$ to push objects to the stack of hypotheses.

- **move** $\Rightarrow$ h pops $\mathcal{T}$ and adds $h:\mathcal{T}$ to the local context.
- **move** $\Rightarrow$ $_$ removes $\mathcal{T}$. More precisely, it removes it after the complete tactic has been executed. This is why you sometimes see tactics like **move** $\Rightarrow$ $_$ $\rightarrow$ that seemingly delete something and then rewrite it, in fact deletion is not performed right away.
- **move**: h_1 puts t_1 as $\mathcal{T}$ (and removes $h_1:t_1$ from the local context, this is often what the user wants).
- **move**: (h_1) puts t_1 as $\mathcal{T}$ but does not remove h_1 from the local context.
- **move** $\Rightarrow$ $/h$ applies the lemma h to $\mathcal{T}$.
- **move** $\Rightarrow$ $/[dup]$ duplicates $\mathcal{T}$.
- **move** $\Rightarrow$ $/[swap]$ swaps $\mathcal{T}$ and $\mathcal{T}_1$.
- **move** $\Rightarrow$ $/[apply]$ replaces $\mathcal{T}$ and $\mathcal{T}_1$ by $(\mathcal{T} \mathcal{T}_1)$.
- **move** $\Rightarrow$ $/(_ x)$ replaces $\mathcal{T}$ with $\mathcal{T} x$.
- **move** $\Rightarrow\{h_1\}$ removes $h_1:t_1$, this is called a *clear-switch*.

That is essentially it about **move**.

Exercise 2.2.1. Correct the spelling:

Lemma ISAAC I S A C : I $\rightarrow$ A $\rightarrow$ S $\rightarrow$ C.

Exercise 2.2.2. Correct the spelling:

Lemma ISAAC I S A C : I $\rightarrow$ (I $\rightarrow$ A) $\rightarrow$ S $\rightarrow$ C.

The `apply` Tactic

From the view point of the λ -calculus, the `apply` tactic corresponds to function application. When used alone, `apply` applies $\mathcal{T}$ to $\mathcal{T}_1 \rightarrow \dots \rightarrow \mathcal{C}$ (so that $\mathcal{T}$ ought better be a function). If you want to apply the lemma `h` to the conclusion, you use `apply: h`. You can often omit the `:` and use `apply h` but this is not recommended in general (MATHCOMP expects `apply:`, in some advanced cases we risk performance problems).

Here is an example of interactive proof:

```

Lemma axiomS (A B C : Prop) : (A -> B -> C) -> (A -> B) -> A -> C.
Proof.
move=> f.
move=> g.
move=> a.
apply: f.
apply: a.
apply: g.
apply: a.
Qed.

```

`exact` is a variant of `apply` that must prove the current goal. This is a *terminating tactic*. Terminating tactic appear in `red`.

Exercise 2.2.3. Prove `forall P Q R : Prop, (P -> Q) -> ((Q -> R) -> (P -> R))`.

2.3 Discoverability of Definitions and Lemmas

In practice, one spends a lot of time searching for existing lemmas to build a proof. It is therefore important to learn as soon as possible how to explore existing libraries to find out already available definitions and lemmas.

2.3.1 Checking Existing Lemmas

The `Print` command prints the term corresponding to an identifier. This is the command that we want to use to see the body of a function when we know its name.

The command `Check t : T` checks that term `t` has type `T`; it prints the type of the term `t`. This is the command to use to see the statement of a lemma given its name.

The command `About t` provides information about the identifier `t`. It is more informative than `Check t`. In particular, it provides information about *implicit arguments* (see § 2.10) that is often useful to understand how to pass a lemma its arguments to apply it.

2.3.2 Searching for Lemmas and Notations

Search is performed using the command **Search**.

Once naming conventions are understood, searching using substrings of the name of a lemma is very effective. Concretely, **Search** "abc" "d". returns the lemmas with names such as **abc*d** or **d*abc*. See § 3.3 for naming conventions in MATHCOMP.

Searching can also be done using patterns to represent the shape of a lemma. It is therefore important to anticipate what can be the shape of the lemma searched for. Here again it is useful to know the naming conventions. Let us assume that the theory of natural numbers has been imported. What are the lemmas using the expression " $2 \times _$ "?

```
Search (2 * _).
```

does not give much information but it indicates the existence of a *.*2* notation that gives better results:

```
Search ( _.*2 ).
```

Another example: How to find the right-distributivity of multiplication over addition of natural numbers?

```
Search ( _ * ( _ + _ ) ).
```

does not give much information. The right way to look for it is by knowing the naming conventions (see § 3.3):

```
Search "mul" "D".
(* mulnDr: right_distributive muln addn
   mulnDl: left_distributive muln addn *)
```

Locate can be used to search for the location of an identifier, i.e., the file in which it has been defined. After having discovered the file, you might want to look at its contents. See Table 2.1 for a few files from the ROCQ distribution worth looking at.

In general, when ROCQ displays symbols that are not characters, this is a notation. For example, the type of pairs is as follows:

```
Inductive prod (A B : Type) : Type := pair : A -> B -> A * B.
```

Here, $A * B$ is a notation for **prod** A B.

Locate can also be used to look for the term behind a notation **Locate** "xyz" looks for a notation that has xyz as part of it. For example:

```
Locate "*".
(* ...
   Notation "x * y" := (prod x y) : type_scope
   ... *)
```

Another example: What is behind the *.*2* notation we saw just above?

```

Locate ".*2".
(* Notation "n .*2" := (double_rec n) : nat_rec_scope
   Notation "n .*2" := (double n) : nat_scope (default interpretation) *)

```

You can observe that notations belong to *scopes* and in case of conflicting notations this is the latest opened scopes that are given priority. So before starting a proof, one has to open the right scopes in the right order. In case of doubt, the stack of notation scopes can be observed using the following command:

```
Print Visibility.
```

If you think that notations are making it harder to understand the current goal, you can disable the display of notations by using:

```
Unset Printing Notations.
```

Less aggressive but often useful is to force ROCQ to print *all* the parentheses:

```
Set Printing Parentheses.
```

Knowing the precise positions of parentheses is in particular useful to design an adequate *Search* pattern.

The user can create new notations but designing a notation in a proof assistant is not an easy task: which ASCII/unicode symbols should we use? what are the right precedence levels? what will happen behind the scene? etc. In general, when one declares a new notation, one needs to choose a precedence level and a scope. It is often useful to check the existing notations and their precedence levels. This can be done using:

```
Print Grammar constr.
```

See also Syntax extensions and notation scopes in the ROCQ reference manual.

Summary: When you are facing an unknown notation or definition, you should locate it, check for its terms, and maybe also search for related lemmas. That is one way to discover formal libraries.

Exercise 2.3.1. What is the ++ notation and where is it defined?

2.4 Inductive Types

The user is not limited to the types provided by default by GALLINA (§ 2.1.1). The command *Inductive* adds to ROCQ constants (a new type and *constructors* for objects of this type) as well as induction principles (automatically generated and proved). Use *Variant* when the induction principles are not needed.

2.4.1 Boolean Numbers

Definition of boolean numbers:

ROCQ libraries	
<code>Init/Datatypes.v</code>	<code>nat</code> , <code>list</code> , etc.
<code>Init/Logic.v</code>	<code>eq_refl</code> , <code>True</code> , etc.
<code>Init/nat.v</code>	<code>nat</code> , <code>sub</code> , etc.
<code>Logic/Classical_Prop.v</code>	axioms for classical reasoning, etc.
...	...
SSREFLECT libraries	
<code>ssr/ssrfun.v</code>	<code>injective</code> , etc.
<code>ssr/ssrbool.v</code>	notations for boolean numbers, etc.
<code>ssrmatching/ssrmatching.v</code>	LHS, RHS
...	...

Table 2.1: Some files of interest in the ROCQ source code (directory `coq/theories`)

```
Inductive bool : Set :=
  true  : bool
| false : bool.
```

This introduces the type `bool` of boolean numbers with two constructors `true` and `false`.

We can do pattern matching with inductive types, e.g.:

```
Definition negb := fun b : bool => if b then false else true.
```

This is actually a notation for:

```
Definition negb := fun b : bool =>
  match b with
  | true  => false
  | false => true
end.
```

More generally, the notation `if ... is ... then ... else ...` performs pattern-matching in only one branch.

Boolean connectives: `negb` (notation `~~`), `andb` (notation `&&`), `orb` (notation `||`), `implyb` (notation `=>`), etc.

2.4.2 Proof by Case Analysis

The existence of several ways to construct the same type calls for *case analysis*.

The `case` Tactic

The `case` tactic performs case analysis. Given `b : bool`, `case: b` (which can be understood as `move: b; case`, where `;` is a tactical to chain tactics in sequence) creates two subgoals: in the first one, `b` has been replaced by `true`, in the second one, `b` has been replaced by `false`. Used without argument, `case` applies to `⊢`.

It is possible to use the `=>` tactical to perform case analysis. The tactic `case` is actually equivalent to `move=> [... | ... | ... | ...]` where the number of `...` is equal to the number of constructors (so no `|` when there is only one constructor). The tactic creates as many subgoals as there are constructors. This is an example of *intro-pattern*¹.

Exercise 2.4.1. Prove `forall b, b || ~~ b`,
`forall b, ( ~~ ~~ b) ==> b`,
`forall a b, ((b ==> a) ==> b) ==> b`,
`forall a b, (~~ b ==> ~~ a) ==> (a ==> b)`.

2.4.3 Natural Numbers

Definition of natural numbers:

```
Inductive nat : Set :=
  0 : nat
| S : nat -> nat.
```

The constructors really are capital letters: `0` and `S`. Observe that the type of `S` is defined using `nat`: inductive types can be recursive.

Along with the definition `nat`, `0`, and `S`, ROCQ generates the `nat_ind` induction principle:

```
Check nat_ind.
(* nat_ind : forall P : nat -> Prop,
   P 0 ->
   (forall n : nat, P n -> P n.+1) ->
   forall n : nat, P n *)
```

This is a term whose type is the induction principle over natural numbers. ROCQ actually proves a standard induction principle for each defined inductive type. We will see in § 2.4.5 how to use it.

In MATHCOMP, `S n` appears as `n.+1` (`+.1` is a notation). There are also the `+.2`, the `+.3`, etc. notations. Strictly speaking, the notation `+.1` is not part of the ROCQ distribution since it comes from a file of MATHCOMP: `ssrnat.v` (see § 3.5.3). We are however anticipating on the next chapter because `ssrnat.v` is so much more practical than the theory of natural numbers from the ROCQ standard library.

Advanced Induction Principles Note that one can also define mutually recursive inductive types in which case the induction principle needs to be defined by the user using the `Scheme` command. This will not be relevant in this document, see [The Coq Development Team, 2024]. As for strong induction, MATHCOMP will provide a solution in the next chapter (§ 3.5.3).

¹More generally, the complete syntax for case analysis is: `case: d-item+ / d-item*` where *d-item* can be a term, an occurrence switch, or a clear switch (i.e., not an intro-pattern). It is advanced usage.

2.4.4 Recursive Functions

The existence of recursive inductive types calls for recursive functions. They can be written with the `Fixpoint` command.

The functions that one can write in ROCQ need to be terminating. When the recursion is *structural* (i.e., it can be decided by a syntactic criterion), the system detects termination automatically. Otherwise, one needs to resort to ROCQ extensions like `Equations` [Sozeau, 2009] or more generally the `Program/Fix` approach (see, e.g., [Saito and Affeldt, 2022, § 3.1] for details). This is occasionally useful but we will not need it in this document.

Here is the example of Fibonacci numbers (the `if-is-then-else` was explained in § 2.4.2):

```
From mathcomp Require Import ssreflect eqtype ssrbool ssrnat.
```

```
Fixpoint fib n :=
  if n is n'.+1 then
    if n' is n''.+1 then
      fib n' + fib n''
    else
      1
  else
    1.
```

```
Compute fib 5.
```

Here is a variant of Fibonacci numbers borrowed from [Appel, 2022]:

```
From mathcomp Require Import ssreflect eqtype ssrbool ssrnat.
```

```
Fixpoint loop n a b :=
  if n is n'.+1 then
    loop n' (a + b) a
  else
    b.
```

```
Definition fastfib n := loop n 1 1.
```

```
Compute fastfib 5.
```

2.4.5 Proof by Induction

The `elim` Tactic

The `elim` tactic applies an induction principle to the goal. Its syntax is similar to the `case` tactic (§ 2.4.2). It looks at the type of its parameter to decide the induction principle to apply, so that `elim: n` performs a proof by induction on natural numbers when `n : nat`.

Exercise 2.4.2. Prove `forall n, n < 2 ^ n`.

Exercise 2.4.3. Prove `forall n, n ^ 2 < 3 ^ n`.

2.5 List Data Structures

2.5.1 Lists

Lists are an example of inductive type with a *parameter*:

```
Inductive list (A : Type) : Type :=
  | nil : list A
  | cons : A -> list A -> list A.
```

Observe that the parameter is shared by all constructors. We will come back to lists in § 3.5.5.

2.5.2 Vectors

Inductive types can also have *indices*: a “parameter” that changes for each constructor. Here is for example the type of vectors from the ROCQ standard library:

```
Inductive t A : nat -> Type :=
  | nil : t A 0
  | cons : forall (h:A) (n:nat), t A n -> t A (S n).
```

Although an archetypal example of dependent type, this type is notoriously difficult to use and not really recommended in practice if avoidable.

Inductive types with indices can also be referred to as *type families*.

2.6 The Leibniz Equality and Rewriting

Until now, inductive types were defining data structures. We now look at inductive types defining propositions.

Equality is defined as an inductive type:

```
Inductive eq (A : Type) (x : A) : A -> Prop :=
  eq_refl : x = x
```

This defines the predicate “being equal to x ” and the only proof is `eq_refl`. Observe that the index is not used.

The corresponding inductive principle is (defined in `Logic.v`^{Rocq}):

```
eq_ind : forall (A : Type) (x : A) (P : A -> Prop),
  P x -> forall y : A, x = y -> P y
```

It says that given a proof of $x = y$, what holds for x also holds for y . This is called the Leibniz equality.

Exercise 2.6.1. Prove the symmetry of equality using `eq_ind` directly (i.e., no `rewrite`).

The `rewrite` Tactic

The `rewrite` tactic is by far the most used tactic [Gonthier and Tassi, 2012]. When `h` is an equality (see § 2.6), the semantics of `rewrite h` is to rewrite the “first” occurrence of the left-hand side in the goal (it sometimes happens that the first occurrence cannot be determined easily, it may be invisible for example). The tactic `rewrite -h` rewrites with `h` in the reverse direction (i.e., trying to match the right-hand side of `h`).

We can be more precise about what we want to rewrite by using *occurrence switches*: `rewrite {3}h` performs rewriting of the 3rd occurrence of the left-hand side of `h`.

We can use patterns to be even more precise w.r.t. what we want to rewrite with the following syntax: `rewrite [pattern]h`. There is a number of predefined patterns: `LHS` for the left-hand side of a goal when it is an equality, `leLHS` for the left-hand side of a goal when it is a large inequality, etc.

We can even use *contextual patterns* to indicate a precise `X` inside a pattern: `rewrite [X in pattern]h` or `rewrite [in X in pattern]h` where *pattern* contains an occurrence of `X`.

See [Gonthier et al., 2016, § 8] or [Mahboubi and Tassi, 2021, § 2.4.1] for more about contextual patterns.

The following idiom isolates some expression `X` in the goal and rewrites it into `t`, generating the equality `(*1*) = t` as a new subgoal:

```
rewrite [X in pattern](_ : _(*1*) = t)
```

The tactic `rewrite` can also be used with a number of *simplification operations* (“s-item”, [Gonthier et al., 2016, § 5.4]):

- `//`: tries to get rid of easy subgoals
- `/=`: tries to perform reduction in the subgoal
- `//=`: combines both

In fact, simplification operations can also be used with `move` and conversely `move` can also perform rewriting:

- `move=> ->` is equivalent to `rewrite  $\mathcal{T}$`
- `move=> {n}->` is equivalent to `rewrite {n} $\mathcal{T}$`

This might seem confusing at first but in practice this quickly feels very natural.

Last, `rewrite /identifier` unfold the definition of `identifier` when it is transparent. In practice, you often want to do it in the course of a sequence of `rewrites`, that is why it is provided as part of the `rewrite` tactic.

Exercise 2.6.2. Complete the following proof:

```
Lemma fastfibE n : fastfib n = fib n.
Proof.
suff : forall i, i <= n -> loop i (fib (n - i + 1)) (fib (n - i)) = fib n.
by move/(_ _ (leqnn n)); rewrite subnn.
```

2.7 More Propositional Logic with Rocq

Definition of falsity:

```
Inductive False : Prop :=
```

This is not a typo: there is no constructor at all. $\sim A$ is a notation for $A \rightarrow \text{False}$.

Exercise 2.7.1. Prove `forall P Q, (P -> Q) -> (~ Q -> ~ P)`.

Definition of truth:

```
Inductive True : Prop := I : True.
```

Disjunction:

```
Inductive or (A B : Prop) : Prop :=
  or_introl : A -> A \\/ B
| or_intror : B -> A \\/ B.
```

The symbol $\vee$ is a notation. When the goal is of the form $A \vee B$, the tactic `left` turns the goal into A , and `right` turns the goal into B . Note that there is also a version of `or` that resides in `Set`:

```
Inductive sumbool (A B : Prop) : Set :=
  left : A -> {A} + {B}
| right : B -> {A} + {B}.
```

We can perform case analysis on a proof of a disjunction with the `case` tactic or with the intro-pattern `[]` (see § 2.4.2).

Conjunction:

```
Inductive and (A B : Prop) : Prop := conj : A -> B -> A /\ B.
```

The symbol $\wedge$ is a notation. Yes, this is really just the `Prop-Prop` version of the `Type-Type` definition of `prod` we saw page 22. When the goal is of the form $A \wedge B$, it is customary to use the `split` tactic. It has the same effect as `apply: xyz` where `xyz` is the only constructor of the inductive type in question. Conversely, to perform case analysis on a proof of a conjunction, we can use `case` tactic or the intro-pattern `[]` (no `|` since there is no additional subgoal to generate: there is only one constructor).

Exercise 2.7.2. Prove the commutativity of the conjunction without using tactics.

Exercise 2.7.3. Prove that $A \wedge B \rightarrow A \vee B$ without using tactics.

Exercise 2.7.4. Prove that $\perp \rightarrow A$ without using tactics.

2.8 Predicate Logic: the Existential Quantifier and Sigma-types

While the universal quantifier is built-in in the logic of ROCQ, the existential quantifier needs to be defined:

```

Inductive ex (A : Type) (P : A -> Prop) : Prop :=
  ex_intro : forall x : A, P x -> exists y, P y

```

Here, A and P are parameters. The constructor has two parameters: the witness x and a proof that P holds for x , i.e., an object of type $P\ x$. This is a *dependent pair*, in the sense that the second projection depends on the first one.

Note that the P predicate is **Prop**-valued and that an existential formula is **Prop**-valued. There are variants that are at least as useful:

```

Inductive sig (A : Type) (P : A -> Prop) : Type :=
  exist : forall x : A, P x -> {x : A | P x}

```

This is referred to as a *sigma-type*.

```

Inductive sigT (A : Type) (P : A -> Type) : Type :=
  existT : forall x : A, P x -> {x : A & P x}

```

Exercise 2.8.1. Prove:

```

Lemma ex_ex_ex (X : Type) (A B : X -> Prop) :
  (exists x, A x /\ B x) -> ((exists x, A x) /\ (exists x, B x)).

```

The existential $\text{exists2 } x : A, P\ x \ \& \ Q\ x$ is equivalent to $\text{exists } x : A, P\ x \ /\ \ Q\ x$ but is defined in such a way that its case analysis can be done in only one step.

2.9 Views

Equivalences are very important and are therefore given a special treatment. There are equivalences between two propositions in **Prop** ($P \leftrightarrow Q$, which is a notation for $(P \rightarrow Q) \wedge (Q \rightarrow P)$) and equivalence between a proposition in **Prop** and a proposition in **bool** (contributing to blurring the difference between **Prop** and **bool**). In the latter case, the equivalence is expressed using a **reflect** predicate:

```

Inductive reflect (P : Prop) : bool -> Set :=
  ReflectT : P -> reflect P true
| ReflectF : ~ P -> reflect P false.

```

What happens when one does a case analysis on a proof of **reflect** $P\ b$? Case analysis is happening w.r.t. b .

```

Lemma test (P : Prop) (b : bool) : reflect P b -> P = b.
Proof.
case.

```

```

P : Prop
b : bool
=====
P -> P = true

goal 2 is:
~ P -> P = false

```

Here is an example of `reflect` taken from `ssrbool.v`:

```
Variable b1 b2 : bool.
Lemma andP : reflect (b1 /\ b2) (b1 && b2)
```

One can apply the view to the goal by using `apply/view` or `exact/view` or to $\mathcal{T}$ (the top of the stack of hypotheses) by using `move/view`. For example:

```
Lemma andbC' (a b : bool) : a && b -> b && a.
Proof.
```

```
a, b : bool
=====
a && b -> b && a
```

```
move=> /andP.
```

```
a, b : bool
=====
a /\ b -> b && a
```

This way, we have switched from a boolean view to a **Prop** view, and then we can use `case`, `move=>`, etc.

2.10 Implicit Arguments

A term can have several parameters and some of them can sometimes be inferred from others. It would be cumbersome to ask the user to provide systematically all the arguments in such cases. It is therefore possible to declare some arguments as *implicit*: the user does not need to provide them explicitly and ROCQ synthesizes them automatically.

Let us look again at the type of lists (§ 2.5.1):

```
About cons.
(* cons : forall {T : Type}, T -> seq T -> seq T *)
```

The parameter T : **Type** is displayed into curly brackets `{ ... }` to indicate that it is implicit. It means that ROCQ will try to infer the parameter T : **Type** given the other arguments. This is the reason why the following command succeeds:

```
Check cons 0 nil.
(* [:: 0] : seq nat *)
```

It is because we are in a context where ROCQ can figure out that `0` is actually a natural number. In case of doubt, one can disable this automatic inference using the `@` prefix:

```
Check @cons nat 0 nil.
```

or

```
Check @cons _ 0 nil.
```

Recall that the underscore `_` is a place holder that ROCQ tries to fill using type inference.

An implicit argument is *strict* when it is inferable from the type of some other arguments (this is the case of `T` in `cons`).

Implicit arguments may also be marked by square brackets `[ ... ]`. This means that the implicit argument is *non-maximally inserted*.

By default, the user has to distinguish itself between implicit and non-implicit arguments by using parentheses, curly or square brackets. The setting of implicit arguments can be decided globally so that ROCQ decides automatically which arguments are implicit. That is why most MATHCOMP files start with the following commands:

```
Set Implicit Arguments.
Unset Strict Implicit.
Unset Printing Implicit Defensive.
```

`Set Implicit Arguments` sets automatically as implicit arguments that can be detected as such. Thanks to `Unset Strict Implicit`, even non-strict arguments are marked as implicit. `Unset Printing Implicit Defensive` is to simplify the display of arguments.

When in some case the setting of implicit arguments turns out to be wrong, this can be adjusted using the command `Arguments`, e.g.:

```
Arguments cons [T].
About cons.
(* cons : forall [T : Type], T -> seq T -> seq T *)
```

See the ROCQ reference manual on Implicit arguments for more details, in particular, for an example of an implicit argument that is not strict.

2.11 Script Management

IMPORTANT Terminating tactics When a tactic solves a subgoal, it is important for maintenance to mark it as such. The tactic `exact` is a terminating tactic in itself but `rewrite`, `apply`: for example are not. In such cases, the `by` tactical should be used. This way, when a script breaks, it will break as soon as possible, usually at a point where it is easier to fix.

IMPORTANT Indentation Indentation is two spaces and it must be done such that the level of indentation at any time indicates the number of subgoal still to be solved. Bullets (`-`, `+`, `*`) are also available to structure scripts it à la org-mode².

²<https://orgmode.org/>

Reorganizing subgoals The application of a tactic can generate several goals. It is often convenient to get rid of easy subgoals in priority. This is often true after a `rewrite` and this can sometimes be just a matter of using a simplification operation. More generally, it is customary to end the tactic with “; `last first`.” so as to bring the subgoal upfront (if there is only one, use `last 2 first` if there are two, etc.)³.

Forward reasoning On paper, mathematical proofs are often performed by *forward reasoning*: intermediate facts are proved incrementally to reach a goal. *Backward reasoning* is then rather when we massage the goal we want to prove. The basic tactic to do backward reasoning is `apply` and since it is such a primitive tactic the user might be tempted to use backward reasoning more than necessary. The main tactic that implements forward reasoning is `have`.

```
have H : statement.
  (* prove statement here *)
  (* continue with H : statement in the local context *)
```

The `have` tactic is also often used to apply a lemma directly:

```
have := lemma_instance.
```

Note the `:=` syntax. With this tactic, $\mathfrak{T}$ is now `lemma_instance` and the proof can go on by using `move`, etc. It is possible (and better) to put intro-patterns between `have` and `:=`.

The tactic `pose` can be used to introduce definitions locally inside a script (see [Gonthier et al., 2016, § 4.1]). The tactic `set` also introduces a local definition but it does that by pattern-matching an expression in the goal or in the local context (see [Gonthier et al., 2016, § 4.1]).

Factorize Arguments with `Section` and `Variable`

Theories (sets of lemmas that share common parameters) are better organized inside *sections* using the commands `Section/End`. Common parameters are introduced at the top of the section using `Variable`, `Hypothesis`, or `Context`. When used inside `Section/End`, `Let` acts like `Definition` inside the section but it is unfolded outside. See also [Mahboubi and Tassi, 2021, § 1.4].

³Note however that ROCQ makes generated subgoals last in the stack of subgoals to be solved by default, and that this setting can be changed, rendering `last first`’s less useful after `rewrite`’s.

Chapter 3

Introduction to the MathComp Library

Goal of this chapter: In this chapter, we review the parts of the MATHCOMP library that are the most useful for MATHCOMP-ANALYSIS. It is mostly about algebra, not yet about analysis.

3.1 Useful Notation Scopes in MathComp

Mathematics relies a lot on notations. It is therefore no surprise that MATHCOMP relies a lot on ROCQ's notations (see § 2.3.2).

Notations are organized in scopes. MATHCOMP provides several notation scopes. For example, depending on the scope, `+` does not have the same meaning. `(_ + _)%N` is the addition of natural numbers, `(_ + _)%R` is the addition of rings, etc.

Notations need not be declared and defined in the same file. For example, notations about natural numbers need not be declared along the definition and the basic theory of natural numbers. The notation `\sum_{(i <= r | P) F}` belongs to the scope `nat_scope` but is declared along with iterated operations (§ 3.5.7). See Table 3.1 for examples of scopes.

3.2 Generic Definitions and Notations

MATHCOMP introduces a number of generic definitions so that definitions are more uniform. For example, the property of right identity is captured by the generic definition `right_id e op`, meaning that for the binary operation `op`, `e` is the neutral element on the right, i.e., `op x e = x`. See Table 3.2.

The downside of such generic definitions is that it complicates search of lemmas for a given operator (as we saw in § 2.3.2).

Scope	Delimiter	Meaning	Where declared
<code>type_scope</code>	<code>type</code>	product, etc.	^{RocQ} <code>Init/Notations.v</code>
<code>function_scope</code>	<code>function</code>	function	<code>Init/Notations.v</code>
<code>bool_scope</code>	<code>B</code>	boolean numbers	^{RocQ} <code>Init/Datatypes.v</code>
<code>nat_scope</code>	<code>N</code>	natural numbers	^{RocQ} <code>Init/Nat.v</code>
<code>fun_scope</code>	<code>FUN</code>	<code>\o, ^~, +%R, -%R</code> , etc.	^{RocQ} <code>ssr/ssrfun.v</code>
<code>pair_scope</code>	<code>PAIR</code>	projections <code>.1, .2</code>	^{RocQ} <code>ssr/ssrfun.v</code>
<code>seq_scope</code>	<code>SEQ</code>	<code>[::]</code> , <code>[:: ...; ...]</code>	^{RocQ} <code>boot/seq.v</code>
<code>order_scope</code>	<code>O</code>	ordered types	^{MATHCOMP} <code>order/order.v</code>
<code>big_scope</code>	<code>BIG</code>	iterated operations	^{MATHCOMP} <code>boot/bigop.v</code>
<code>ring_scope</code>	<code>R</code>	ring	^{MATHCOMP} <code>algebra/ssralg.v</code>
<code>int_scope</code>	<code>Z</code>	integers	^{MATHCOMP} <code>algebra/ssrint.v</code>

Table 3.1: Examples of scopes used in MATHCOMP

Notations about Functions The fact that the function `f` is (monotonically) non-decreasing is noted

```
{homo f : x y / x <= y >-> x <= y}
```

which means `forall` `x y`, `x <= y -> f x <= f y`.

If one uses `mono` instead of `homo`, one gets an equivalence instead of an implication:

```
{mono f : x y / y <= x >-> y <= x}
```

means `forall` `x y`, `(f x <= f y) = (x <= y)`.

The `mono` notation can be used with partially applied functions:

```
{mono +%R x : y z / y < z}
```

which means `forall` `y z`, `(x + y < x + z) = (y < z)`.

```
{morph f : x y / x + y}
```

means `forall` `a b`, `f (x + y) = f x + f y`. For example, here is an alternative statement for the right-distributivity of multiplication over addition of natural numbers:

Lemma `mulnDr' n : {morph muln n : x y / x + y}`.

Proof. `exact: mulnDr. Qed.`

Exercise 3.2.1. What does **Lemma** `opprD : {morph -%R: x y / x + y : V}` mean? (From ^{MATHCOMP}`ssralg.v`.)

injective f	<code>forall x1 x2, f x1 = f x2 -> x1 = x2</code>
cancel f g	<code>g (f x) = x</code>
involutive f	<code>cancel f f</code>
left_injective op	<code>injective (op^~ x)</code>
right_injective op	<code>injective (op y)</code>
left_id e op	<code>e op x = x</code>
right_id e op	<code>x op e = x</code>
left_zero z op	<code>z op x = z</code>
right_zero z op	<code>x op z = z</code>
self_inverse e op	<code>x op x = e</code>
idempotent op	<code>x op x = x</code>
commutative op	<code>x op y = y op x</code>
associative op	<code>x op (y op z) = (x op y) op z</code>
right_commutative op	<code>(x op y) op z = (x op z) op y</code>
left_commutative op	<code><u>x</u> op (y op z) = y op (<u>x</u> op z)</code>
left_distributive op add	<code>(x + y) * z = (x * z) + (y * z)</code>
right_distributive op add	<code>x * (y + z) = (x * y) + (x * z)</code>
left_loop inv op	<code>cancel (op x) (op (inv x))</code>

Table 3.2: A few generic definitions in MATHCOMP

3.3 IMPORTANT Naming Conventions

The user needs to be able to find lemmas quickly and to type them quickly. The names of lemmas therefore need to be easy to remember, short, with a uniform and precise format. This also makes the lemmas easier to search for (in the sense of § 2.3.2).

More generally, being picky about names is customary in programming, see for example the Hungarian notation, or standard literature:

“A name should be informative, concise, memorable, and pronounceable if possible.” [Kernighan and Pike, 1999, § 1.1]

MATHCOMP libraries enforce a strict naming convention. There are a few rules to remember.

3.3.1 Properties of Operations

There is a couple of a long and a short identifiers for most basic operations. In the name of lemmas, the long identifier is typically used as a prefix when it is the head symbol of the left-hand side. When two operators are involved, the main one is used as a prefix and the other one is referred to by its short identifier, e.g., `D` for the addition. See Table 3.3. Furthermore, there are also one-letter identifiers for standard types. For example, `n` corresponds to natural numbers. See Table 3.4. From these rules, we can guess the names of many

lemmas. For example, the right-distributivity of multiplication over addition is `mulnDr` for the natural numbers and `mulrDr` for rings.

Long identifier	Short identifier	Meaning
<code>add</code>	<code>D</code>	addition
<code>sub</code>	<code>B</code>	subtraction
<code>opp</code>	<code>N</code>	opposite
<code>mul</code>	<code>M</code>	multiplication
<code>exp</code>	<code>X</code>	exponentiation (by a natural number)

Table 3.3: Naming Convention: Identifiers for operations

One-letter identifier	Meaning
<code>n</code>	natural numbers
<code>r</code>	elements of a ring
<code>f</code>	elements of a field
<code>e</code>	extended real numbers
<code>y</code>	∞
<code>Ny</code>	$-\infty$

Table 3.4: Naming Convention: Identifiers for positional notation

When a lemma looks like a rewriting rule (most of them do), the name of the lemma is often prefixed by a long identifier (corresponding to the head symbol of the left-hand side) and it is followed by a pattern that corresponds to the shape of the left-hand side. This pattern uses one-letter identifiers. Constants are referred as such (e.g., `0` for the 0 of the addition). Standard types are referred to by their one-letter identifier from Table 3.4. This way, the name of the lemma gives a good idea of what the lemma does. For example, `n0` indicates that the lemma is of the form $n \boxed{\text{op}} 0$. For illustration, `addn0` should correspond to `forall n, n + 0 = n` and indeed:

```

About addn0.
(* addn0 : right_id 0%N addn *)
(* Print right_id.
fun (S T : Type) (e : T) (op : S -> T -> S) => forall x : S, op x e = x
   : forall S T : Type, T -> (S -> T -> S) -> Prop *)

```

Let us call that this naming scheme the *positional notation*.

There is also a number of one-letter identifiers used as suffixes for properties of operations (see Table 3.5). From this table, we can guess that the associativity of `nat` is `mulnA`, its commutativity is `mulnC`, etc. In particular, the cancellation property is typically written with the generic `cancel` predicate and marked with `K`. The suffix `E` is for rewriting lemmas that “unfold” a term into its definition. See Table 3.5.

Identifier	Property
A	associativity
C	commutativity
C	set complement
D	set theoretic difference
K	cancellation
E	equality

Table 3.5: Naming Convention: Suffixes for the properties of operations

le	$\leq$ for ordered types
leq	$\leq$ for <code>nat</code>
lt	$<$ for ordered types
ltn	$<$ for <code>nat</code>

Table 3.6: Naming Convention: Identifiers for relations

3.3.2 Properties of Relations

Table 3.6 does not provide all the information but it is supposed to lead you to valuable lemmas such as:

Lemma `leq_pmul2l m n1 n2 : 0 < m -> (m * n1 <= m * n2) = (n1 <= n2)`.

or

Lemma `ltrDl x y : (x < x + y) = (0 < y)`.

3.4 About Mathematical Structures

In mathematics, structures are organized as a hierarchy, in the sense that, e.g., a field is defined using a ring. In consequence, an element of a field is also an element of the underlying ring. It is an important issue in formal mathematics to get this inheritance right. A *mathematical structure* consists typically of:

1. a carrier (typically an object in `Set` or `Type`)
2. a set of operations (including constants, i.e., 0-ary operations)
3. the properties of the operations (one also says the “axioms” of the base theory, not to be confused with ROCQ `Axioms` which are unproved lemmas)

In proof assistants, the elements of a mathematical structure are declared by an interface. In MATHCOMP, an interface is essentially a *record* (see Record types in the reference manual). A record is in fact an inductive type, and since the axioms depend on the operations, which depend on the carrier, it should be obvious that dependent types are here again put at work. An *instance* of an interface is an implementation of this interface. For example, `nat` (and its

comparison function `eqn`) is an instance of `eqType` (a type with a decidable equality).

But how should we use records? There are several strategies depending on what we put in parameters. *Bundled*: everything as fields. *Semi-bundled*: only the carrier is a parameter. *Unbundled*: only the axioms are fields. In MATHCOMP, mathematical structures are bundled. However, they are implemented by first providing a semi-bundled record (the “class”) and then a bundled record using the class (the “structure”). This is the *packed classed* methodology. To make inference work in presence of inheritance, MATHCOMP uses the mechanism of *canonical structures* of ROCQ in a clever way [Garillot et al., 2009].

Until very recently, the construction of hierarchies was done by hand and it was error-prone. Today, MATHCOMP uses a dedicated tool called HIERARCHY-BUILDER [Cohen et al., 2020]. See § 3.5.

The hierarchy of mathematical structures provided by MATHCOMP is displayed in Fig. 3.1. You can also check for [Gonthier et al., 2016, page 65, version 16 not 17)] for an older but easier to read hierarchy.

The main thing to remember is that the definition of a mathematical structure is to be found in interfaces. In MATHCOMP, they are referred to as *mixins*. It is a technical term that comes from object-oriented programming and it can be understood as an interface.

directory	files of interest
<code>mathcomp/boot</code>	<code>ssrnat.v</code> (§ 3.5.3), <code>seq.v</code> (§ 3.5.5), etc.
<code>mathcomp/order</code>	<code>order.v</code> (§ 3.5.6), etc.
<code>mathcomp/algebra</code>	<code>ssralg.v</code> (§ 3.6.1), <code>ssrnum.v</code> (§ 3.6.3), <code>ssrint.v</code> (§ 3.6.3), <code>matrix.v</code> (not a topic in this document), etc.

Table 3.7: Some files of interest in MATHCOMP, see also Table 2.1 for MATHCOMP files distributed with ROCQ

3.5 Building Hierarchies with Hierarchy-Builder

HIERARCHY-BUILDER is a tool introduced in [Cohen et al., 2020] to facilitate the construction of hierarchies of mathematical structures. It adds new commands to ROCQ, the main ones being `HB.mixin`, `HB.structure`, `HB.factory`, and `HB.instance`. The rest of this document will provide concrete examples of their usage.

Let us just explain in a generic way the most basic scenario. Here is the pattern to declare a structure `Struct` intended to sit at the bottom of a hierarchy. The interface of the structure goes into a mixin:

```
HB.mixin Record isStruct params carrier := {
  ... properties about the carrier ...
}
```

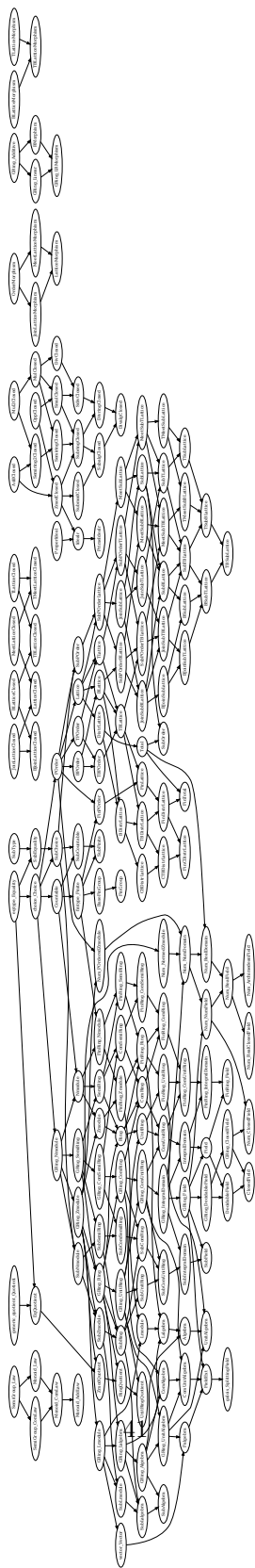



Figure 3.1: The hierarchy of mathematical structures of MATHCOMP as of 2023-05-10 (MATHCOMP 2)

The structure `per se` (and its type) is declared in a way that is reminiscent of sigma-types (§ 2.8):

```
#[short(type=structType)]
HB.structure Definition Struct := {carrier of isStruct carrier}
```

HIERARCHY-BUILDER is using ROCQ *attributes* (`#[...]`) to declare the type corresponding to a structure. See the ROCQ reference manual for more information about ROCQ attributes, although there is not much to know about them for our purpose.

Here is the pattern to declare a new structure `NewStruct` that extends an existing structure `Struct`; note the `of` syntax.

```
HB.mixin Record Struct_isNewStruct params carrier
  of Struct params carrier := {
    ... more properties about the carrier ...
  }
```

In the case of the extended structure, the structure definition makes appear the dependency to the parent structure.

```
#[short(type=newStructType)]
HB.structure NewStruct params :=
  {carrier of Struct_isNewStruct params carrier
    & Struct params carrier}.
```

This process results in the creation of the types `structType` and `newStructType` such that elements of the latter are also understood to be elements of the former.

3.5.1 `ssrbool.v`: Boolean Reasoning

The file `ssrbool.v` comes with the ROCQ proof assistant. It contains definitions and lemmas about boolean numbers that we already discussed. See also Appendix A.

To blur the difference between `Prop` and `bool`, `ssrbool.v` contains a *coercion* `is_true` that the user should be aware of:

```
(* Definition is_true b := b = true. defined in Init/Datatypes.v *)
Coercion is_true : bool >-> Sortclass.
```

The effect of that command is that instead of printing `is_true b`, i.e., `b = true` when `b` is a boolean number (actually, anything that has the type of a boolean number), ROCQ displays simply `b`. Because of the heavy use of boolean numbers with MATHCOMP, this makes for clearer statements and goals, but it might surprise you from time to time, so it is good to keep it in mind. In case of doubt, use

```
Set Printing Coercions.
```

to see the coercions.

A boolean statement `b` can be rewritten using `rewrite` since it is actually `b = true` thanks to the `is_true` coercion.

The file `ssrbool.v` also defines the type of boolean predicates

Definition `pred T := T -> bool.`

and a few boolean predicates such as the predicate that is always true:

Notation `xpredT := (fun => true).`

Because of the pervasive use of `if ... then .... else ...`, there is a couple of useful lemmas about branching. The lemma `ifT` rewrites `if b then t1 else t2` into `t1` and generates `b` as a subgoal. Similarly for `ifF`. To perform a case analysis on a goal that contains a branching expression, one can do:

case: `ifPn.`

The lemma `ifPn` is often more appropriate than using `ifP`.

The idiom to do a case analysis on an arbitrary boolean expression `b` is:

have `[] := boolP b.`

The implementation of `boolP` is similar to the one of the `reflect` predicate of § 2.9.

There is a family of contraposition lemmas that are very useful, such as

```
contra  : forall [c b : bool], (c -> b) -> ~~ b -> ~~ c
contraTN: forall [c b : bool], (c -> ~~ b) -> b -> ~~ c
contraTT: forall [c b : bool], (~~ c -> ~~ b) -> b -> c
```

Given this partial list, `contraNT` should be easy to guess. Search for "`contra`" to look for an appropriate contraposition lemma when in need.

3.5.2 eqtype.v: Decidable Equality

This file contains the most basic mathematical structure in `MATHCOMP`. It introduces the type `eqType` of types with a decidable equality, i.e., types that can be related with the boolean equality `==`. To find the interface of `eqType`, look for `mixin` in `eqtype.v` (`eqtype.v` on github):

Definition `eq_axiom T (e : rel T) := forall x y, reflect (x = y) (e x y).`

`HB.mixin Record hasDecEq T := { eq_op : rel T; eqP : eq_axiom eq_op }.`

```
#[mathcomp(axiom="eq_axiom"), short(type="eqType")]
HB.structure Definition Equality := { T of hasDecEq T }.
```

So, an `eqType` is a type that has an equality relation `==` that satisfies `axiom` which is a `reflect` relation (§ 2.9).

Natural numbers and boolean numbers are declared to be `eqTypes` by providing instances, so that one can check:

```

Check 0 == 1.
(* 0 == 1 : bool *)
Check true == false.
(* true == false : bool *)

```

This is actually the same as `@eq_op _ 0 1` and `@eq_op _ true false` but ROCQ fills the placeholder with the right instance. See [Mahboubi and Tassi, 2021, § 6.3–6.4] for more details.

The lemma `eqVneq` from ^{MATHCOMP}`eqtype.v` is useful to do case analysis in the course of proofs with the idiom:

```

have [->|ab] := eqVneq a b.

```

In the first subgoal, `a` is replaced by `b`, in the second subgoal, the local context now contains the hypothesis `ab : a != b`. As a side effect, occurrences of `a == b` and `b == a` are replaced by their truth values.

Case analysis with `eqVneq` is possible this way because it is specified using an inductive predicate with two constructors:

```

Variant eq_xor_neq (T : eqType) (x y : T) : bool -> bool -> Set :=
| EqNotNeq of x = y : eq_xor_neq x y true true
| NeqNotEq of x != y : eq_xor_neq x y false false.

```

```

Lemma eqVneq (T : eqType) (x y : T) : eq_xor_neq x y (y == x) (x == y).

```

This way of providing case analysis is very common in MATHCOMP, this is actually similar to the `reflect` relation (§ 2.9).

3.5.3 ssrnat.v: Natural Numbers

The file ^{MATHCOMP}`ssrnat.v` contains the theory of natural numbers. The proofs there are short (many one-liners) and it is a good place to learn and appreciate SSREFLECT tactics. See also Appendix A.

This file does not redefine totally the operations on natural numbers. For example, subtraction and multiplication are coming from ^{ROCQ}`Init/Nat.v`.

The comparison operations are boolean functions, i.e., they have type `bool`, not `Prop`. In the past, relations were given the type `Prop` in the standard library of ROCQ but this turned out to be inconvenient in practice. For example, large inequality is formalized as follows:

```

Definition leq m n := m - n == 0.

```

This is the notation `m <= n`. The notation `m < n` is really just `m.+1 <= n`. It might be surprising at first to define inequalities in such a convoluted way, but this enables more sharing of proofs and contributes to simplify the theory.

There is a number of useful lemmas that have been designed to make case analysis more efficient. E.g.:

```

Variant leq_xor_gtn m n : nat -> nat -> nat -> nat -> bool -> bool -> Set :=
| LeqNotGtn of m <= n : leq_xor_gtn m n m m n n true false
| GtnNotLeq of n < m : leq_xor_gtn m n n n m m false true.

```

```

Lemma leqP m n : leq_xor_gtn m n (minn n m) (minn m n) (maxn n m) (maxn m n)
(m <= n) (n < m).

```

Case analysis on `leqP m n` will generate two subgoals. In the first one, `m <= n` is true and `n < m` is false. In the second one, `m <= n` is false and `n < m` is true. Occurrences of `m <= n` and `n < m` are replaced by their truth values. In addition, `maxn m n`, etc., are also replaced by the adequate values.

Exercise 3.5.1. Prove `leqP`.

Other interesting contents: the notation `. *2` we already saw, the theory of the predicate `odd`, etc.

Related files: Properties about division in ^{MATHCOMP}`div.v` (`m %/ d, m %% d`: quotient and remainder of euclidean division) and prime numbers in ^{MATHCOMP}`prime.v` (`prime p`: `p` is a prime number)

Strong Induction Recent versions of MATHCOMP suggest to use the predicate `ubnP` to perform proofs by strong induction:

```

Check ubnP.
(* ubnP : forall m : nat, {n : nat / m < n} *)

```

The lemma `ubnP` is using the type `sig` from § 2.8.

.../...

Example of proof by strong induction:

```
Fixpoint a n :=
  match n with
  | 0 => 2
  | 1 => 4
  | n'.+1 => a n' + (a n'-.1).*2
  end.
```

Lemma ha n : $2^n \leq a\ n$.

Proof.

have [m nm] := ubnP n.

```
n, m : nat
nm : n < m
-----
2 ^ n <= a n
```

```
elim: m => // m ih in n nm *.
(* this is the same as move: m n nm; elim=> // m ih n nm. *)
```

```
m : nat
ih : forall n : nat, n < m -> 2 ^ n <= a n
n : nat
nm : n < m.+1
-----
2 ^ n <= a n
```

Exercise 3.5.2. Complete the proof of `forall n,  $2^n \leq a\ n$` .

3.5.4 fintype.v: Finite Types

The file `fintype.v` contains a theory of finite types, i.e., types with a finite number of inhabitants. The example of interest for us is the type of so-called *ordinals* `'I_n` where `n` is a natural number (and `'I_` is a notation). It is the type of the natural numbers $\{0, \dots, n-1\}$.

Though conceptually simple, ordinals can be painful to manipulate and should therefore not be used without a good reason. They are however important for iterated operations (see § 3.5.7). In particular, for the type `'I_n.+1` (note the `.+1`), `ord0` is 0 and `ord_max` is `n`. Ordinals are automatically turned into natural numbers if needed thanks to the coercion `nat_of_ord` (that is therefore hidden to the user by default). Turning a natural number into an ordinal requires a bit of help from the user:

```
From mathcomp Require Import fintype.
```

```
Fail Check 0 : 'I_3.
```

```
Check inord 0 : 'I_3.
```

```
About inord.
```

```
(* inord : forall {n' : nat}, nat -> 'I_n'.+1 *)
```

3.5.5 seq.v: Lists

This is recap of lists notations (in scope `seq_scope`) and of standard functions about lists:

- **Variables** $(T1\ T2 : \text{Type})\ (f : T1 \rightarrow T2)$.
Fixpoint `map s := if s is x :: s' then f x :: map s' else [::]`.
- Notation: `[seq f i | i <- s]`
- **Variable** `a : pred T`.
Fixpoint `filter s := if s is x :: s' then if a x then x :: filter s' else filter s' else [::]`.
- Notation: `[seq i <- s | a i]`

- **Variables** $(T : \text{Type})\ (R : \text{Type})\ (f : T \rightarrow R \rightarrow R)\ (z0 : R)$.
Fixpoint `foldr s := if s is x :: s' then f x (foldr s') else z0`.

`foldr f z0 [:: a; b; c]` is `f a (f b (f c z0 ))`

- **Fixpoint** `iota m n := if n is n'.+1 then m :: iota m.+1 n' else [::]`.

So `iota m n` is the list `[:: m; m + 1; ...; m + n - 1]`.

Extended explanations about lists can be found in [Mahboubi and Tassi, 2021, § 1.3.1].

3.5.6 order.v: Ordered Types

There are several ordered types, most notably `porderType` and `orderType`. See MATHCOMP `order.v`, beware: this is a huge file (try looking at Fig. 3.1). Theories are organized in modules, so that, for example, to enjoy the lemmas about totally ordered types, the development should start with `Import Order.Ttheory`.

Transitivity lemmas are particularly useful. Given a goal of the form:

$a < K$

`rewrite (@le_lt_trans _ _ M)` // generates two goals:

$a \leq M$

and

$M < K$

Similarly for `lt_le_trans`, `le_trans`, `lt_trans`. They are (of course) often used in practice.

Similarly to `leqP` (§ 3.5.3) for natural numbers, we can do case analysis with any ordered type using:

`have [] := leP a b.`

Although there is a bridge to treat natural numbers as an ordered type, the former have been kept separated because what is gained by generalization is not obvious, hence the apparent duplication of lemmas.

3.5.7 IMPORTANT `bigop.v`: Iterated Operations

Iterated operations is the formalization of notations such as Σ_i , Π_i , $\cup_i$, etc. They were introduced in MATHCOMP by [Bertot et al., 2008] and were instrumental to complete the formal proof of the Odd Order Theorem [Gonthier et al., 2013].

The first lines of Table 3.8 introduce the generic notation. The implementation `\big[op/idx]_ (i <- s | P i) F i` of iterated operations is essentially a wrapper for a `foldr` (§ 3.5.5) of a function `F` that represents each term (of the sum, product, etc.) along a list `s`, filtered by a boolean predicate `P` (§ 3.5.1).

It is important to understand that in Table 3.8, in `i < n`, `i` is an ordinal (§ 3.5.4). In contrast, the enumeration `m <= i < n` is implemented by the `iota` function as `iota m (n - m)` (§ 3.5.5), so there `i` is a natural number.

From the user point of view, the lemmas listed in Appendix A are maybe among the most useful ones (e.g., `big1`, `eq_bigr`).

From the developer point of view, it is sometimes useful to use more technical lemmas, that reveal a bit about the formalization of iterated operations. For example, `big_mkcond` is sometimes useful to get rid (temporarily) of the filtering predicate by putting it into the function body:

```
Lemma big_mkcond I r (P : pred I) F :
  \big[*M/1]_ (i <- r | P i) F i =
  \big[*M/1]_ (i <- r) (if P i then F i else 1).
```

Similarly, `big_seq` “duplicates” the enumeration list as a predicate so that it can be available to prove properties of `F` if needed:

```
Lemma big_seq (I : eqType) (r : seq I) F :
  \big[op/idx]_ (i <- r) F i = \big[op/idx]_ (i <- r | i \in r) F i.
```

There is a number of generic lemmas that are useful for proving properties of iterated operations. For example, to prove a property for an iterated operation knowing it is true for each element, one can use `elim/big_ind : _ => //` (this is the generic syntax for case analysis that we mentioned in § 2.4.2) where `big_ind` is:

```
big_ind : K idx ->
  (forall x y : R, K x -> K y -> K (op x y)) ->
  forall (I : Type) (r : seq I) (P : pred I) (F : I -> R),
  (forall i : I, P i -> K (F i)) -> K (\big[op/idx]_ (i <- r | P i) F i)
```

Reminder: To do the following exercises, you should go to ^{MATHCOMP}`bigop.v` and copy the header to a new file (and append `From mathcomp Require Import bigop.`)

Exercise 3.5.3. Prove `forall n, 2 * (\sum_(0 <= x < n.+1) x) = n * n.+1`. (`\sum_` is a notation for `\big[addn/0]_`)

Exercise 3.5.4. Prove

```
forall n : nat, \sum_(0 <= x < n.+1) (x + x) = 2 * \sum_(0 <= x < n.+1) x
using big_ind2.
```

Exercise 3.5.5. Prove $1 + 2 + \dots + 2^n = 2^{n+1} - 1$

Exercise 3.5.6. Prove

```
forall n, (6 * \sum_(k < n.+1) k ^ 2) = n * n.+1 * (n.*2).+1.
```

Exercise 3.5.7. Prove

```
forall (x n : nat) :
  1 < x -> (x - 1) * (\sum_(k < n.+1) x ^ k) = x ^ n.+1 - 1
```

Iterated operations are generic. It suffices for the operation to meet some requirements to enjoy a particular lemma. For example, provided that the carrier with the operation `op` forms a monoid ^{MATHCOMP}(`bigop.v`)

```
(* in Module Monoid *)
HB.factory Record isLaw T (idm : T) (op : T -> T -> T) := {
  opA : associative op;
  op1m : left_id idm op;
  opm1 : right_id idm op;
}.
```

the lemmas `big1`, `big_nat_recr` become available (see Appendix A). This is because `addn`, the addition of natural numbers, has been shown to form a monoid that one can use these lemmas. More precisely, `addn` has been shown to form a commutative monoid

```
HB.instance Definition _ := isComLaw.Build nat 0 addn addnA addnC addOn.
```

from which the monoid structure is automatically derived by `HIERARCHY-BUILDER`.

Searching lemmas about iterated operations is notoriously difficult. It is maybe better to look for the most generic notation

```
Search (\big[_/_]_(i <- _ | _) _).
```

or for lemmas with the substring "big" while restricting the search with `in bigop`.

The notation `\big[op/idx]_(i \in D) f i` (Table 3.8) assumes that `f` takes a finite number of values (note the `\in` notation). This is useful as a notation because it allows to write sums like `\sum_(x \in [set: R]) f x` when the support of `f` is finite, as we will do in § 6.2.1 to define integration. The definition is a bit technical, see file ^{MATHCOMP-ANALYSIS}`fsbig.v`.

Finitely iterated operations:	
$\text{\texttt{\big[op/idx]_ (i <- s P i) f i}}$	$\overline{\text{op}}_{i < s , i \in P} f(s_i)$
$\text{\texttt{\big[op/idx]_ (i < n P i) f i}}$	$\overline{\text{op}}_{0 \leq i < n, i \in P} f(i)$
$\text{\texttt{\big[op/idx]_ (m <= i < n P i) f i}}$	$\overline{\text{op}}_{m \leq i < n, i \in P} f(i)$
Application to numeric functions:	
$\text{\texttt{\sum_ (i <- s P i) f i}}$	$\sum_{i < s , i \in P} f(s_i)$
Iterated operations over finite support:	
$\text{\texttt{\big[op/idx]_ (i \text{ \textcolor{teal}{in}} D) f i}}$	$\overline{\text{op}}_{i \in D} f(i)$ if $f(i)$ has a finite number of values in D s.t. $f(i) \neq \text{idx}$ o.w. idx
Countably iterated sum of numeric functions (see § 4.8):	
$\text{\texttt{\sum_ (i <oo P i) f i}}$	$\sum_{i=0, i \in P}^{\infty} f(i)$
$\text{\texttt{\sum_ (m <= i <oo P i) f i}}$	$\sum_{i=m, i \in P}^{\infty} f(i)$
Sum of extended real numbers over general sets (see § 5.1):	
$\text{\texttt{\esum_ (i \text{ \textcolor{teal}{in}} P) f i}}$	$\sum_{i \in P} f(i)$

Table 3.8: Summary of iterated operations. The symbol $\overline{\text{op}}$ is the iterated operation corresponding to op .

The `under` Tactic With iterated operators the need to use `rewrite` below λ -abstractions becomes more pressing. The `under` tactic can be used for that purpose [Martin-Dorel and Tassi, 2019]. A common usage with iterated operators is `under eq_bigr do rewrite ...`, with series `under eq_eseriesr do rewrite ...`.

3.5.8 About Finite Sets

MATHCOMP comes with a theory of finite sets *over finite types* ($\text{\texttt{finset.v}}$)^{MATHCOMP}. It is moderately useful¹ outside of MATHCOMP, which has been designed originally to develop the theory of finite groups. See also Appendix A.

The `finmap` library [Cohen and Sakaguchi, 2015] provides an alternative where the carrier type only needs to be a `choiceType`, intuitively a type equipped with a form of the axiom of choice (which extends `eqType` in the hierarchy of mathematical structures in MATHCOMP). This is less restrictive and useful in MATHCOMP-ANALYSIS (for example to define countable sums, see § 5.1).

3.6 Mathematical Structures in algebra

3.6.1 `ssralg.v`: Algebraic Structures

Most algebraic mathematical structures can be found in $\text{\texttt{ssralg.v}}$ ^{MATHCOMP}. They can also be found in Fig. 3.1. Let us just mention the most important ones.

¹except maybe for the formalization of finite distributions in [Infotheo, 2024]...

- `zmodType` for abelian groups. It provides one constant (`0`), one unary operation (`-%R`), one binary operation (`+%R`) (all in `ring_scope`, see Table 3.1), and the axioms of an abelian group (`addrA`, `addrC`, `add0r`, `addNr`). They are in the module `GRing` so to use them one often starts its development with `Import GRing.Theory`. (Otherwise, identifiers should be fully qualified.)
- `ringType`: rings, provides one constant (`1`), one binary operation (`*%R`), and the axioms of a ring (`mulrA`, `mul1r`, `mulr1`, `mulrDl`, `mulrDr`, `oner_neq0` for $1 \neq 0$, which means that the trivial ring is excluded).
- `comRingType`: commutative rings, adds `mulrC`
- `lmodType R`: left modules over `R` which have the following mixin:

```
HB.mixin Record Zmodule_isLmodule (R : ringType) V of Zmodule V := {
  scale : R -> V -> V;
  scalerA : forall a b v, scale a (scale b v) = scale (a * b) v;
  scale1r : left_id 1 scale;
  scalerDr : right_distributive scale +%R;
  scalerDl : forall v, {morph scale^~ v: a b / a + b}
}.
#[short(type="lmodType")]
HB.structure Definition Lmodule (R : ringType) :=
  {M of Zmodule M & Zmodule_isLmodule R M}.
```

The contents of this mixin should be entirely readable since we have explained in previous sections all the syntax. The notation for the scaling operation is `*.`. Properties are available as the lemmas `scalerA`, `scale1r`, `scalerDr`, `scalerDl`. Left modules will be used to define normed modules in `MATHCOMP-ANALYSIS` (§ 4.6.5).

- `idomainType`: integral domains, with the axiom
- ```
forall x y : R, x * y = 0 -> (x == 0) || (y == 0)
```
- `fieldType`: fields. Note that the units of a field (more generally of a `unitRingType`) are available though the predicate `unit` (which comes with the `unitRingType`). The properties of units are recovered via lemmas such as:

```
Variable F : fieldType.
Lemma unitfE x : (x \in unit) = (x != 0).
```

Needless to say, the properties of units will be useful to deal with real numbers in `MATHCOMP-ANALYSIS`.

### 3.6.2 poly.v: Polynomials

The file `poly.v`<sup>MATHCOMP</sup> defines polynomial. A polynomial with coefficients in `R` has type `{poly R}`. The formal derivative of the univariate polynomial `p` is noted

$p^{\sim}()$  (in notation scope `ring_scope`). Note that this notation is reserved in the MATHCOMP file `ssrnotations.v`.

### 3.6.3 `ssrnum.v`: Numeric Types

The file MATHCOMP `ssrnum.v` provides mathematical structures for numeric types.

The type `normedZmodType` (file MATHCOMP `numdomain.v`) combines a norm (notation  $\|\cdot\|$ ) with an abelian group.

The basic numeric type is `numDomainType` (file MATHCOMP `numdomain.v`). It combines an integral domain, a partially ordered abelian group, a `normedZmodType`, and properties specific to numeric types.

The formalization of these two mathematical structures is a bit technical. To smoothly use the definitions and lemmas from MATHCOMP `ssrnum.v`, developments usually start with

```
Import Num.Def Num.Theory.
```

The type `realDomainType` (file MATHCOMP `numdomain.v`) extends `numDomainType` by considering the order as total.

The type `numFieldType` (file MATHCOMP `numfield.v`) extends `numDomainType` by considering its elements as invertible.

The type `realFieldType` (file MATHCOMP `numfield.v`) extends `numFieldType` by considering its elements as invertible.

The type `rcfType` (file MATHCOMP `numfield.v`) extends `realFieldType` with the property of real closure.

*Exercise 3.6.1.* Show that  $\sqrt{4 + 2\sqrt{3}} = 1 + \sqrt{3}$ . Look at `sqrt` in MATHCOMP `numfield.v`.

**Integers** The numeric type of relative integers is provided by the file MATHCOMP `ssrint.v`. We can inject a natural number `n` to an integer using the notation `n%:Z`.

**Summary About Numeric Types** There are several numeric types in MATHCOMP and going from one to another might sometimes feel painful. That actually seems to be a common difficulty in proof assistants based on type theory [Harrison, 2018]. Indeed, on paper, we take the following inclusions for granted:

$$\mathbb{N} \subset \mathbb{Z} \subset \mathbb{Q} \subset \mathbb{R} \subset \overline{\mathbb{R}}$$

Figure 3.2 illustrates some ways to go between numeric types used in MATHCOMP-ANALYSIS.

### 3.6.4 `archimedean.v`: floor and ceil

The type `archiRealFieldType` (file MATHCOMP `archimedean.v`) extends `realFieldType` with a floor operation and a ceil operation. These operations return integers (Sect. 3.6.3).

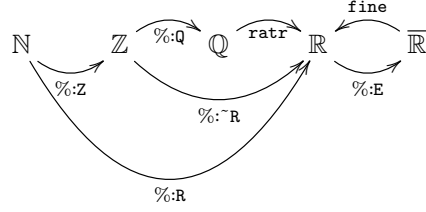


Figure 3.2: Some conversions between numeric types. See § 5.1 for extended real numbers.

### 3.6.5 interval.v: Intervals

In MATHCOMP, the type of intervals is `interval R`, where `R` is typically an ordered type (§ 3.5.6). The ASCII notation uses square brackets. An interval is defined as the pairs of bounds of type `itv_bound`:

```

Variant itv_bound (T : Type) : Type :=
 BSide : bool -> T -> itv_bound T | BInfty : bool -> itv_bound T.
Variant interval (T : Type) := Interval of itv_bound T & itv_bound T.

```

The constructor `BSide` is for open or closed bounds, `BInfty` is for infinite bounds. How the boolean parameter distinguishes between open and closed bounds is better explained with illustrations.

For example, the left bounds of the intervals ``[x, +oo[` and ``]x, +oo[` are respectively `BSide true x` and `BSide false x`, while the right bound of the interval ``]-oo, x[` is `BSide true x`. This type allows for the statement of generic lemmas about intervals, when they happen to hold independently of whether a bound is open or closed.



## Chapter 4

# Classical Reasoning using MathComp

*Goal of this chapter:* This chapter introduces the basics of the formalization of analysis in `MATHCOMP-ANALYSIS`. It covers material that can be found in [Affeldt et al., 2018, Rouhling, 2019, Affeldt et al., 2020a].

The `MATHCOMP-ANALYSIS` [Affeldt et al., 2017] library contains several directories, one of them is `classical`. The contents of `classical` is generic, it essentially develops classical reasoning on top of `MATHCOMP`. This is the purpose of the first part of this chapter. The rest of this chapter deals with the topic of convergence, which spans the files `filter.v`, `topology_structure.v`, `uniform_structure.v`, `separation_axioms.v`, `normedtype.v`, and `sequences.v` from the directories `classical` and `theories`.

### 4.1 Axioms Introduced by MathComp-Analysis

`MATHCOMP` is constructive: it relies solely on `ROCQ`, it does not rely on classical reasoning. `MATHCOMP-ANALYSIS` starts by giving up on constructivism by adding a bunch of axioms, which are known to be compatible with the logic of `ROCQ`.

One of the motivation is to be able to do set-theoretic reasoning and to further blur the difference between `Prop` and `bool`.

#### 4.1.1 Propositional Extensionality

The equivalence between two propositions implies their equality:

```
Axiom propositional_extensionality :
 forall P Q : Prop, P <-> Q -> P = Q.
```

*Exercise 4.1.1.* Prove `True = true`. Similarly, prove `true = True`.

### 4.1.2 Functional Extensionality

Pointwise equality of functions implies their equality. This is stated more generally for functions with dependent types `forall x : A, B x` (instead of the less general `A → B` type, remember § 2.1.1).

```
Axiom functional_extensionality_dep :
 forall (A : Type) (B : A → Type) (f g : forall x : A, B x),
 (forall x : A, f x = g x) → f = g.
```

Here is an alternative representation of functional extensionality:

```
Lemma eq_fun T rT (U V : T → rT) :
 (forall x : T, U x = V x) → (fun x => U x) = (fun x => V x).
```

It can be used together with the `under` tactic (§ 3.5.7) to do rewriting under  $\lambda$ -abstractions as in `under eq_fun do rewrite ....`

### 4.1.3 Constructive Indefinite Description

The `Prop`-valued existential quantifier implies the `Type`-valued one (see § 2.8).

```
Axiom constructive_indefinite_description :
 forall (A : Type) (P : A → Prop),
 (exists x : A, P x) → {x : A | P x}.
```

The existential on the left is in `Prop`, the one on the right is the one in `Type` (as we saw in § 2.8).

### 4.1.4 Consequences of Classical Axioms

We can derive a version of the axiom of choice which is very useful:

```
Lemma choice X Y (P : X → Y → Prop) :
 (forall x, exists y, P x y) → {f & forall x, P x (f x)}.
```

We can derive the law of the excluded middle:

```
Lemma pselect (P : Prop) : {P} + {¬P}.
```

Recall that the notation `{ ... } + { ... }` is for a `Set`-version of the disjunction (§ 2.7).

We can turn a proposition in `Prop` into a boolean number:

```
Definition asbool (P : Prop) :=
 if pselect P then true else false.
```

`[< P >]` is a notation for `asbool P`.

Contraposition lemmas that are true classically become provable, e.g.:

```
Lemma contra_notP (Q P : Prop) : (¬ Q → P) → ¬ P → Q.
```

can be found in <sup>MATHCOMP-ANALYSIS</sup> `boolp.v` whereas

```
Lemma contraPnot (P Q : Prop) : (Q → ¬ P) → (P → ¬ Q).
```

was already in <sup>ROCQ</sup> `ssrbool.v` (§ 3.5.1)



## 4.2 Naive Set Theory

We now define a theory of sets which are not necessarily finite. This comes as an addition to § 3.5.8 and the naming actually overlaps.

### 4.2.1 Basic Set-theoretic Operations

In MATHCOMP-ANALYSIS, a set is formalized as a (characteristic) function from some type  $T$  to  $\text{Prop}$  (see § 2.1.1):

**Definition**  $\text{set } T := T \rightarrow \text{Prop}$ .

The identifier  $\text{set0}$  represents the empty set (the function that returns  $\text{False}$ ) and  $\text{setT}$  is the full set (the function that returns  $\text{True}$ , notation  $[\text{set} : T]$  where  $T$  is the support type). Any  $\text{Prop}$ -valued function  $P$  gives rise to a set using the notation  $[\text{set } x \mid P]$  to define a set by comprehension. The notation scope is  $\text{classical\_set\_scope}$ , the delimiter is  $\text{classic}$ .

Given an element  $x : T$ , we can write  $x \setminus \text{in } A$  for a set  $A$ , this is a  $\text{bool}$  expression. Of course, this is equivalent to  $A \ x$ , which is in  $\text{Prop}$ . Rewriting with  $\text{inE}$  turns  $x \setminus \text{in } A$  expression into a function application  $A \ x$ . Similarly, the lemma  $\text{mem\_set}$  allows to move from  $A \ x$  to  $x \setminus \text{in } A$  (and back with the lemma  $\text{set\_mem}$ ). One can use either  $x \setminus \text{in } A$  or  $A \ x$  to state that an element belongs to a set.

Basic operations on sets can be formalized using basic logic operators (see § 2.7). Intersection is essentially conjunction:

**Definition**  $\text{setI } A \ B := [\text{set } x \mid A \ x \wedge B \ x]$ .

One can use the notation  $A \ \&\` B$  instead of  $\text{setI } A \ B$ , this is a notation in the scope  $\text{classical\_set\_scope}$ .

Union is essentially disjunction (notation:  $A \ \mid\` B$ ):

**Definition**  $\text{setU } A \ B := [\text{set } x \mid A \ x \vee B \ x]$ .

Complement (notation:  $\sim\` A$ ):

**Definition**  $\text{setC } A := [\text{set } a \mid \sim A \ a]$ .

Subset relation (notation:  $A \ \leq\` B$ ):

**Definition**  $\text{subset } A \ B := \text{forall } t, A \ t \rightarrow B \ t$ .

*Exercise 4.2.1.* State and prove De Morgan's laws.

*Exercise 4.2.2.* Find De Morgan's laws in  $\text{classical\_sets.v}$ .

*Exercise 4.2.3.* Given a type  $T$ , show that  $\text{set } T$  with inclusion is a *poset* (reflexivity and transitivity). Show that  $\text{set } T$  with containment is a poset.

### 4.2.2 More Set-theoretic Constructs

The preimage of the set  $A$  by the function  $f$  is noted  $f^{-1} A$  (yes, you can get used to this notation). This is a notation for `[set t | A (f t)]`.

The iterated operations of core MATHCOMP are finite (§ 3.5.7). With classical sets, we can deal with countable iterated unions:

**Definition** `bigcup T I (P : set I) (F : I -> set T) :=  
[set a | exists2 i, P i & F i a]`.

Notations are similar to the ones for finite sets, that is: `\bigcup_(i in P) F`, `\bigcup_(i : T) F`, `\bigcup_(i < n) F`, etc. And similarly for countable iterated intersections `\bigcap_(i in P) F`, etc.

In measure theory in particular, there is a pervasive use of families of pairwise disjoint sets. `trivIset D F` is a predicate stating that the family of sets  $F$  indexed by  $D$  is made of pairwise disjoint sets:

**Definition** `trivIset T I (D : set I) (F : I -> set T) :=  
forall i j : I, D i -> D j -> F i ~&~ F j != set0 -> i = j`.

There is an operation to pick a particular element from an arbitrary set defined by comprehension: `xget x0 P` returns an element of the set  $P$  or `x0` if  $P$  is empty.

There is also a predicate `finite_set` defined in the file `MATHCOMP-ANALYSIS` `cardinality.v` that discriminates sets that are actually finite. It uses a relation `#=` for the cardinality of sets.

## 4.3 Supremum and Infimum

Using classical sets (§ 4.2) and ordered types (§ 3.5.6), we can develop a theory for suprema and infima.

The set of upper bounds of a set  $A$  is formed by the elements  $y$  such that  $\forall x \in A, y \geq x$ :

**Definition** `ubound A : set T := [set y | forall x, A x -> (x <= y)%0]`.

Similarly for the set of lower bounds:

**Definition** `lbound A : set T := [set y | forall x, A x -> (y <= x)%0]`.

A *supremum* is an upper bound that is less than or equal to any other upper bound:

**Definition** `supremums A := ubound A ~&~ lbound (ubound A)`.

We call supremum an element of `supremums`:

**Definition** `supremum x0 A :=  
if A == set0 then x0 else xget x0 (supremums A)`.

(`xget` is defined in § 4.2)



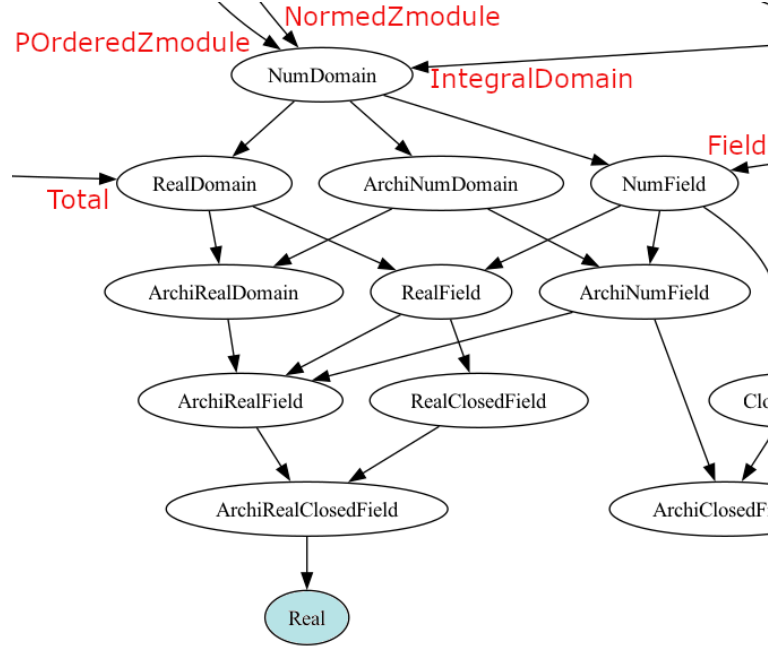


Figure 4.2: Hierarchy of mathematical structures to defined real numbers

- a real-closed field from MATHCOMP (`rcfType` in § 3.6.3), and
- properties of the `sup`:
  1. for any non-empty set  $E$  with an upper bound, `supremum 0 E` (Sect. 4.3) is an upper bound (`supremum 0` is in fact noted `sup`)
  2. for any non-empty set  $E$  with an upper bound, for any  $\varepsilon > 0$ , there is an element  $e \in E$  such that `sup E` -  $\varepsilon < e$  (the property of “adherence”).

This is compatible with the real numbers of ROCQ’s standard library because the latter form a model of the above interface (this model is provided by MATHCOMP-ANALYSIS).

Look for the mixin `ArchimedeanField_isReal` in `MATHCOMP-ANALYSIS` `reals.v`.

To construct a real number from a natural number  $n$ , one can write `n%:R`.

*Exercise 4.4.1.* Let  $d$  is a semimetric ( $d(x, x) = 0$ ,  $d(x, y) = d(y, x) \geq 0$ ,  $d(x, z) \leq d(x, y) + d(y, z)$ ). Show that  $\frac{d}{1+d}$  is a semimetric.

## 4.5 Convergence

### 4.5.1 Filters

Convergence in MATHCOMP-ANALYSIS is expressed using *filters*<sup>1</sup>. The notion of filter was introduced by Henri Cartan:

Ça n'avait pas de nom naturellement, cette notion que je venais de trouver, alors, pour se convaincre que ça marchait, on prenait des exemples, et puis au moment où l'instrument arrivait, on disait : “Boum !” Alors on a appelé ça les “boums” ! Évidemment ça ne pouvait pas rester longtemps les boums, et surtout s'il fallait publier le résultat.  
(Henri Cartan, [Broué, 2012])

The axioms of a filter  $F$  of type `set_system T` (i.e., a set of sets of elements of type  $T$ ) are defined as a typeclass called `Filter` in `MATHCOMP-ANALYSIS filter.v`:

1. The full set belongs to  $F$  (`filterT`).
2.  $F$  is closed by (binary) intersection (`filterI`).
3.  $F$  is closed by containment:

```
filterS : forall P Q : set T, P `<= ` Q -> F P -> F Q
```

The empty set can belong to a filter in MATHCOMP-ANALYSIS; this is the type of *proper filter* that excludes the empty set. See the typeclass `ProperFilter` in `MATHCOMP-ANALYSIS filter.v`. We thus recover the standard definition of filter [Bourbaki, 1971, Chap. I][Wilansky, 2008, § 3.2].

Given a family of sets  $B : I \rightarrow \text{set } T$  indexed by a set  $D : \text{set } I$ , the expression `filter_from D B` is the set of sets  $P$  that are supersets of  $B$  i's:

```
Definition filter_from
 {I T : Type} (D : set I) (B : I -> set T) : set_system T :=
 [set P | exists2 i, D i & B i `<= ` P].
```

The expression `filter_from setT B` forms a filter if  $I$  is not empty and if

```
forall i j, exists k, B k `<= ` B i `& ` B j
```

See lemma `filter_fromT_filter` in `MATHCOMP-ANALYSIS filter.v`. This corresponds to the definition of a *filterbase* [Bourbaki, 1971, Chap. 1][Wilansky, 2008, § 3.2].

*Example 4.5.1.* The filter based on the sets  $\{n \mid N \leq n\}$  for all natural numbers  $N$  (i.e., the “intervals”  $[N, +\infty[$ ) is the “eventually filter”  $\backslash\infty$  to talk about the behavior of sequences when the index tends to infinity. This is a notation in the scope `classical_set_scope` and it is defined as follows:

```
filter_from setT (fun N => [set n | (N <= n)%N])
```

<sup>1</sup>Nothing to do with the `filter` function of lists from `MATHCOMP seq.v`.

See `eventually` and `eventually_filter` in `MATHCOMP-ANALYSIS filter.v`.

*Example 4.5.2.* The filter formed by the sets  $P$  such that there exists an  $M$  such that for all  $M < x$ ,  $x \in P$  is noted `+oo` (i.e., this filter contains all intervals  $]M, +\infty[$ ). See `pinfty_nbhs`, declared to be a proper filter via `proper_pinfty_nbhs`, in `MATHCOMP-ANALYSIS num_normedtype.v`.

See `normedtype.v` for the examples of filters corresponding to convergence to the right (`at_right`, notation  $\overset{+}{\cdot}$ ), to the left (`at_left`, notation  $\overset{-}{\cdot}$ ), etc.

## 4.5.2 Convergence using Filters

The notation for convergence in `MATHCOMP-ANALYSIS` is  $F \dashrightarrow G$  where  $F$  and  $G$  are filters. What is behind is just an inclusion, namely  $G \subseteq F$ .

We define the image of a filter  $F$  by a function  $f$  as the set of sets  $P : \text{set } U$  such that the preimage of  $P$  by  $f$  is in  $F$ :

**Definition** `fmap`  $\{T \ U : \text{Type}\} (f : T \rightarrow U)$   
 $(F : \text{set\_system } T) : \text{set\_system } U :=$   
 $[\text{set } P \mid F (f \text{ @ } \sim 1 \text{ } P)]$ .

The notation  $E \text{ @ } [x \dashrightarrow F]$  is the image of the filter  $F$  by the function `fun x => E`.  $E \text{ @ } [x \dashrightarrow F]$  is the same as  $E \text{ @ } F$ .

By combining the notation for convergence of filters and the notation for the image of a filter, we obtain a notation for convergence of functions (and of sequences):  $f \text{ @ } [x \dashrightarrow a] \dashrightarrow 1$ , standing for

$$f(x) \xrightarrow{x \rightarrow a} l.$$

We can already define continuity: a function  $f$  is *continuous* iff  $\forall a, f(x) \xrightarrow{x \rightarrow a} f(a)$ . See the notation `continuous` in `MATHCOMP-ANALYSIS filter.v`.

## 4.5.3 Filtered Types

There is a notion of *filtered type* for types whose points can each be equipped with a filter. Given a point  $x$  in some filtered type, `nbhs x` is the set of sets associated with this point. See the structure `Filtered` in `MATHCOMP-ANALYSIS filter.v`, the notation for filtered types is `filteredType`.

### Limits

Given a filter  $F$ ,  $\lim F$  is defined as being a  $1$  such that  $F \dashrightarrow 1$ . See `lim_in` in `MATHCOMP-ANALYSIS filter.v`. This definition uses the `get` operation seen in § 4.2. Since  $\lim F$  is a notation, so it should be `Searched` as “`(lim _)`” rather than “`lim`”.

It can be proved that if the support set is Hausdorff (i.e.,  $x \neq y$  can be separated by neighborhoods, see `hausdorff_space` in `MATHCOMP-ANALYSIS separation_axioms.v`), then the limit is unique, and we have:

**Lemma** `cvg_lim`  $\{U : \text{Type}\} \{F\} \{FF : \text{ProperFilter } F\} (f : U \rightarrow T) (l : T) :$   
 $f @ F \rightarrow l \rightarrow \lim (f @ F) = l.$

## 4.6 Other Structures in MathComp-Analysis

### 4.6.1 Topological Spaces

In MATHCOMP-ANALYSIS, the interface of a *topological space* is defined by the following mixin (file `topology_structure.v`). It extends the structure `Nbhs` which means that  $T$  is in particular equipped with a function `nbhs` that returns a set of sets.

```
HB.mixin Record Nbhs_isTopological (T : Type) of Nbhs T := {
 open : set_system T;
 nbhs_pfilter_subproof : forall p : T, ProperFilter (nbhs p) ;
 nbhsE_subproof : forall p : T, nbhs p =
 [set A : set T | exists B : set T, [/\ open B, B p & B `<=` A]] ;
 openE_subproof : open = [set A : set T | A `<=` nbhs ~ A]
}.
```

Recall that `set_system T` means `set (set T)`. A topological space is therefore given by a set of sets that corresponds to the *open sets*. The axiom `nbhs_pfilter_subproof` adds that `nbhs` returns (proper) filters (§ 4.5.3). The axiom `nbhsE_subproof` ensures that `nbhs` returns *neighborhoods*, i.e., given a point  $p$ , a set in `nbhs p` is a set that contains an open that contains  $p$ . The axiom `openE_subproof` is an alternative characterization of open sets: an open set is a neighborhood of all of its points, i.e., for all  $p$ ,  $A p$  implies `nbhs p A` when  $A$  is an open. This gives rise to the type `topologicalType`. All the axioms are named `xyz_subproof` but they are also available as lemmas `xyz` (this has been a customary naming scheme in ROCQ but it is likely to change).

The structure of topological spaces is defined by combining the structure `Nbhs` with the above mixin:

```
#[short(type="topologicalType")]
HB.structure Definition Topological :=
 {T of Nbhs T & Nbhs_isTopological T}.
```

The definition of topological space in MATHCOMP-ANALYSIS departs from the standard, textbook definition of topological space according to which a topological space is a set equipped with a set of subsets which is stable by union and by finite intersection, and that contains the empty set and the full set. The factory `isOpenTopological` provides a way to construct the above mixin from the standard, textbook axioms:

```
HB.factory Record isOpenTopological T of Choice T := {
 op : set T -> Prop;
 opT : op setT;
 opI : forall (A B : set T), op A -> op B -> op (A `&` B);
 op_bigU : forall (I : Type) (f : I -> set T), (forall i, op (f i)) ->
```

```

 op (\bigcup_i f i);
 }.

```

*Exercise 4.6.1.* Find the lemmas in `topology_structure.v` corresponding to the standard, textbook definition of topological spaces.

## 4.6.2 Uniform Spaces

Before defining uniform spaces, we define neighborhoods using *entourages*:

**Definition** `nbhs_ {T T'} (ent : set_system (T * T')) (x : T) :=`  
`filter_from ent (fun A => xsection A x).`

The expression `xsection A x` is for the  $x$ -section of  $A$ , i.e., `[set y | (x, y) \in A]`.

Like the default interface for topological spaces, the default interface of *uniform spaces* (file `uniform_structure.v`) extends the `Nbhs` structure:

```

HB.mixin Record Nbhs_isUniform_mixin M of Nbhs M := {
 entourage : set_system (M * M);
 entourage_filter : Filter entourage;
 entourage_diagonal_subproof :
 forall A, entourage A -> diagonal `<= A;
 entourage_inv_subproof : forall A, entourage A -> entourage A^-1;
 entourage_split_ex_subproof :
 forall A, entourage A -> exists2 B, entourage B & B \; B `<= A;
 nbhsE_subproof : nbhs = nbhs_ entourage;
}.

```

A uniform space is therefore given by a set of entourages `entourage`. The axiom `entourage_filter` tells that `entourage` is a filter. The axiom `entourage_diagonal_subproof` ensures that entourages include their *diagonal* (`[set x | x.1 = x.2]`), i.e., they are *connectors*. The axiom `entourage_inv_subproof` tells that the inverse entourage is an entourage. The axiom `entourage_split_ex_subproof` tells that entourages include the self-composition of an entourage. The axioms so far correspond precisely to the standard definition [Bourbaki, 1971, Chap. II]. The axiom `nbhsE_subproof` says that the notion of neighborhood using entourages is the same as the notion of neighborhoods from topological spaces.

The structure of uniform spaces extends a topological space with the above interface:

```

#[short(type="uniformType")]
HB.structure Definition Uniform :=
 {T of Topological T & Nbhs_isUniform_mixin T}.

```

## 4.6.3 Pseudometric Spaces

A *pseudometric space* extends a uniform space with a notion of ball (`ball x r` is a ball centered at  $x$  of radius  $r$ ) and three intuitive axioms (reflexivity `ball_center`, symmetry `ball_sym`, triangle inequality `ball_triangle`) (file `pseudometric_structure.v`):



```

HB.mixin Record Uniform_isPseudoMetric
 (R : numDomainType) M of Uniform M := {
 ball : M -> R -> M -> Prop ;
 ball_center_subproof : forall x (e : R), 0 < e -> ball x e x ;
 ball_sym_subproof : forall x y (e : R), ball x e y -> ball y e x ;
 ball_triangle_subproof :
 forall x y z e1 e2, ball x e1 y -> ball y e2 z -> ball x (e1 + e2) z;
 entourageE_subproof : entourage = entourage_ball
 }.

```

The axiom `entourageE_subproof` states that the notion of entourage using balls is the same as the notion of entourage from uniform spaces:

```

Definition entourage_
 {R : numDomainType} {T T'} (ball : T -> R -> set T') :=
 @filter_from R _ [set x | 0 < x] (fun e => [set xy | ball xy.1 e xy.2]).

```

The structure of pseudometric space follows:

```

#[short(type="pseudoMetricType")]
HB.structure Definition PseudoMetric (R : numDomainType) :=
 {T of Uniform T & Uniform_isPseudoMetric R T}.

```

In a pseudometric space, we can define the set of balls centered at  $x$  with a positive radius:

```

Definition nbhs_ball_ {R : numDomainType} {T T'} (ball : T -> R -> set T')
 (x : T) := @filter_from R _ [set e | e > 0] (ball x).
Definition nbhs_ball {R : numDomainType} {M : pseudoMetricType R} :=
 nbhs_ball_ (@ball R M).

```

The definition `nbhs_ball` is provably equivalent to the more primitive `nbhs` (§ 4.6.1).

Given a norm, we can define balls the usual way:

```

Definition ball_
 (R : numDomainType) (V : zmodType) (norm : V -> R) (x : V) (e : R) :=
 [set y | norm (x - y) < e].

```

We use this definition of balls (`ball_`) with the definition of neighborhoods defined using balls (`nbhs_ball_`) to define a filtered type for `normedZmodType`'s (see § 3.6.3) and by extension for `numDomainType`, `realDomainType`, `numFieldType`, `realFieldType`, `realType`, etc. so that for the latter mathematical structures, the notion of ball coincides with the notion of ball defined using the norm.

## 4.6.4 Complete Spaces

We only mention for the sake of exhaustiveness that pseudometric spaces can furthermore be extended to complete spaces but we do not use them in this document. See `pseudometric_structure.v`.

### 4.6.5 Normed Modules

*Normed modules* extend pseudometric spaces with a scaling operation.

We first introduce an intermediate (a bit artificial) mathematical structure of `pseudoMetricNormedZmodType` (`pseudometric_normed_Zmodule.v`):

```
MATHCOMP-ANALYSIS
HB.mixin Record NormedZmod_PseudoMetric_eq (R : numDomainType) T
 of Num.NormedZmodule R T & PseudoPointedMetric R T := {
 pseudo_metric_ball_norm : ball = ball_ (fun x : T => `| x |)
 }.
```

The latter axiom is accessible as the lemma `ball_normE`. This mixin combines a `normedZmodType` (§ 3.6.3) with a (pointed) pseudometric space (§ 4.6.3). Here is the definition of the structure of `pseudoMetricNormedZmodType`:

```
#[short(type="pseudoMetricNormedZmodType")]
HB.structure Definition PseudoMetricNormedZmod (R : numDomainType) :=
 {T of Num.NormedZmodule R T & PseudoMetric R T
 & NormedZmod_PseudoMetric_eq R T & isPointed T}.
```

Then we combine a `pseudoMetricNormedZmodType` with a left module (see § 3.6.1) and the following axiom:

```
HB.mixin Record PseudoMetricNormedZmod_Lmodule_isNormedModule K V
 of PseudoMetricNormedZmod K V & GRing.Lmodule K V := {
 normrZ : forall (l : K) (x : V), `| l *: x | = `| l | * `| x |;
 }.
```

The latter axiom is accessible by the lemma `normrZ`.

For example, the type of real numbers (§ 4.4.2) can be equipped with the structure of normed module.

## 4.7 near Notations and Tactics

As we saw in § 4.5, `MATHCOMP-ANALYSIS` is using filters for convergence. The use of filters calls for the introduction of dedicated notations and tactics. In general one does not use filters directly to prove a statement of the form  $f @ F \rightarrow y$  but rather a combination of  $\varepsilon$  reasoning and filter reasoning through the `near` notations and tactics [Affeldt et al., 2018].

The notation `\forall t \nearrow F, P` is a notation for a proposition  $P$  that holds when  $t$  “is near”  $F$ . For example, when  $F$  is the `\infty` or the `+∞` filter, this intuitively means that  $t$  tends towards  $\infty$ . Of course, if one can prove `\forall x, P x`, one can also prove `\forall x \nearrow F, P x` for any filter  $F$ :

```
Lemma nearW {T : Type} {F : set_system T} (P : T -> Prop) :
 Filter F -> (forall x, P x) -> (\forall x \nearrow F, P x).
```

Switching from a convergence statement of the form  $f x @ [x \rightarrow F] \rightarrow y$  to a combination of  $\varepsilon$  and `near` notations is the matter of a family of lemmas such as:

**Lemma** `cvgrPdist_lt` {T} {F : set\_system T} {FF : Filter F} (f : T → V) (y : V) :  
 f @ F --> y <-> forall eps, 0 < eps -> \forallall t \nearrow F, `|y - f t| < eps.

**Lemma** `cvgr_dist_lt` {T} {F : set\_system T} {FF : Filter F} (f : T → V) (y : V) :  
 f @ F --> y -> forall eps, eps > 0 -> \forallall t \nearrow F, `|y - f t| < eps.

**Lemma** `cvgrPdist_le` {T} {F : set\_system T} {FF : Filter F} (f : T → V) (y : V) :  
 f @ F --> y <-> forall eps, 0 < eps -> \forallall t \nearrow F, `|y - f t| <= eps.

...

MATHCOMP-ANALYSIS

See `pseudometric_normed_Zmodule.v`.

Introducing a near variable is performed by the tactic `near=>`. Discharging a near variable is performed by the tactic `near:.. near do rewrite ...` is a shortcut for `near=> x; rewrite ...; near: x`. Note that at the time of this writing these tactics are *not* obtained as a combination with the `=>` and `:` tacticals of § 2.2. Also, for technical reasons, scripts using the `near` tactic need to be concluded with

`Unshelve. all: end_near. Qed.`

instead of `Qed`.

Example:

.../...

**Lemma** `opp_continuous` : `continuous (@GRing.opp V)`.

**Proof.**

`move=> y.`

```
y : V

- x @[x --> y] --> - y
```

`apply/cvgrPdistr_lt => e e0.`

```
y : V
e : K
e0 : 0 < e

\forall t \nearrow y, `|- y - - t| < e
```

This is a notation to say that the set `[set t | `|- y - - t| < e]` belongs to `nbhs`, the neighboring filter of `y`. The neighboring filter of `y` is indeed defined using the balls that are centered at `x`: `[set z | norm (y - z) < e]` for all `e` (§ 4.6.3).

`near=> t.`

```
y : V, e : K, e0 : 0 < e
t : V
Hyp : t \is_near (nbhs y)

`|- y - - t| < e
```

`rewrite -opprD normrN.`

```
y : V, e : K, e0 : 0 < e
t : V
Hyp : t \is_near (nbhs y)

`|y - t| < e
```

`near: t.`

```
y : V, e : K, e0 : 0 < e

\forall t \nearrow nbhs y, `|y - t| < e
```

**exact:** `cvgr_dist_lt.`

**Unshelve.** all: **by** `end_near.` **Qed.**

Another example:

.../...

**Lemma** `cvgD f g a b : f @ F --> a -> g @ F --> b -> (f + g) @ F --> a + b.`

**Proof.**

`move=> fFa gFb; apply/cvgrPdist_lt => e e0.`

`near=> t.`

```
fFa : f x @[x --> F] --> a
gFb : g x @[x --> F] --> b
e : K
e0 : 0 < e
t : T
Hyp : t \is_near (nbhs F)
=====
`|a + b - (f + g) t| < e
```

`rewrite opprD addrAC addrA -(addrA (a - _)) -(addrC b) (splitr e).`

```
=====
`|a - f t + (b - g t)| < e / 2 + e / 2
```

`rewrite (le_lt_trans (ler_normD _ _))// ltrD//.`

```
t : T
Hyp : t \is_near (nbhs F)
=====
`|a - f t| < e / 2

goal 2 is:
`|b - g t| < e / 2
```

`near: t.`

```
e : K
e0 : 0 < e
=====
\forall x \nearrow nbhs F, `|a - f x| < e / 2
```

`apply: cvgr_dist_lt => //.`

```
e : K
e0 : 0 < e
=====
0 < e / 2
```

`by rewrite divr_gt0.`

The proof is similar for the other goal.

.../...

Other lemmas to look at if time permits:

- In <sup>MATHCOMP-ANALYSIS</sup>`topology_structure.v`: `closed_cvg`, etc.
- In <sup>MATHCOMP-ANALYSIS</sup>`ereal_normed.v`: `lime_le`; in <sup>MATHCOMP-ANALYSIS</sup>`pseudometric_normed_Zmodule.v`: `cvgr_lt`; etc.

## 4.8 Sequences

Sequences are defined in the eponymous file <sup>MATHCOMP-ANALYSIS</sup>`sequences.v`. They are just functions with domain `nat`:

**Definition** `sequence R := nat -> R`.

where `R` is expected to be a numeric type. `R ^ nat` is a notation for `sequence R`.

Because sequences are a special case of functions, the lemmas from <sup>DIRECTORY</sup>`topology_theory` and <sup>DIRECTORY</sup>`normedtype_theory` are readily available to deal with sequences.

Example: the squeeze lemma .../...

```

Context {T : Type} {a : set_system T} {Fa : Filter a} {R : realFieldType}.
Lemma squeeze_cvgr f g h : (\nearrow a, f a <= g a <= h a) ->
 forall (l : R), f @ a --> l -> h @ a --> l -> g @ a --> l.
Proof.
move=> fgh l lfa lga.

```

```

fgh : \nearrow a, f a <= g a <= h a
l : R
lfa : f x @[x --> a] --> l
lga : h x @[x --> a] --> l
=====
g x @[x --> a] --> l

```

```

apply/cvgrPdist_lt => e e_gt0; near=> x.

```

```

e : R
e_gt0 : 0 < e
x : T
Hyp : x \is_near (nbhs a)
=====
`|l - g x| < e

```

```

have := near fgh x. (* near lemma here *)

```

```

x : T
Hyp : x \is_near (nbhs a)
=====
(x \is_near (nbhs a) -> f x <= g x <= h x) -> `|l - g x| < e

```

```

move=> /(_ _)/andP[//|fg gh].

```

```

fg : f x <= g x
gh : g x <= h x
=====
`|l - g x| < e

```

```

rewrite distrC ltr_distl (lt_le_trans _ fg) ?(le_lt_trans gh)//=.

```

```

fg : f x <= g x
gh : g x <= h x
=====
h x < l + e

goal 2 is:
l - e < f x

```

```

by near: x; apply: (cvgr_lt l); rewrite // ltrDl.

```

And the other goal is similar.

.../...

Countable sums are defined in <sup>MATHCOMP-ANALYSIS</sup>`sequences.v`. This is just a combination of the iterated operations of MATHCOMP (§ 3.5.7) and of the notion of limit from <sup>MATHCOMP-ANALYSIS</sup>`filter.v` (§ 4.5.3):

**Notation** `"\big [ op / idx ]_ ( i < oo | P ) F" :=`  
`(lim (fun n => (\big [ op / idx ]_ (i < n | P) F))) : big_scope.`

In the development of measure theory (Chapter 5), we are going to use in particular countable sums of extended real numbers.



## Chapter 5

# Measure Theory with MathComp-Analysis

*Goal of this chapter: We introduce the basics of measure theory with MATHCOMP-ANALYSIS and illustrate the use of HIERARCHY-BUILDER (§ 3.5) to build a hierarchy of mathematical structures for measure theory.*

### 5.1 Extended Real Numbers

At the time of this writing, the theory of extended real numbers spans two files: `constructive_ereal.v` and `ereal.v`. The definition in itself is in `constructive_ereal.v` and it is a deceptively simple data structure ( $\overline{\mathbb{R}} \stackrel{\text{def}}{=} \mathbb{R} \cup \{-\infty, +\infty\}$ ):

```
Variant extended (R : Type) := EFin of R | EPIInf | ENInf.
```

However, the theory is surprisingly long (for example, `constructive_ereal.v` was 4796 l.o.c. in September 2025).

The notation scope is `ereal_scope`, with delimiter `E`. The notation `r%:E` is for `EFin r`; it is to inject a real number into the extended real numbers. The notation `+oo` is for `EPIInf` and the notation `-oo` is for `ENInf`. Regarding naming conventions, `y` means  $\infty$ , `Ny` means  $-\infty$  (remember Table 3.4).

The extended real numbers do not have the best structure. They do not form a group because  $\infty - \infty$  is undefined. How to deal with such exceptional

| Scope                    | Delimiter      | Meaning               | Where declared                    |
|--------------------------|----------------|-----------------------|-----------------------------------|
| <code>ereal_scope</code> | <code>E</code> | extended real numbers | <code>constructive_ereal.v</code> |

Table 5.1: Examples of scopes used in MATHCOMP-ANALYSIS

cases is always a delicate matter. The choice of MATHCOMP-ANALYSIS is to define  $\infty - \infty$  to be  $-\infty$  so that the addition ( $+\%E$ ) is associative and that the set of extended real numbers forms an (additive) monoid (see § 3.5.7) so that MATHCOMP's iterated operators [Bertot et al., 2008] can be used:

```

Definition adde_subdef x y :=
 match x, y with
 | x%:E , y%:E => (x + y)%:E
 | -oo, _ => -oo
 | _ , -oo => -oo
 | +oo, _ => +oo
 | _ , +oo => +oo
 end.

```

*Exercise 5.1.1.* Define the addition so that  $\infty - \infty = 0$  and show that the addition is not associative.

The multiplication is defined such that  $0 \times \pm\infty = 0$  as customary in measure theory.

“Inversion” is also defined for  $\overline{\mathbb{R}}$  as follows:

- $\frac{1}{0} = +\infty$ ,  $\frac{1}{+\infty} = 0$
- But  $\frac{1}{-\infty} = -\infty$ , thus  $\frac{1}{-\infty} \neq -\frac{1}{+\infty}$ .
- In fact,  $\frac{1}{-x} = -\frac{1}{x}$  only when  $x \in \mathbb{R} \setminus \{0\}$ .

We define the supremum of a set of extended real numbers using `supremum`, like we did for real numbers in § 4.4.2, except that we can now take the default value to be  $-\infty$ :

```

Definition ereal_sup S := supremum -oo S

```

**Sequences of Extended Real Numbers** Sequences of extended real numbers are heavily used in measure theory.

The lemmas about sequences of extended real numbers should be reminiscent of their real numbers counterparts. For example, compare this lemma for extended real numbers (where  $+\infty$  refers to  $+\infty\%E$ )

```

Lemma cvgeyPge : f @ F --> +oo <-> forall A, \forall x \nearrow F, A%:E <= f x.

```

with its counterpart for real numbers (where  $+\infty$  refers to the filter seen in § 4.5.1):

```

Lemma cvgryPge : f @ F --> +oo <-> forall A, \forall x \nearrow F, A <= f x.

```

In § 4.8, we explained that countable (generic) sums are formally defined as a combination of (finite) iterated operations and limits. We instantiate this definition with the addition of extended real numbers:

```

Notation "\sum_ (m <= i <oo | P) F" :=
 (\big[+\%E/0\%E]_ (m <= i <oo | P%B) F)%E : ereal_scope.

```

and then go on developing the theory of series of extended real numbers with lemmas reminiscent of iterated operations such as:

```

Lemma eq_eseriesr (R : numFieldType) (f g : ($\bar{R}$)nat) (P : pred nat)
 {N} :
 (forall i, P i -> f i = g i) ->
 \sum_(N <= i <oo | P i) f i = \sum_(N <= i <oo | P i) g i.

```

You can [Search](#) for the `eseries` substring in <sup>MATHCOMP-ANALYSIS</sup> `sequences.v` to find out generic lemmas about countable sums and for the `nneseries` substring for lemmas about non-negative terms.

The notations for countable sums that we introduced so far were defined as a combination of MATHCOMP (finite) iterated operations and limit. We introduce another, compatible definition expressed as the combination of finitely-supported sums (§ 3.5.7) and supremum: *sums over general sets*.

$$\sum_{i \in S} a_i \stackrel{\text{def}}{=} \sup \left\{ \sum_{i \in A} a_i \mid A \text{ finite subset of } S \right\}.$$

In ROCQ:

```

Definition esum S a := ereal_sup [set \sum_(x \in A) a x | A in fsets S].
Notation "\esum_ (i 'in' P) A" := (esum P (fun i => A)).

```

where `fsets S` is the set of finite sets (defined using classical sets—§ 4.2) included in `S`.

## 5.2 Formalization of $\sigma$ -algebras

The type of a  $\sigma$ -algebra can be defined as the result of a hierarchy of mathematical structures comprising semiring of sets, ring of sets, and algebra of sets or  $\sigma$ -rings [Klenke, 2013, Fig. 1.1]. It is defined in this way in particular when we build measures by extension using Carathéodory's theorem.

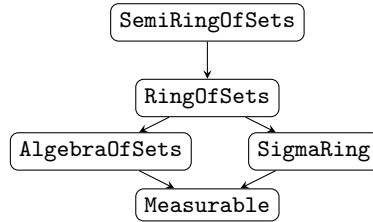


Figure 5.1: Hierarchy of measure theory structures

Figure 5.1 pictures the hierarchy of measure theory structures in MATHCOMP-ANALYSIS.

|                           | semi ring<br>of sets | ring of<br>sets | algebra<br>of sets | $\sigma$ -ring (see, e.g.,<br>[Halmos, 1974]) | $\sigma$ -algebra |
|---------------------------|----------------------|-----------------|--------------------|-----------------------------------------------|-------------------|
| contains $\emptyset$      | ✓                    | ✓               | ✓                  | ✓                                             | ✓                 |
| closed by $\cap$          | ✓                    | ✓               | ✓                  | ✓                                             | ✓                 |
| closed by semi difference | ✓                    | ✓               | ✓                  | ✓                                             | ✓                 |
| closed by $\cup$          |                      | ✓               | ✓                  | ✓                                             | ✓                 |
| contains the whole set    |                      |                 | ✓                  |                                               | ✓                 |
| closed by countable union |                      |                 |                    | ✓                                             | ✓                 |

Table 5.2: Comparison of measurable structures

A *semiring of sets* is a set of sets called `measurable` that contains the empty set, that is closed under intersection (`setI_closed`) and that is “closed by semi difference” (`semi_setD_closed`):

```
HB.mixin Record isSemiRingOfSets (d : measure_display) T := {
 measurable : set (set T) ;
 measurable0 : measurable set0 ;
 measurableI : setI_closed measurable;
 semi_measurableD : semi_setD_closed measurable;
}.
```

Here the carrier is `T` : `Type`. Do not care too much about the `measure_display` parameter, this is a trick to get nice notations (check [Affeldt and Cohen, 2023, § 3.4]).

The definition of `setI_closed` is obvious:

```
Definition setI_closed := forall A B, G A -> G B -> G (A `&` B).
```

The definition of `semi_setD_closed` is more contrived:

```
Definition semi_setD_closed := forall A B, G A -> G B -> exists D,
 [/\ finite_set D,
 D `<=<` G,
 A `<` B = \bigcup_(X in D) X &
 trivIset D id].
```

To paraphrase, it means that given two sets  $A$  and  $B$  belonging to  $G$ , there exists a set of sets  $D$  such that: (1)  $D$  is finite, (2)  $D \subseteq G$ , (3)  $A \setminus B = \bigcup_{X \in D} X$ , and (4) sets in  $D$  are pairwise disjoint.

The type of semirings of sets is `semiRingOfSetsType`; it is defined using HIERARCHY-BUILDER (§ 3.5) as follows, with a carrier that satisfies the mixin `isSemiRingOfSets`:

```
#[short(type=semiRingOfSetsType)]
HB.structure Definition SemiRingOfSets d :=
 {T of Pointed T & isSemiRingOfSets d T}.
```

Note that the structure is pointed (§ 4.4.1) by default.

A *ring of sets* is a semiring of sets that is closed by finite union (predicate `setU_closed`):

```
HB.mixin Record SemiRingOfSets_isRingOfSets d T of SemiRingOfSets d T :=
 { measurableU : @setU_closed T measurable }.
```

Observe that the mixin of ring of sets uses the `measurable` field of the mixin of semiring of sets.

```
#[short(type=ringOfSetsType)]
HB.structure Definition RingOfSets d :=
 {T of SemiRingOfSets d T
 & SemiRingOfSets_isRingOfSets d T }.
```

An *algebra of sets* is a ring of sets that contains the full set:

```
HB.mixin Record RingOfSets_isAlgebraOfSets d T of RingOfSets d T :=
 { measurableT : measurable [set: T] }.
HB.structure Definition AlgebraOfSets d :=
 {T of RingOfSets d T
 & RingOfSets_isAlgebraOfSets d T }.
```

A  $\sigma$ -*algebra* is an algebra of sets that is closed by countable union. We start by introducing a mixin for structures that are closed by countable union:

```
HB.mixin Record hasMeasurableCountableUnion d T of SemiRingOfSets d T := {
 bigcupT_measurable : forall F : (set T)^nat, (forall i, measurable (F i)) ->
 measurable (\bigcup_i (F i))
}.
```

We use this mixin to define the structure of  $\sigma$ -algebras:

```
#[short(type=measurableType)]
HB.structure Definition Measurable d :=
 {T of AlgebraOfSets d T
 & AlgebraOfSets_isMeasurable d T }.
```

In the end, we thus arrive at a type `measurableType` that is now available to declare  $\sigma$ -algebras and develop their theory.

*Exercise 5.2.1.* Understand how the structure of  $\sigma$ -rings is added to the hierarchy explained above.

Example of a derived property: .../...

**Lemma** bigcap\_measurable F P :  
 (forall k, P k -> measurable (F k)) -> measurable ( $\bigcap_{i \in P} F i$ ).

**Proof.**

move=> PF; rewrite -[X in measurable X]setCK.

```
d : measure_display
T : measurableType d
F : (set T) ^ nat
P : set nat
PF : forall k : nat, P k -> d.-measurable (F k)
=====
d.-measurable (~` ~` ($\bigcap_{i \in P} F i$))
```

rewrite setC\_bigcap.

```
=====
d.-measurable (~` ( $\bigcup_{i \in P} \sim` F i$))
```

apply: measurableC.

```
=====
d.-measurable ($\bigcup_{i \in P} \sim` F i$)
```

apply: bigcup\_measurable => k Pk.

```
PF : forall k : nat, P k -> d.-measurable (F k)
k : nat
Pk : P k
=====
d.-measurable (~` F k)
```

apply: measurableC.

```
PF : forall k : nat, P k -> d.-measurable (F k)
k : nat
Pk : P k
=====
d.-measurable (F k)
```

exact/PF.

Qed.

.../...

We have successfully defined  $\sigma$ -algebras but we are in a situation similar to topological spaces in § 4.6.1: the mixin of  $\sigma$ -algebra does not correspond to the standard, stand-alone definition of a  $\sigma$ -algebra as a set of subsets that contains the full set, and that is closed under complementation and countable unions.

For topological spaces, we saw in § 4.6.1 that `topology_structure.v` provides a factory `isOpenTopological` whose signature corresponds to the standard, stand-alone definition. The `HB.factory` command of HIERARCHY-BUILDER is for dealing with such situations. A *factory* is very much like a mixin in the sense that it is an interface:

```
HB.factory Record isMeasurable (d : measure_display) T of Pointed T := {
 measurable : set (set T) ;
 measurable0 : measurable set0 ;
 measurableC : forall A, measurable A -> measurable (~ A) ;
 measurable_bigcup : forall F : (set T)^nat,
 (forall i, measurable (F i)) -> measurable (\bigcup_i (F i))
}.
```

The difference with a mixin is that the developer has to provide a proof that from the factory one can build the original mixin that defined the  $\sigma$ -algebra in the first place. This is performed by the `HB.builders` command of HIERARCHY-BUILDER [Cohen et al., 2020]. From the user perspective, this is an improvement in terms of usability: the user can use either interface to create a  $\sigma$ -algebra.

### 5.3 Generated $\sigma$ -algebra

The goal of this section is to show that we can define a concrete example of  $\sigma$ -algebra that inhabits the type `measurableType`.

A *generated  $\sigma$ -algebra*  $\ll G \gg$  is the smallest  $\sigma$ -algebra that contains some set of sets  $G$ .

We start by defining the notion of “smallest”. We want that `smallest p G` defines the smallest set  $M$  such that  $G \preceq M$  and such that  $M$  verifies the property  $C$ . We can take the intersection of all such  $M$ ’s:

```
Context {T} (C : set T -> Prop) (G : set T).
Definition smallest := \bigcap_{A in [set M | C M /\ G \preceq M]} A.
```

This is actually already defined in `classical_sets.v`.

We now define a predicate to qualify  $\sigma$ -algebras:

```
Definition sigma_algebra T (G : set (set T)) :=
 [/\ G set0, (forall A, G A -> G (~ A)) &
 (forall A : (set T)^nat, (forall n, G (A n)) -> G (\bigcup_k A k))].
```

This is not a type like `measurableType`, even though its contents are essentially the axioms of a  $\sigma$ -algebra.

Therefore, the smallest  $\sigma$ -algebra that contains a set of sets  $G$  is:

```
Notation "<< G >>" := (smallest (@sigma_algebra _) G).
```

Can we, for any  $G$ , use  $\ll G \gg$  to instantiate the interface of  $\sigma$ -algebra from § 5.2? (MATHCOMP-ANALYSIS already provides this, as the notation  $\ll s \ G \gg$ ; here we are defining it again as an example of application of HIERARCHY-BUILDER.)

We start by proving that  $\ll G \gg$  is a  $\sigma$ -algebra in the sense of the predicate `salgebra`:

```
Variables (T : Type) (G : set (set T)).

Lemma salgebra0 : << G >> set0.
Proof. ... Qed.

Lemma salgebraC : forall A, << G >> A -> << G >> (~` A).
Proof. ... Qed.

Lemma salgebraU : forall A : (set T)^nat,
 (forall n, << G >> (A n)) -> << G >> (\bigcup_k A k).
Proof. ... Qed.
```

*Exercise 5.3.1.* Prove `salgebra0`, `salgebraC`, `salgebraU`. Do that in a new file MATHCOMP-ANALYSIS that starts by reproducing the header of `measurable_structure.v` to which you append

```
From mathcomp Require Import measurable_structure.
```

Since we have verified all the properties of a  $\sigma$ -algebra, we can instantiate the factory of § 5.2. This factory has one parameter (a display), a carrier (a `Type`), a set of sets (the measurable sets), and three properties. Let us prepare one identifier `sdisplay` for the display, one identifier `salgType` to extract the carrier from a set of sets, and take  $\ll G \gg$  to be the set of sets. We can instantiate using the `HB.instance` command:

```
Variables (T : pointedType) (G : set (set T)).

HB.instance Definition _ := @isMeasurable.Build
 (sdisplay G)
 (salgType G)
 << G >>
 (@salgebra0 _ G) (@salgebraC _ G) (@salgebraU _ G).
```

The function `isMeasurable.Build` is a constructor function that has been produced by HIERARCHY-BUILDER upon definition of the `isMeasurable` interface, be it a mixin or a factory.

As a consequence of the above instantiation, we are now given for any set of sets  $G$ , the type `salgType G` of a generated  $\sigma$ -algebra:

```
Variable T : pointedType.
Variable G : set (set T).
Check << G >> : set (set T).
Check salgType G : measurableType _.
```



## 5.4 Formalization of Measures

In the same way that a  $\sigma$ -algebra is also an algebra of sets, a  $\sigma$ -ring, a ring of sets, and a semiring of sets, a measure is also an *additive measure*, a.k.a. a *content*. HIERARCHY-BUILDER can also deal with hierarchy of morphisms, where the carrier is not a `Type` but a function.

A function of type `set T -> \bar R` is *semiadditive* when for any sequence of measurable, pairwise-disjoint sets  $F$  we have

$$\forall n, \text{measurable} \left( \bigcup_{k < n} F_k \right) \rightarrow \mu \left( \bigcup_{k < n} F_k \right) = \sum_{k < n} \mu(F_k)$$

Formal paraphrase in MATHCOMP-ANALYSIS:

```
1 Definition semi_additive := forall F n,
2 (forall k : nat, measurable (F k)) -> trivialIset setT F ->
3 measurable (\big[setU/set0](k < n) F k) ->
4 mu (\big[setU/set0](i < n) F i) = \sum_(i < n) mu (F i).
```

Of course, the condition line 3 is trivially satisfied when  $T$  is a `measurableType`.

A function of type `set T -> \bar R` is *semi- $\sigma$ -additive* when, for any sequence of measurable, pairwise-disjoint sets  $F$  we have

$$\text{measurable} \left( \bigcup_k F_k \right) \rightarrow \sum_{k < n} \mu(F_k) \xrightarrow{n \rightarrow \infty} \mu \left( \bigcup_k F_k \right)$$

Formal paraphrase in MATHCOMP-ANALYSIS:

```
Definition semi_sigma_additive :=
 forall F, (forall i : nat, measurable (F i)) -> trivialIset setT F ->
 measurable (\bigcup_n F n) ->
 (fun n => \sum_(0 <= i < n) mu (F i)) --> mu (\bigcup_n F n).
```

**Hierarchy of Measures** At the bottom of the hierarchy of measures, we put contents. They are non-negative functions that are *semiadditive*:

```
HB.mixin Record isContent d
 (T : semiRingOfSetsType d) (R : numFieldType)
 (mu : set T -> \bar R) := {
 measure_ge0 : forall x, 0 <= mu x ;
 measure_semi_additive : semi_additive mu }.
```

```
HB.structure Definition Content d
 (T : semiRingOfSetsType d) (R : numFieldType) := {
 mu & isContent d R T mu }.
```

We do not restrict ourselves to the type of real numbers `realType` but instead use the more general `numFieldType`. Similarly, we take the domain of the content to be over a semiring of sets. The fact that the measure of the empty set is 0 is a consequence of semiadditivity.

*Measures* extend semiadditive measures by adding semi- $\sigma$ -additivity:

```

HB.mixin Record Content_isMeasure d
 (T : semiRingOfSetsType d) (R : numFieldType)
 mu of Content d mu := {
 measure_semi_sigma_additive : semi_sigma_additive mu }.

#[short(type=measure)]
HB.structure Definition Measure d
 (T : semiRingOfSetsType d) (R : numFieldType) :=
 {mu of Content d mu
 & Content_isMeasure d R T mu}.

```

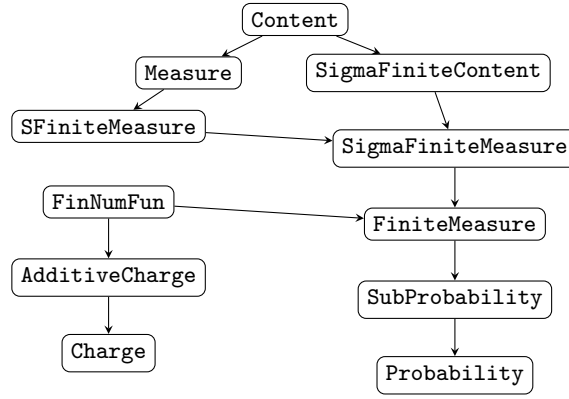


Figure 5.2: Hierarchy of measure structures

The hierarchy of measures is displayed in Fig. 5.2. Contents and measures can be found at the root. The hierarchy also contains  $\sigma$ -finite measures with notation

```
{sigma_finite_measure set T -> \bar R}
```

See [Ishiguro and Affeldt, 2024, § 3.1] for *charges* (a.k.a. *signed measures*).

Of course, it is not necessary to go through the trouble of systematically defining an intermediate content to define a measure, the user can instantiate directly a measure by using the `isMeasure` factory as we did for  $\sigma$ -algebras in § 5.2:

```

HB.factory Record isMeasure
 d (T : semiRingOfSetsType d) (R : realFieldType)
 (mu : set T -> \bar R) := {
 measure0 : mu set0 = 0 ;
 measure_ge0 : forall x, 0 <= mu x ;
 measure_semi_sigma_additive : semi_sigma_additive mu }.

```

See the code [Affeldt et al., 2017, file `measure_function.v`].

### 5.4.1 Example: the Dirac Measure

In the same way we instantiated the interface of  $\sigma$ -algebra with generated  $\sigma$ -algebras, we can populate the interface of measures with concrete measure. The *Dirac measure*  $\delta_a$  is the measure that is 1 for sets  $A$  such that  $a \in A$  and 0 otherwise:

```
Context d (T : sigmaRingType d) (a : T) (R : realFieldType).
Definition dirac (A : set T) : \bar R := (\1_A a)%E.
```

The notation  $\backslash 1_A a$  is for the *indicator function* defined in <sup>MATHCOMP-ANALYSIS</sup> `numfun.v`. It is really just a wrapper for the boolean test `a \in A`.

To declare it as a measure we need three proofs

```
Let dirac0 : dirac set0 = 0. Proof. by rewrite /dirac indic0. Qed.
Let dirac_ge0 B : 0 <= dirac B. Proof. by rewrite /dirac indicE. Qed.
Let dirac_sigma_additive : semi_sigma_additive dirac.
Proof. (* see page 84 *) Qed.
```

and an invocation of `HB.instance`:

```
HB.instance Definition _ := isMeasure.Build _ _ _
 dirac dirac0 dirac_ge0 dirac_sigma_additive.
```

.../...

```
Let dirac_sigma_additive : semi_sigma_additive dirac.
```

```
Proof.
```

```
move=> F mF tF mUF; rewrite /dirac indicE; have [|aFn] /= := boolP (a \in _).
```

```
F : nat -> set T
mF : forall i : nat, d.-measurable (F i)
tF : trivIset [set: nat] F
mUF : d.-measurable (\bigcup_n F n)
=====
a \in \bigcup_n F n ->
 (fun n : nat => \sum_(0 <= i < n) (\1_(F i) a)%:E) --> 1
```

```
rewrite inE => -[n _ Fna].
```

```
n : nat
Fna : F n a
=====
(fun n0 : nat => \sum_(0 <= i < n0) (\1_(F i) a)%:E) --> 1
```

```
have naF m : m != n -> a \notin F m.
 (* omitted *)
```

```
n : nat
Fna : F n a
naF : forall m : nat_eqType, m != n -> a \notin F m
=====
(fun n0 : nat => \sum_(0 <= i < n0) (\1_(F i) a)%:E) --> 1%:E
```

```
apply/cvg_ballP => _/posnumP[e]; near=> m.
```

```
e : {posnum R}
m : nat
Hyp : m \is_near (nbhs \oo)
=====
ball 1%:E e%:num (\sum_(0 <= i < m) (\1_(F i) a)%:E)
```

```
have mn : (n < m)%N by near: m; exists n.+1.
```

```
mn : (n < m)%N
=====
ball 1%:E e%:num (\sum_(0 <= i < m) (\1_(F i) a)%:E)
```

We can show that the sum is equal to 1 since there is only one  $n < m$  such that  $a \in F n$ . Finally, we use a property of balls of a pseudometric spaces (§ 4.6.3) to conclude.

The second goal (that was generated by the case analysis at the very first line of the script) is:

```

aFn : a \notin \bigcup_n F n
=====
(fun n : nat => \sum_ (0 <= i < n) (\1_ (F i) a)%:E) --> 0

```

This holds because the left-hand side is the constant function 0.      .../...

### 5.4.2 Other Instances of Measures

MATHCOMP-ANALYSIS provides several instances of non-negative measures. In DIRECTORY `measure_theory`, besides the Dirac measure, one can find the pushforward measure, the null measure, the sum of measures (be it finite or countable), the scaled (by a non-negative number) measure, the restriction of a measure, the counting measure, the product measure (notation  $\times$ ), and other kinds of a bit more abstract measures that are involved in the construction of the *Lebesgue measure* (MATHCOMP-ANALYSIS `lebesgue_measure.v`) [Affeldt and Cohen, 2023] or the *Lebesgue-Stieltjes measure* [Ishiguro and Affeldt, 2024, § 5] (MATHCOMP-ANALYSIS `lebesgue_stieltjes_measure.v`).

## 5.5 Measurable Functions

A function with domain  $D$  is *measurable* when the preimage of any measurable set is measurable:

**Definition** `measurable_fun`  $d\ d'$  ( $T : \text{sigmaRingType } d$ ) ( $U : \text{sigmaRingType } d'$ )  
 $(D : \text{set } T) (f : T \rightarrow U) :=$   
`measurable`  $D \rightarrow \text{forall } Y, \text{measurable } Y \rightarrow \text{measurable } (D \setminus f^{-1} Y).$

Note that this definition does not rely on the definition of measure, only on the definition of  $\sigma$ -ring. Measurable functions are precisely the functions that we will integrate in the next chapter (Chapter 6).

There is a fairly large theory of measurable functions developed in MATHCOMP-ANALYSIS `measurable_function.v` and `measurable_realfun.v`. You may want to know, e.g., whether the set of measurable functions is stable by composition (lemma `measurable_comp`), or whether the set of real number-valued measurable functions is stable by point-wise addition:

**Context**  $d$  ( $T : \text{measurableType } d$ ) ( $R : \text{realType}$ ).  
**Lemma** `measurable_funD`  $D$  ( $f\ g : T \rightarrow R$ ) :  
`measurable_fun`  $D\ f \rightarrow \text{measurable_fun } D\ g \rightarrow \text{measurable_fun } D\ (f \setminus +\ g).$

*Exercise 5.5.1.* Show that  $\lambda x.x^2 + x^3$  is a measurable function. You need to use `measurable_realfun.v`.

There is also an interface for measurable functions defined in MATHCOMP-ANALYSIS `measurable_function.v` by:

**HB.mixin** **Record** `isMeasurableFun`  $d\ d'$  ( $aT : \text{sigmaRingType } d$ ) ( $rT : \text{sigmaRingType } d'$ )  
 $(f : aT \rightarrow rT) := \{$   
`measurable_funPT` : `measurable_fun` [`set`:  $aT$ ]  $f$   
 $\}.$

**HB.structure** **Definition** `MeasurableFun`  $d\ d'$   $aT\ rT :=$   
 $\{f\ \text{of } @isMeasurableFun\ d\ d'\ aT\ rT\ f\}.$

Let  $\{mfun\ aT \gg T\}$  be the notation for these HIERARCHY-BUILDER-defined measurable functions.

## Chapter 6

# Integration Theory with MathComp-Analysis

*Goal of this chapter: We explain how we use the measure theory of MATHCOMP-ANALYSIS to develop integration theory.*

### 6.1 Simple Functions

On paper, a *simple function*  $f$  is typically defined by a sequence of pairwise-disjoint and measurable sets  $A_0, \dots, A_{n-1}$  and a sequence of elements  $a_0, \dots, a_{n-1}$  such that  $f(x) = \sum_{k=0}^{n-1} a_k \mathbf{1}_{A_k}(x)$ . One can choose to formalize this definition directly, for example by representing the  $a_k$ 's by a list without duplicate, and use this representation to develop the necessary theory to formalize integration.

MATHCOMP-ANALYSIS is taking a bit more abstract and compositional approach by formalizing the hierarchy of Fig. 6.1 where simple functions are measurable functions with a finite image.

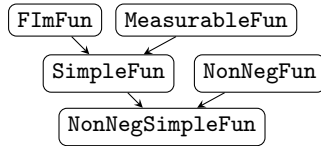


Figure 6.1: Hierarchy for non-negative simple functions

At the top of Fig. 6.1, we find functions with a finite image. Functions with a finite image are defined in MATHCOMP-ANALYSIS `cardinality.v` by:

```
HB.mixin Record FiniteImage aT rT (f : aT -> rT) := {
 fimfunP : finite_set (range f)
}.
HB.structure Definition FImFun aT rT := {f of @FiniteImage aT rT f}.
```

The notation `range f` stands for `[set f x | x in setT]`. Let `{fimfun aT -> rT}` be a notation for functions from `T` to `R` with a finite image. Given a function with a finite image, we can prove that it decomposes into a sum of indicator functions, like a simple function should, except that we do not need any measurability hypotheses:

```
Lemma fimfunE T (R : ringType) (f : {fimfun T -> R}) x :
 f x = \sum_(y \in range f) (y * \1_(f @~1` [set y]) x).
```

See MATHCOMP-ANALYSIS `numfun.v` for this lemma.

The structure of *simple functions* is obtained by combining the interfaces of functions with a finite image and of measurable functions (see § 5.5). Notation for simple functions: `{sfun aT -> R}`

Similarly, we can define the interface of non-negative functions with a notation `{nnfun T -> R}` and combine with simple functions to get *non-negative simple functions* with notation `{nnsfun T -> R}`.

### 6.1.1 Approximation Theorem

Before defining integration, we prove a theorem to approximate measurable functions using simple functions. This is an important theorem because it is used pervasively, in particular to prove the monotone convergence theorem (§ 6.3.3). The idea to build the approximation function is explained in Fig. 6.2. It relies on the definition of dyadic intervals and two sets  $B_n$  and  $A_{n,k}$  defined below.

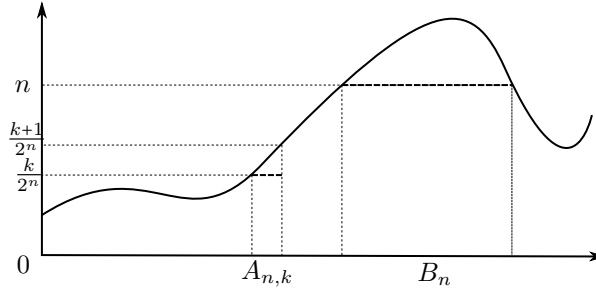


Figure 6.2: Approximation of a measurable function using simple functions

**Definition 6.1.1** (Dyadic Interval). Given  $n$  and  $k$ , we call *dyadic interval* the interval  $I_{n,k} \stackrel{\text{def}}{=} \left[ \frac{k}{2^n}, \frac{k+1}{2^n} \right]$ .

Let  $f$  be an extended real number-valued measurable function with domain  $D$ . Given a natural number  $n$ , we define  $B_n$  to be the set  $D \cap \{x \mid n \leq f(x)\}$ :

```
Definition integer_approx n := D `&` [set x | n%:R%:E <= f x]%E.
```

Given  $n$  and  $k$ , we define  $A_{n,k}$  to be the set  $D \cap \{x \mid f(x) \in I_{n,k}\}$  if  $k < n2^n$  and  $\emptyset$  otherwise:



**Definition** `dyadic_approx`  $n\ k := \text{if } (k < n * 2^n) \% N \text{ then}$   
 $D \ \&\` \text{ [set } x \mid f\ x \ \backslash \text{in } E\text{Fin } @\` \text{ [set } I\ n\ k]] \text{ else set0.}$

Using  $B_n$  and  $A_{n,k}$ , we define the  $n$ th approximation of the function  $f$  to be the function

$$x \mapsto \sum_{k < n2^n} \frac{k}{2^n} 1_{A_{n,k}}(x) + n 1_{B_n}(x)$$

**Local Notation**  $A := \text{dyadic\_approx.}$

**Local Notation**  $B := \text{integer\_approx.}$

**Definition** `approx` :  $(T \rightarrow R)^{\text{nat}} := \text{fun } n\ x \Rightarrow$   
 $\backslash \text{sum\_} (k < n * 2^n) \ k \% : R * 2^n - n * \backslash 1\_ (A\ n\ k) \ x + n \% : R * \backslash 1\_ (B\ n) \ x.$

**Theorem 6.1.2** (Approximation Theorem). *For any  $(*)$  measurable set  $D$ , any  $(*)$  function  $f$  that is  $(*)$  measurable, the sequence defined by `approx` seen a  $(*)$  sequence of non-negative simple functions is  $(*)$  non-decreasing and,  $f$  is  $(*)$  non-negative,  $(*)$  converges towards  $f$ .*

```
Context d (T : measurableType d) (R : realType).
Variables (D : set T) (mD : measurable D) (*1*).
Variables (f : T -> \bar R) (*2*) (mf : measurable_fun D f) (*3*).

Definition nnsfun_approx : {nnsfun T -> R}^nat := fun n =>
 (* see approx *) (*4*)

Lemma nd_nnsfun_approx :
 nondecreasing_seq (nnsfun_approx : (T -> R)^nat). (*5*)
Lemma cvg_nnsfun_approx
 (f0 : forall x, D x -> (0 <= f x) % E) x : (*6*)
 D x -> EFin \o nnsfun_approx^x @ \oo --> f x. (*7*)
```

The notation `nondecreasing_seq`  $f$  is for

$\{\text{homo } f : n\ m / (n \leq m) \% N \rightarrow (n \leq m) \% 0\}$

which is a specialization of a notation that we saw in § 3.2. The notation  $\sim$  has been explained in § 2.1.1; informally,  $E\text{Fin } \backslash \circ g \ \sim \ x$  means  $\lambda n. g_n(x)$ .

## 6.2 Integral of Measurable Functions

### 6.2.1 Integral of a Simple Function

Let  $f$  be a simple function and  $\mu$  be a non-negative measure. The integral of  $f$  w.r.t.  $\mu$  is defined by

$$\sum_{x \in \mathbb{R}} x \mu(f^{-1}\{x\}).$$

This definition uses summation over a finite support (see § 3.5.7), so that we are truly using the fact that  $f$  has a finite image.

**Variables**  $(T : \text{Type}) (R : \text{numDomainType}) (\mu : \text{set } T \rightarrow \backslash \bar R) (f : T \rightarrow R).$

**Definition** `sintegral` :=  $\backslash \text{sum\_} (x \ \backslash \text{in } [\text{set} : R]) \ x \% : E * \mu (f \ @^{-1} \text{ [set } x]).$

### 6.2.2 Integral of a Non-negative Function

Let  $f$  be an extended real-valued function over some `measurableType`. Its integral is defined by

$$\sup_h \left\{ \int_x h(x)(d\mu) \mid h \text{ non-negative simple function } \leq f \right\}.$$

(Note that we are abusing the integral sign notation.)

```
Let nnintegral mu f := ereal_sup [set sintegral mu h |
 h in [set h : {nnsfun T -> R} | forall x, (h x)%:E <= f x]].
```

The definition does not insist on having  $f$  non-negative but this will be necessary to obtain the desired properties.

### 6.2.3 Integral of a Measurable Function

Let  $f$  be a extended real-valued function over some `measurableType`. We define its positive part as the (non-negative) function  $\lambda x. \max(f(x), 0)$  defined in `MATHCOMP-ANALYSIS` `numfun.v` with the notation  $f \wedge +$ . Similarly, we define its negative part as the (non-negative) function  $\lambda x. \max(-f(x), 0)$  with the notation  $f \wedge -$ .

The *integral* of  $f$  over the domain  $D$  is defined by the difference between the positive part and the negative part of the restriction of  $f$  to  $D$ :

```
Definition integral mu D f (g := f _ D) :=
 nnintegral mu (g ^\+) - nnintegral mu (g ^\+).
```

Remember the definition of  $f \wedge -$  D from § 4.4.1.

We introduce the ASCII notation `\int[mu]_(x in D) f x` for  $\int_{x \in D} f(x)(d\mu)$ . Observe that this notation has the same form as the iterated operations. Compare

```
\int[mu]_(x in D) f x
```

with, say,

```
\big[op/idx]_(x in D) f x
```

After all, we expect integration to have properties reminiscent of sums, so we'd better have similar-looking notations and naming conventions to help us name and search lemmas of the forthcoming theory of integration.

### 6.2.4 Properties of the Integral

Let  $f_1$  and  $f_2$  be two non-negative, measurable, extended real-valued functions with domain  $D$ . We have the monotone integral property:

```
Lemma ge0_le_integral : (forall x, D x -> f1 x <= f2 x) ->
 \int[mu]_(x in D) f1 x <= \int[mu]_(x in D) f2 x.
```

The proof uses the approximation theorem (Theorem 6.1.2).

## 6.3 Monotone Convergence Theorem

Informal statement: For any non-decreasing sequence of non-negative measurable functions  $g_n$ , we have  $\int_{x \in D} (\lim g_n)(x)(d\mu) = \lim(\int_{x \in D} g_n(x)(d\mu))$ .

The proof of the monotone convergence theorem is in three steps:

- proof for simple functions only (§ 6.3.1),
- proof for simple functions converging to a measurable function (§ 6.3.2),
- proof for measurable functions converging to a measurable function (§ 6.3.3).

### 6.3.1 Monotone Convergence for Simple Functions

For any  $(*)1$  sequence of non-negative simple functions  $g$  that is  $(*)3$  nondecreasing and that  $(*)4$  converges towards a  $(*)2$  non-negative simple function  $f$ , we have

$$\int_x f(x)(d\mu) = \lim_{n \rightarrow \infty} \int_x g_n(x)(d\mu).$$

```
Context d (T : measurableType d) (R : realType).
Variable mu : {measure set T -> \bar R}.
Variables (g : {nnsfun T -> R}^nat) (*1*) (f : {nnsfun T -> R}) (*2*).
Hypothesis nd_g : forall x, nondecreasing_seq (g ~ x) (*3*).
Hypothesis gf : forall x, g ~ x @ \oo --> f x (*4*).

Lemma nd_sintegral_lim : sintegral mu f = limn (sintegral mu \o g).
```

The proof is by proving the  $\leq$  part and the  $\geq$  part. The difficult part is  $\text{sintegral mu f} \leq \lim (\text{sintegral mu } \text{\o } g)$ . See the file MATHCOMP-ANALYSIS `lebesgue_integral_definition.v`.

### 6.3.2 Monotone Convergence Intermediate Lemma

For any  $(*)1$  function  $f: T \rightarrow \overline{\mathbb{R}}$  that is  $(*)2$  non-negative and  $(*)3$  measurable, any  $(*)4$  sequence  $g$  of non-negative simple functions that is  $(*)5$  non-decreasing and  $(*)6$  converging towards  $f$ , we have

$$\int_x f(x)(d\mu) = \lim_{n \rightarrow \infty} \int_x g_n(x)(d\mu).$$

```
Context d (T : measurableType d) (R : realType).
Variables (mu : {measure set T -> \bar R}) (f : T -> \bar R) (*1*)
 (g : {nnsfun T -> R}^nat) (*4*).
Hypothesis f0 : forall x, 0 <= f x (*2*).
Hypothesis mf : measurable_fun setT f (*3*).
Hypothesis nd_g : forall x, nondecreasing_seq (g ~ x) (*5*).
Hypothesis gf : forall x, EFin \o g ~ x @ \oo --> f x (*6*).

Lemma nd_ge0_integral_lim : \int[mu]_x f x = limn (sintegral mu \o g).
```

Again, the proof is by proving successively the  $\leq$  part and the  $\geq$  part.

### 6.3.3 Proof of the Monotone Convergence Theorem

For  $(*)$  any measurable set  $D$  and any  $(*)$  non-decreasing sequence of functions  $(*)$   $g_n : T \rightarrow \mathbb{R}$  that are  $(*)$  measurable and  $(*)$  non-negative, we have

$$\int_{x \in D} \left( \lim_{n \rightarrow \infty} g_n(x) \right) (d\mu) = \lim_{n \rightarrow \infty} \int_{x \in D} g_n(x) (d\mu).$$

```
Context d (T : measurableType d) (R : realType).
Variables (mu : {measure set T -> \bar R}) (D : set T).
Variables (mD : measurable D) (*1*) (g : (T -> \bar R) ^ nat) (*2*).
Hypothesis mg : forall n, measurable_fun D (g n) (*4*).
Hypothesis g0 : forall n x, D x -> 0 <= g n x (*5*).
Hypothesis nd_g : forall x, D x -> nondecreasing_seq (g ^~ x) (*3*).

Lemma monotone_convergence :
 \int[mu]_ (x in D) limn (g ^~ x) = limn (fun n => \int[mu]_ (x in D) g n x).
```

#### Easy Direction

$$\lim_{n \rightarrow \infty} \int_{x \in D} g_n(x) (d\mu) \leq \int_{x \in D} \left( \lim_{n \rightarrow \infty} g_n(x) \right) (d\mu)$$

In ROCQ, this appears as

```
lim (fun n => \int[mu]_x g n x) <= \int[mu]_x f x
```

where  $f$  is  $\text{fun } x \Rightarrow \lim (g \text{ } ^\sim x)$ . In particular, the domain of integration  $D$  has been put under the integral by using the notation of restriction of a function (notation  $\backslash_$ , see § 4.4.1).

The proof is by appealing to properties of sequences of extended real numbers and to the fact that the integral is monotone (§ 6.2.4). Indeed, we can use `ge0_le_integral` to show that:

- the sequence of integrals on the left-hand side is non-decreasing:

```
nondecreasing_seq (fun n => \int[mu]_x g n x)
```

- each of its terms is upper bounded by the right-hand side:

```
\int[mu]_x g n x <= \int[mu]_x f x
```

Therefore, the sequence on the left-hand side is convergent and its limit is bounded by the right-hand side.

#### Difficult Direction

$$\int_{x \in D} \underbrace{\left( \lim_{n \rightarrow \infty} g_n \right) (x)}_{f(x)} (d\mu) \leq \lim_{n \rightarrow \infty} \int_{x \in D} g_n(x) (d\mu)$$

In ROCQ, this appears as

$\int [\mu]_x f x \leq \lim (\text{fun } n \Rightarrow \int [\mu]_x g_n x)$

with the same proviso as above.

The idea is to build a sequence of non-negative simple functions  $h_n$  (see below) that is non-decreasing and such that  $h_n \leq g_n$  and  $\lim_{n \rightarrow \infty} h_n = f$ .

Then we can use the lemma from § 6.3.2 to show

$$\int_{x \in D} f(x)(d\mu) = \lim_{n \rightarrow \infty} \int_{x \in D} h_n(x)(d\mu)$$

which leads to

$$\lim_{n \rightarrow \infty} \int_{x \in D} h_n(x)(d\mu) \leq \lim_{n \rightarrow \infty} \int_{x \in D} g_n(x)(d\mu)$$

and then we are able to conclude by appealing to the monotone properties of limits and of integrals.

So, our problem is to find simple functions  $h_n$  such that  $\lim_{n \rightarrow \infty} h_n = f$ .

We approximate (in the sense of the approximation theorem—§ 6.1.2) each measurable function  $g$  by a function  $g_2$ :

```
(* raw functions *)
Let g2' n : (T -> R)^nat := approx setT (g n).
(* same functions but with their properties embedded in types *)
Let g2 n : {nnsfun T -> R}^nat := nnsfun_approx measurableT (mg n).
```

(Note that  $g2'$  is a sequence of sequences.) And then use these  $g2$  functions to create the desired function  $h$  that we call  $\text{max\_g2}$  because it is defined by taken the max of all  $g2$  functions:

```
(* raw functions *)
Let max_g2' : (T -> R)^nat :=
 fun k t => (\big[maxr/0](i < k) (g2' i k) t)%R.
(* same functions but with their properties embedded in types *)
Let max_g2 : {nnsfun T -> R}^nat := fun k => bigmax_nnsfun (g2^~ k) k.
```

Does  $\text{max\_g2}/h$  has the right properties? (I.e., non-decreasing, upper-bounded by  $g$ , and converging to  $f$ .)

- $h_n$  non-decreasing? Yes, essentially because each  $g2$  is.
- $h_n \leq g_n$ ? Yes, essentially because  $g2_n \leq g_n$ .
- $\lim_{n \rightarrow \infty} h_n = f$ ? This is a bit more technical, this is `cvg_max_g2_f` in the MATHCOMP-ANALYSIS file `lebesgue_integral_monotone_convergence.v`.

How do we prove  $\lim_{n \rightarrow \infty} h_n = f = \lim_{n \rightarrow \infty} g_n$ ?

- $\lim_{n \rightarrow \infty} h_n \leq \lim_{n \rightarrow \infty} g_n$  is easy, this is by construction.
- $\lim_{n \rightarrow \infty} g_n \leq \lim_{n \rightarrow \infty} h_n$  requires a bit of work.

- Suppose that the right-hand side is  $< +\infty$  (otherwise this is obvious).
- It suffices to prove:

`\forall n \nearrow \infty, g n t <= lim (EFin \o max_g2 ^~ t)`

Use the `near=> n` tactic (§ 4.7) to get a large enough  $n$ .

```

* If g n t is +∞:
 then (approx D (g n)) ^~ t diverges,
 then lim (EFin \o g2 n ^~ t) = +∞,
 then lim (EFin \o max_g2 ^~ t) = +∞.

* If g n t < +∞:
 then (approx D (g n)) ^~ t converges towards g n t,
 then lim (EFin \o g2 n ^~ t) = g n t,
 we conclude because each g2 is smaller or equal to max_g2.

```

That concludes the proof of the monotone convergence theorem.

## 6.4 Fubini's Theorem

In § 1.4, we set our goal as going as far as Fubini's theorem. The reader should now be in a position to read the file `lebesgue_integral_fubini.v` to understand how the proof is carried out.

Fubini's theorem is about a function with two arguments that is measurable and *integrable* (i.e., the integral of its absolute value is not  $\infty$ , see definition *integrable*). See also [Affeldt and Cohen, 2023].

As an intermediate theorem, one uses Fubini-Tonelli's theorem, which is a similar statement for non-negative functions. Let us state one part of Fubini-Tonelli's theorem, which states the equality of the integration over the product measure of  $m_1$  and  $m_2$  (notation:  $m_1 \times m_2$ ) and of the successive integration over  $m_2$  and then over  $m_1$  (Fubini-Tonelli's theorem is obtained by combining with the symmetric statement):

```

Context d1 d2 (T1 : measurableType d1) (T2 : measurableType d2)
 (R : realType).
Variables (m1 : {sigma_finite_measure set T1 -> \bar R})
 (m2 : {sigma_finite_measure set T2 -> \bar R}).
Variable f : T1 * T2 -> \bar R.
Hypothesis mf : measurable_fun setT f.
Hypothesis f0 : forall x, 0 <= f x.

Lemma fubini_tonelli1 : \int[m1 \times m2]_z f z = \int[m1]_x \int[m2]_y f (x, y).

```

This is proved by using the approximation theorem (Theorem 6.1.2) and the monotone convergence theorem (§ 6.3.3).

Using the approximation theorem, we turn the function  $f$  into a non-decreasing sequence of non-negative simple functions that converges towards  $f$ , thus trans-

forming the problem into an integration problem of non-negative simple functions, which is arguably simpler (see lemma `sfun_fubini_tonelli` in the file `MATHCOMP-ANALYSIS` `lebesgue_integral_fubini.v`).

Since non-negative simple functions can be expressed as sums of indicator functions (see lemma `fimfunE` in § 6.1), we can furthermore simplify the problem to the integration of indicator functions (see lemma `indic_fubini_tonelli` in the file `MATHCOMP-ANALYSIS` `lebesgue_integral_fubini.v`).

The formal proof of Fubini's theorem is an application of Fubini-Tonelli's theorem with a bit of “almost everywhere” reasoning, which is provided by the notation `{ae mu, forall x, P x}` where `mu` is a measure and `P` is a predicate in the file `MATHCOMP-ANALYSIS` `measure.v`. See `Theorem Fubini` towards the end of the file `MATHCOMP-ANALYSIS` `lebesgue_integral_fubini.v` for the formal proof of Fubini's theorem.





## Chapter 7

# Derivation with MathComp-Analysis

*Goal of this chapter: This chapter will introduce the basics of differentiation and connect with measure theory (Chapter 5) with the Fundamental Theorem of Calculus [Affeldt and Stone, 2024]. This chapter is however still work in progress.*

### 7.1 Differentiation

Differentiation is defined in the file `MATHCOMP-ANALYSIS/derive.v`. The derivative of an univariate function `f` is noted `f^`()` (in notation scope `classical_set_scope`). This is the same notation as for polynomials (§ 3.6.2). Alternatively, `f^`()` can be written `'D_1 f`, using the more general notation `'D_v` for any `v`. We can switch between the two notations using the lemma `derive1E`.

**Variables** `(R : numFieldType) (V : normedModType R).`

**Lemma** `derive1E (f : R -> V) a : f^`() a = 'D_1 f a.`

Differentiation of univariate functions has the expected properties, e.g., `derived`.



## Chapter 8

# Conclusion

This document tried to provide a self-contained introduction to MATHCOMP-ANALYSIS so that newcomers can start exploring its formal proofs about real analysis and eventually contribute. It is fair to say that formalization of mathematics using type theory still looks too technical but researchers have been working hard to find a way to bridge the gap between the foundations of mathematics (type theory) and practical applications (e.g., software development): many limitations have been lifted and it is arguably starting to scale. There are still countless, unexplored topics of interest though and, regarding MATHCOMP-ANALYSIS, discussions are happening online in particular with team chat tools (mostly Zulip) and distributed software development tools (mostly GitHub). Hope to see you there.



# Bibliography

- [Affeldt, 2017] Affeldt, R. (2017). Formal verification using the Coq proof assistant. <https://staff.aist.go.jp/reynald.affeldt/ssrcoq/ssrcoq.pdf>. In Japanese. Slides.
- [Affeldt et al., 2017] Affeldt, R., Bertot, Y., Bruni, A., Cohen, C., Kerjean, M., Mahboubi, A., Rouhling, D., Roux, P., Sakaguchi, K., Stone, Z., Strub, P.-Y., and Théry, L. (2017). Mathematical components compliant analysis library. <https://github.com/math-comp/analysis>. Last stable version 1.3.0 (2024).
- [Affeldt and Cohen, 2017] Affeldt, R. and Cohen, C. (2017). Formal foundations of 3D geometry to model robot manipulators. In *6th ACM SIGPLAN Conference on Certified Programs and Proofs (CPP 2017), Paris, France, January 16–17, 2017*, pages 30–42. ACM Press. doi.
- [Affeldt and Cohen, 2023] Affeldt, R. and Cohen, C. (2023). Measure construction by extension in dependent type theory with application to integration. *J. Autom. Reason.*, 67(3):28.
- [Affeldt et al., 2020a] Affeldt, R., Cohen, C., Kerjean, M., Mahboubi, A., Rouhling, D., and Sakaguchi, K. (2020a). Competing inheritance paths in dependent type theory: a case study in functional analysis. In *10th International Joint Conference on Automated Reasoning (IJCAR 2020), Paris, France, June 29–July 6, 2020*, volume 12167 of *Lecture Notes in Artificial Intelligence*, pages 3–20. Springer. doi.
- [Affeldt et al., 2018] Affeldt, R., Cohen, C., and Rouhling, D. (2018). Formalization techniques for asymptotic reasoning in classical analysis. *J. Formalized Reasoning*, 11(1):43–76.
- [Affeldt et al., 2023] Affeldt, R., Cohen, C., and Saito, A. (2023). Semantics of probabilistic programs using s-finite kernels in Coq. In *12th ACM SIGPLAN International Conference on Certified Programs and Proofs (CPP 2023), Boston, Massachusetts, USA, January 16–17, 2023*. ACM Press.
- [Affeldt et al., 2021] Affeldt, R., Garrigue, J., Nowak, D., and Saikawa, T. (2021). A trustful monad for axiomatic reasoning with probability and non-

- determinism. *Journal of Functional Programming*, 31(E17). doi arXiv: cs.LO 2003.09993.
- [Affeldt et al., 2020b] Affeldt, R., Garrigue, J., and Saikawa, T. (2020b). A library for formalization of linear error-correcting codes. *Journal of Automated Reasoning*, 64:1123–1164.
- [Affeldt et al., 2014] Affeldt, R., Hagiwara, M., and Sénizergues, J. (2014). Formalization of Shannon’s theorems. *Journal of Automated Reasoning*, 53(1):63–103.
- [Affeldt and Nowak, 2021] Affeldt, R. and Nowak, D. (2021). Extending equational monadic reasoning with monad transformers. In *26th International Conference on Types for Proofs and Programs (TYPES 2020)*, volume 188 of *Leibniz International Proceedings in Informatics*, pages 2:1–2:21. Schloss Dagstuhl.
- [Affeldt and Stone, 2024] Affeldt, R. and Stone, Z. (2024). A comprehensive overview of the Lebesgue differentiation theorem in Coq. In *15th International Conference on Interactive Theorem Proving (ITP 2024)*, September 9–14, 2024, Tbilisi, Georgia, volume 309 of *LIPICs*, pages 5:1–5:19. Schloss Dagstuhl - Leibniz-Zentrum für Informatik.
- [Appel, 2022] Appel, A. W. (2022). Coq’s vibrant ecosystem for verification engineering (invited talk). In *11th ACM SIGPLAN International Conference on Certified Programs and Proofs (CPP 2022)*, Philadelphia, PA, USA, January 17–18, 2022, pages 2–11. ACM.
- [Bernard et al., 2021] Bernard, S., Cohen, C., Mahboubi, A., and Strub, P.-Y. (2021). Unsolvability of the Quintic Formalized in Dependent Type Theory. In *12th International Conference on Interactive Theorem Proving (ITP 2021)*, volume 193 of *Leibniz International Proceedings in Informatics (LIPICs)*, pages 8:1–8:18, Dagstuhl, Germany. Schloss Dagstuhl – Leibniz-Zentrum für Informatik.
- [Bertot and Castéran, 2004] Bertot, Y. and Castéran, P. (2004). *Interactive Theorem Proving and Program Development—Coq’Art: The Calculus of Inductive Constructions*. Texts in Theoretical Computer Science. An EATCS Series. Springer.
- [Bertot and Castéran, 2015] Bertot, Y. and Castéran, P. (2015). *Le Coq’Art (V8)*. In French.
- [Bertot et al., 2008] Bertot, Y., Gonthier, G., Biha, S. O., and Pasca, I. (2008). Canonical big operators. In *International Conference on Theorem Proving in Higher Order Logics (TPHOLs 2008)*, Montreal, Canada, August 18–21, 2008, volume 5170 of *Lecture Notes in Computer Science*, pages 86–101. Springer.

- [Bourbaki, 1971] Bourbaki, N. (1971). *Éléments de mathématique—Topologie générale*. Hermann.
- [Broué, 2012] Broué, I. (2012). *Actes des journées X-UPS 2012*, chapter Quelques Moments de Vie Priviliégiés avec Henri et Nicole Cartan. Éditions de l'École polytechnique.
- [Church, 1940] Church, A. (1940). A formulation of the simple theory of types. *The Journal of Symbolic Logic*, 5(2):56–68.
- [Cohen and Sakaguchi, 2015] Cohen, C. and Sakaguchi, K. (2015). A finset and finmap library: Finite sets, finite maps, multisets and order. Available at <https://github.com/math-comp/finmap>. Last stable release: 2.1.0 (2024).
- [Cohen et al., 2020] Cohen, C., Sakaguchi, K., and Tassi, E. (2020). Hierarchy builder: Algebraic hierarchies made easy in coq with elpi (system description). In *5th International Conference on Formal Structures for Computation and Deduction (FSCD 2020), June 29–July 6, 2020, Paris, France (Virtual Conference)*, volume 167 of *LIPICs*, pages 34:1–34:21. Schloss Dagstuhl - Leibniz-Zentrum für Informatik.
- [Corry, 1992] Corry, L. (1992). Nicolas bourbaki and the concept of mathematical structure. *Synthese*, 92(3):315–348.
- [Garillot et al., 2009] Garillot, F., Gonthier, G., Mahboubi, A., and Rideau, L. (2009). Packaging mathematical structures. In *22nd International Conference on Theorem Proving in Higher Order Logics (TPHOLs 2009), Munich, Germany, August 17–20, 2009*, volume 5674 of *Lecture Notes in Computer Science*, pages 327–342. Springer.
- [Gonthier, 2008] Gonthier, G. (2008). Formal proof—the four-color theorem. *Notices of the AMS*, 55(11):1382–1393.
- [Gonthier et al., 2013] Gonthier, G., Asperti, A., Avigad, J., Bertot, Y., Cohen, C., Garillot, F., Roux, S. L., Mahboubi, A., O'Connor, R., Biha, S. O., Pasca, I., Rideau, L., Solovyev, A., Tassi, E., and Théry, L. (2013). A machine-checked proof of the odd order theorem. In *4th International Conference on Interactive Theorem Proving (ITP 2013), Rennes, France, July 22–26, 2013*, volume 7998 of *Lecture Notes in Computer Science*, pages 163–179. Springer.
- [Gonthier et al., 2016] Gonthier, G., Mahboubi, A., and Tassi, E. (2016). A Small Scale Reflection Extension for the Coq system. Research Report RR-6455, Inria Saclay Ile de France.
- [Gonthier and Tassi, 2012] Gonthier, G. and Tassi, E. (2012). A language of patterns for subterm selection. In *Proceedings of the 3rd International Conference on Interactive Theorem Proving (ITP 2012), Princeton, NJ, USA, August 13–15, 2012*, volume 7406 of *Lecture Notes in Computer Science*, pages 361–376. Springer.

- [Hagiwara and Affeldt, 2018] Hagiwara, M. and Affeldt, R. (2018). *Formal Proof using Coq/SSReflect/MathComp: Start Formalization of Mathematics with Free Software*. Morikita Publishing. In Japanese.
- [Halmos, 1974] Halmos, P. R. (1974). *Measure Theory*. Springer.
- [Harrison, 2018] Harrison, J. (2018). Let’s make set theory great again. Invited talk at the 3rd Conference on Artificial Intelligence and Theorem Proving (AITP 2018), March 25–30, 2018, Aussois, France.
- [Howard, 1980] Howard, W. A. (1980). The formulae-as-types notion of construction.
- [Infotheo, 2024] Infotheo (2024). A Coq formalization of information theory and linear error-correcting codes. <https://github.com/affeldt-aist/infotheo>. Open source software. Since 2009. Last stable version: 0.7.2.
- [Ishiguro and Affeldt, 2024] Ishiguro, Y. and Affeldt, R. (2024). The Radon-Nikodým theorem and the Lebesgue-Stieltjes measure in Coq. *Computer Software*, 41(2):41–59.
- [Kernighan and Pike, 1999] Kernighan, B. W. and Pike, R. (1999). *The Practice of Programming*. Addison-Wesley.
- [Klenke, 2013] Klenke, A. (2013). *Probability Theory: A Comprehensive Course*. Springer. 2nd edition.
- [Mahboubi and Tassi, 2021] Mahboubi, A. and Tassi, E. (2021). *Mathematical Components*. Zenodo.
- [Martin-Dorel and Tassi, 2019] Martin-Dorel, E. and Tassi, E. (2019). SSReflect in Coq 8.10. In *The Coq Workshop 2019, September 8, 2019, Portland, OR, USA*.
- [Rouhling, 2019] Rouhling, D. (2019). *Outils pour la Formalisation en Analyse Classique: Une Étude de Cas en Théorie du Contrôle*. PhD thesis, Université Côte d’Azur.
- [Russell, 1908] Russell, B. (1908). Mathematical logic as based on the theory of types. *American Journal of Mathematics*, 30(3):222–262.
- [Saito and Affeldt, 2022] Saito, A. and Affeldt, R. (2022). Towards a practical library for monadic equational reasoning in Coq. In *14th International Conference on Mathematics of Program Construction (MPC 2022), Tbilisi, Georgia, September 26–28, 2022*, volume 13544 of *Lecture Notes in Computer Science*, pages 151–177. Springer.
- [Saito and Affeldt, 2023] Saito, A. and Affeldt, R. (2023). Experimenting with an intrinsically-typed probabilistic programming language in Coq. In *21st Asian Symposium on Programming Languages and Systems (APLAS 2023), November 26–29, 2023, Taipei, Taiwan*, volume 14405 of *Lecture Notes in Computer Science*, pages 182–202. Springer.



- [Sozeau, 2009] Sozeau, M. (2009). Equations—a function definitions plugin. Available at <https://mattam82.github.io/Coq-Equations/>. Last stable release: 1.3 (2021).
- [The Coq Development Team, 2024] The Coq Development Team (2024). *The Coq Proof Assistant Reference Manual*. Inria. Available at <https://coq.inria.fr>. Version 8.19.2.
- [Voevodsky, 2014] Voevodsky, V. (2014). Univalent foundations. [https://www.math.ias.edu/vladimir/sites/math.ias.edu.vladimir/files/2014\\_IAS.pdf](https://www.math.ias.edu/vladimir/sites/math.ias.edu.vladimir/files/2014_IAS.pdf). Lecture at IAS.
- [Whitehead and Russell, 1927] Whitehead, A. N. and Russell, B. A. W. (1927). *Principia mathematica; 2nd ed.* Cambridge Univ. Press, Cambridge.
- [Wilansky, 2008] Wilansky, A. (2008). *Topology for Analysis*. Dover Publications. Republication. Originally work from 1970.



## Appendix A

# Cheat Sheets

The following cheat sheets might be useful to memorize naming conventions at first, but after a while the [Search](#) command and navigation into the appropriate files of MATHCOMP should be a better substitute.

# cheat sheet ssrbool.v (Coq v8.16)

## ssrfun.v naming conventions

K cancel  
LR move an op from the lhs of a rel to the rhs  
RL move an op from the rhs to the lhs

## ssrfun.v notations

f ~ y                      fun x => f x y  
p .1                      fst p  
p .2                      snd p  
f =1 g                    f x = g x  
{morph f : x / aF x >-> rR x}      f (aF x) = rF (f x)  
{morph f : x y / aOp x y >-> rOp x y}      f (aOp x y) = rOp (f x) (f y)

## ssrfun.v definitions

injective f  
cancel f g  
involutive f  
left\_injective op  
right\_injective op  
left\_id e op  
right\_id e op  
left\_zero z op  
right\_zero z op  
self\_inverse e op  
idempotent op  
commutative op  
associative op  
right\_commutative op  
left\_commutative op  
left\_distributive op add  
right\_distributive op add  
left\_loop inv op  
forall x1 x2, f x1 = f x2 -> x1 = x2  
g (f x) = x  
cancel f f  
injective (op ~ x)  
injective (op y)  
e □ x = x  
x □ e = x  
z □ x = z  
x □ z = z  
x □ x = e  
x □ x = x  
x □ y = y □ x  
x □ (y □ z) = (x □ y) □ z  
(x □ y) □ z = (x □ z) □ y  
x □ (y □ z) = y □ (x □ z)  
(x + y) \* z = (x \* z) + (y \* z)  
x \* (y + z) = (x \* y) + (x \* z)  
cancel (op x) (op (inv x))

## ssrbool.v naming conventions

A associativity  
AC right commutativity  
b a boolean argument  
C commutativity/complement  
D predicate difference  
E elimination  
F/f boolean false  
T/t boolean truth  
U predicate union

```
(* bool_scope *)
Notation "~~ b" := (negb b)
Notation "b ==> c" := (implb b c).
Notation "b1 (+) b2" := (addb b1 b2).
Notation "a && b" := (andb a b) Generalized to [&& b1 , b2 , ... , bn & c]
Notation "a || b" := (orb a b) Generalized to [|| b1 , b2 , ... , bn | c]
Notation "x \in A" := (in_mem x (mem A)).
Notation "x \notin A" := (~ (x \in A)).

negbT b = false -> ~~ b
negbTE ~~ b -> b = false
negbK involutive negb
contra (c -> b) -> ~~ b -> ~~ c
contraNF (c -> b) -> ~~ b -> c = false
contraFF (c -> b) -> b = false -> c = false
ifP if_spec (b = false) b (if b then vT else vF)
ifT b -> (if b then vT else vF) = vT
ifF b = false -> (if b then vT else vF) = vF
ifN ~~ b -> (if b then vT else vF) = vF

boolP alt_spec b1 b1 b1
negP reflect (~ b1) (~ b1)
negPn reflect b1 (~ ~ b1)
andP reflect (b1 /\ b2) (b1 && b2)
orP reflect (b1 \/ b2) (b1 || b2)
nandP reflect (~ b1 \/ ~ b2) (~ (b1 && b2))
norP reflect (~ b1 /\ ~ b2) (~ (b1 || b2))
implyP reflect (b1 -> b2) (b1 ==> b2)
andTb left_id true andb
andbT right_id true andb
andbb idempotent andb
andbC commutative andb
andbA associative andb
orFb left_id false orb
orbN b || ~ b = true
negb_and ~~ (a && b) = ~~ a || ~ b
negb_or ~~ (a || b) = ~ a && ~ b

Variant if_spec (not_b : Prop) : bool -> A -> Set :=
| IfSpecTrue of b : if_spec not_b true vT
| IfSpecFalse of not_b : if_spec not_b false vF.

Inductive reflect (P : Prop) : bool -> Set :=
| ReflectT of P : reflect P true
| ReflectF of ~ P : reflect P false.

Variant alt_spec (P : Prop) (b : bool) : bool -> Type :=
| AltTrue of P : alt_spec P b true
| AltFalse of ~ b : alt_spec P b false.

Notation xpred0 := (fun => false).
Notation xpredT := (fun => true).
Notation xpredU := (fun (p1 p2 : pred _) x => p1 x || p2 x).
Notation xpredC := (fun (p : pred _) x => ~ p x).
Notation "A =i B" := (eq_mem (mem A) (mem B)).
```

# cheat sheet `ssrnat.v` (SSREFLECT v1.15)

## `ssrfun.v` naming conventions

`K` cancel  
`LR` move an op from the lhs of a rel to the rhs  
`RL` move an op from the rhs to the lhs

## `ssrfun.v` notations

`f -- y`  
`p .1` `fset p`  
`p .2` `snd p`  
`f =1 g` `f x = g x`  
`{morph f : x / aF x -> rR x}` `f (aF x) = rF (f x)`  
`{morph f : x y / aOp x y -> rOp x y}` `f (aOp x y) = rOp (f x) (f y)`

## `ssrfun.v` definitions

`injective f`  
`cancel f g`  
`involutive f`  
`left_injective op`  
`right_injective op`  
`left_id e op`  
`right_id e op`  
`left_zero z op`  
`right_zero z op`  
`self_inverse e op`  
`idempotent op`  
`commutative op`  
`associative op`  
`right_commutative op`  
`left_commutative op`  
`left_distributive op add`  
`right_distributive op add`  
`left_loop inv op`  
`forall x1 x2, f x1 = f x2 -> x1 = x2`  
`g (f x) = x`  
`cancel f f`  
`injective (op^^ x)`  
`injective (op y)`  
`e □ x = x`  
`x □ e = x`  
`z □ x = z`  
`x □ z = z`  
`x □ x = e`  
`x □ x = x`  
`x □ y = y □ x`  
`x □ (y □ z) = (x □ y) □ z`  
`(x □ y) □ z = (x □ z) □ y`  
`x □ (y □ z) = y □ (x □ z)`  
`(x + y) * z = (x * z) + (y * z)`  
`x * (y + z) = (x * y) + (x * z)`  
`cancel (op x) (op (inv x))`

## `ssrbool.v` naming conventions

`A` associativity  
`AC` right commutativity  
`b` a boolean argument  
`C` commutativity/complement  
`D` predicate difference  
`E` elimination  
`F/f` boolean false  
`T/t` boolean truth  
`U` predicate union

## `ssrnat.v` naming conventions

`A`(infix) conjunction  
`B` subtraction  
`D` addition  
`p`(prefix) positive  
`S` successor  
`V`(infix) disjunction

`(* nat_scope *)`

`Notation "n .+1" := (succn n).` `Notation "n .*2" := (double n).` `Notation "n '!" := (factorial n).`

`Notation "n .-1" := (predn n).` `Notation "n ./2" := (half n).`

`Notation "m <n" := (m.+1 <=n).` `Notation "m ^ n" := (expn m n).`

`addOn/addn0` `left_id 0 addn/right_id 0 addn`  
`addin/addn1` `1 + n = n.+1/n + 1 = n.+1`  
`addn2` `n + 2 = n.+2`  
`addSn` `m.+1 + n = (m + n).+1`  
`addnS` `m + n.+1 = (m + n).+1`  
`addSnnS` `m.+1 + n = m + n.+1`  
`addnC` `commutative addn`  
`addnA` `associative addn`  
`addnCA` `left_commutative addn`  
`eqn_add2l` `(p + m == p + n) = (m == n)`  
`eqn_add2r` `(m + p == n + p) = (m == n)`  
`subOn/subn0` `left_zero 0 subn/right_id 0 subn`  
`subnn` `self_inverse 0 subn`  
`subSS` `m.+1 - n.+1 = m - n`  
`subn1` `n - 1 = n.-1`  
`subnDl` `(p + m) - (p + n) = m - n`  
`subnDr` `(m + p) - (n + p) = m - n`  
`addKn` `cancel (addn n) (subn^^ n)`  
`addnK` `cancel (addn^^ n) (subn^^ n)`  
`subSnn` `n.+1 - n = 1`  
`subnDA` `n - (m + p) = (n - m) - p`  
`subnAC` `right_commutative subn`  
`ltnS` `(m < n.+1) = (m <= n)`  
`prednK` `0 < n -> n.-1.+1 = n`  
`leqNgt` `(m <= n) = ^^ (n < m)`  
`ltnNge` `(m < n) = ^^ (n <= m)`  
`ltnn` `n < n = false`  
`subnDA` `n - (m + p) = (n - m) - p`  
`leq_eqVlt` `(m <= n) = (m == n) || (m < n)`  
`ltn_neqA1e` `(m < n) = (m != n) && (m <= n)`  
`ltn_add2l` `(p + m < p + n) = (m < n)`  
`leq_addr` `n <= n + m`  
`addn_gt0` `(0 < m + n) = (0 < m) || (0 < n)`  
`subn_gt0` `(0 < n - m) = (m < n)`  
`leq_sub2r` `m <= n -> m - p <= n - p`  
`leq_subLR` `(m - n <= p) = (m <= n + p)`  
`ltn_sub2r` `p < n -> m < n -> m - p < n - p`

`ltn_subRL` `(n < p - m) = (m + n < p)`  
`subnKC` `m <= n -> m + (n - m) = n`  
`subnK` `m <= n -> (n - m) + m = n`  
`addnBA` `p <= n -> m + (n - p) = m + n - p`  
`subnBA` `p <= n -> m - (n - p) = m + p - n`  
`subKn` `m <= n -> n - (n - m) = m`  
`mulOn/muln0` `left_zero 0 muln/right_zero 0 muln`  
`mul1n/muln1` `left_id 1 muln/right_id 1 muln`  
`mul2n/muln2` `2 * m = m.*2/m * 2 = m.*2`  
`mulnC` `commutative muln`  
`mulnA` `associative muln`  
`mulSn` `m.+1 * n = n + m * n`  
`mulnS` `m * n.+1 = m + m * n`  
`mulnDl` `left_distributive muln addn`  
`mulnDr` `right_distributive muln addn`  
`mulnBl` `left_distributive muln subn`  
`mulnBr` `right_distributive muln subn`  
`mulnCA` `left_commutative muln`  
`muln_gt0` `(0 < m * n) = (0 < m) && (0 < n)`  
`leq_pmulr` `n > 0 -> m <= m * n`  
`leq_mul2l` `(m * n1 <= m * n2) = (m == 0) || (n1 <= n2)`  
`leq_pmul2r` `0 < m -> (n1 * m <= n2 * m) = (n1 <= n2)`  
`ltn_pmul2r` `0 < m -> (n1 * m < n2 * m) = (n1 < n2)`  
`leqP` `leq_xor_gtn m n (m <= n) (n < m)`  
`ltngtP` `compare_nat m n (m < n) (n < m) (m == n)`  
`expn0` `m ^ 0 = 1`  
`expn1` `m ^ 1 = m`  
`expnS` `m ^ n.+1 = m * m ^ n`  
`exp0n` `0 < n -> 0 ^ n = 0`  
`exp1n` `1 ^ n = 1`  
`expnD` `m ^ (n1 + n2) = m ^ n1 * m ^ n2`  
`expn_gt0` `(0 < m ^ n) = (0 < m) || (n == 0)`  
`fact0` `0'! = 1`  
`factS` `(n.+1)'! = n.+1 * n'!`  
`odd_add` `odd (m + n) = odd m (+) odd n`  
`odd_double_half` `odd n + n./2.*2 = n`

`Variant leq_xor_gtn m n : nat -> nat -> nat -> nat -> bool -> bool -> Set :=`  
`| LeqNotGtn of m <= n : leq_xor_gtn m n m m n n true false`  
`| GtnNotLeq of n < m : leq_xor_gtn m n n n m m false true.`

`Variant compare_nat m n : nat -> nat -> nat -> bool -> bool -> bool -> bool -> bool -> Set :=`  
`| CompareNatLt of m < n : compare_nat m n m m n n false false false true false true`  
`| CompareNatGt of m > n : compare_nat m n n n m m false false true false true false`  
`| CompareNatEq of m = n : compare_nat m n m m m m true true true false false.`

$$\text{big\_morph} \quad (\forall x y, f(x + y) = f(x) \hat{+} f(y)) \rightarrow f(0) = \hat{0} \rightarrow f\left(\sum_{P(i)}^{i \leftarrow r} F(i)\right) = \hat{\sum}_{P(i)}^{i \leftarrow r} f(F(i))$$

### Section Extensionality

$$\begin{aligned} \text{eq\_bigl} \quad & P_1 =_1 P_2 \rightarrow \sum_{P_1(i)}^{i \leftarrow r} F(i) = \sum_{P_2(i)}^{i \leftarrow r} F(i) \\ \text{eq\_bigr} \quad & (\forall i, P(i) \rightarrow F_1(i) = F_2(i)) \rightarrow \sum_{P(i)}^{i \leftarrow r} F_1(i) = \sum_{P(i)}^{i \leftarrow r} F_2(i) \\ \text{big\_nil} \quad & \sum_{P(i)}^{i \leftarrow \emptyset} F(i) = 0 \\ \text{big\_pred0} \quad & P =_1 \text{xpred0} \rightarrow \sum_{P(i)}^{i \leftarrow r} F(i) = 0 \\ \text{big\_pred1} \quad & P =_1 \text{pred1}(i) \rightarrow \prod_{P(j)}^j F(j) = F(i) \\ \text{big\_ord0} \quad & \sum_{P(i)}^{i < 0} F(i) = 0 \\ \text{big\_tnth} \quad & \sum_{P(i)}^{i \leftarrow r} F(i) = \sum_{P(r_i)}^{i < \text{size}(r)} F(r_i) \\ \text{big\_nat\_recl} \quad & m \leq n \rightarrow \sum_{m \leq i < n+1} F(i) = F(m) + \sum_{m \leq i < n} F(i+1) \\ \text{big\_ord\_recl} \quad & \sum_{i < n+1} F(i) = F(\text{ord0}) + \sum_{i < n} F(\text{lift}((n+1), \text{ord0}, i)) \\ \text{big\_const\_ord} \quad & \sum_{i < n} x = \text{iter}(n, \lambda y. x + y, 0) \end{aligned}$$

### Section MonoidProperties

$$\begin{aligned} \text{bigl} \quad & (\forall i, P(i) \rightarrow F(i) = 1) \rightarrow \prod_{P(i)}^{i \leftarrow r} F(i) = 1 \\ \text{big\_nat\_recr} \quad & m \leq n \rightarrow \prod_{m \leq i < n+1} F(i) = \left(\prod_{i < n} F(i)\right) \times F(n) \end{aligned}$$

### Section Abelian

$$\begin{aligned} \text{big\_split} \quad & \prod_{P(i)}^{i \leftarrow r} (F_1(i) \times F_2(i)) = \prod_{P(i)}^{i \leftarrow r} F_1(i) \times \prod_{P(i)}^{i \leftarrow r} F_2(i) \\ \text{bigU} \quad & A \cap B = \emptyset \rightarrow \prod_{i \in A \cup B} F(i) = \left(\prod_{i \in A} F(i)\right) \times \left(\prod_{i \in B} F(i)\right) \\ \text{partition\_big} \quad & (\forall i, P(i) \rightarrow Q(p(i))) \rightarrow \prod_{P(i)}^{i \leftarrow s} F(i) = \prod_{Q(j)}^{j:J} \prod_{P(i)}^i F(i) \\ \text{reindex\_onto} \quad & (\forall i, P(i) \rightarrow h(h'(i)) = i) \rightarrow \prod_{P(i)}^i F(i) = \prod_{P(h(j))}^j F(h(j)) \\ \text{pair\_big} \quad & \prod_{P(i)}^i \prod_{Q(j)}^j F(i, j) = \prod_{P(p) \wedge Q(q)}^{(p, q)} F(p, q) \\ \text{exchange\_big} \quad & \prod_{P(i)}^{i \leftarrow rI} \prod_{Q(j)}^{j \leftarrow rJ} F(i, j) = \prod_{Q(j)}^{j \leftarrow rJ} \prod_{P(i)}^{i \leftarrow rI} F(i, j) \end{aligned}$$

### Section Distributivity

$$\begin{aligned} \text{big\_distr1} \quad & \left(\sum_{P(i)}^{i \leftarrow r} F(i)\right) \times a = \sum_{P(i)}^{i \leftarrow r} (F(i) \times a) \quad (\text{also } \text{big\_distr}) \\ \text{big\_distr\_big\_dep} \quad & \prod_{P(i)}^i \sum_{Q(i, j)}^j F(i, j) = \sum_{f \in \text{pfamily}(j_0, P, Q)} \prod_{P(i)}^i F(i, f(i)) \\ \text{big\_distr\_big} \quad & \prod_{P(i)}^i \sum_{Q(j)}^j F(i, j) = \sum_{f \in \text{pffun\_on}(j_0, P, Q)} \prod_{P(i)}^i F(i, f(i)) \\ \text{bigA\_distr\_big} \quad & \prod_i \sum_{Q(j)}^i F(i, f(i)) = \sum_{f \in \text{ffun\_on}(Q)} \prod_i F(i, f(i)) \\ \text{bigA\_distr\_bigA} \quad & \prod_{i \in I} \sum_{j \in J} F(i, j) = \sum_{f \in J^I} \prod_{i \in I} F(i, f(i)) \end{aligned}$$

$\text{pfamily}(j_0, P, Q) \simeq$   
functions  $Q^P$

$$\begin{aligned} \text{partition\_big\_imset} \quad & \sum_{i \in A} F(i) = \sum_{i \in h \circ \bullet : A} \sum_{h(i)=j}^{i \in A} F(i) \\ \text{big\_trivIset} \quad & \text{trivIset}(P) \rightarrow \sum_{x \in \text{cover}(P)} E(x) = \sum_{A \in P} \sum_{x \in A} E(x) \\ \text{partition\_disjoint\_bigcup} \quad & (\forall i, j, i \neq j \rightarrow F(i) \cap F(j) = \emptyset) \rightarrow \sum_{x \in \bigcup_i F(i)} E(x) = \sum_i \sum_{x \in F(i)} E(x) \end{aligned}$$

#### ssrfun.v naming conventions

K cancel  
LR move an op from the lhs of a rel to the rhs  
RL move an op from the rhs to the lhs

#### ssrfun.v definitions

```

injective f forall x1 x2, f x1 = f x2 -> x1 = x2
cancel f g g (f x) = x
involutive f cancel f f
left_injective op injective (op^^ x)
right_injective op injective (op y)
left_id e op e □ x = x
right_id e op x □ e = x
left_zero z op z □ x = z
right_zero z op x □ z = z
self_inverse e op x □ x = e
idempotent op x □ x = x
commutative op x □ y = y □ x
associative op x □ (y □ z) = (x □ y) □ z
right_commutative op (x □ y) □ z = (x □ z) □ y
left_commutative op x □ (y □ z) = y □ (x □ z)
left_distributive op add (x + y) * z = (x * z) + (y * z)
right_distributive op add x * (y + z) = (x * y) + (x * z)
left_loop inv op cancel (op x) (op (inv x))

```

#### ssrbool.v naming conventions

A associativity  
AC right commutativity  
b a boolean argument  
C commutativity/complement  
D predicate difference  
E elimination  
F/f boolean false  
T/t boolean truth  
U predicate union

#### finset.v naming conventions

0 the empty set  
T the full set  
1 singleton set  
C complement  
U union  
I intersection  
D difference

(\* set\_scope \*)

```

× set0
A :|: B setU
a |: A [set a] :|: A
A :&: B setI
~: A setC A
A :\: B setD A B
A :\: a A :\: [set a]
f @~-1: A preimset f (mem A)
f @: A imset f (mem A)
f @2: (A , B) imset2 f (mem A) (fun _ =>mem B) f(A, B)

```

(\* bool\_scope \*)

```

a \in A see ssrbool.v a ∈ A
A \subset B see fintype.v A ⊆ B
[disjoint A & B] see fintype.v A ∩ B = ∅

```

```

setP A =i B <-> A = B
in_set0 x \in set0 = false
subset0 (A \subset set0) = (A == set0)
in_set1 (x \in [set a]) = (x == a)
in_setD1 (x \in A :\: b) = (x != b) && (x \in A)
in_setU (x \in A :|: B) = (x \in A) || (x \in B)
in_setC (x \in ~: A) = (x \notin A)
(NB: inE is a multi-rule corresponding to in_set0, in_set1, in_setD1, in_setU, in_setC, etc.)
setUC A :|: B = B :|: A
setIC A :&: B = B :&: A
setKI A :|: (B :&: A) = A
setCI ~: (A :&: B) = ~: A :|: ~: B
setCK involutive (@setC T)
setD0 A :\: set0 =A
cardsE #|[set x in pA]| = #|pA| (NB: cardE : #|A|=size (enum A) in fintype.v)
cards0 #|@set0 T| = 0 (NB: card0 : #|@pred0 T|=0 in fintype.v)
cards_eq0 (#|A| == 0) = (A == set0)
cardsU #|A :|: B| = #|A| + #|B| - #|A :&: B|
cardsT #|[set: T]| = #|T| (NB: cardT : #|T|=size (enum T) in fintype.v)
setOPn reflect (exists x, x \in A) (A != set0)
subsetI1 A :&: B \subset A
subsetUr B \subset A :|: B
subsetI (A \subset B :&: C) = (A \subset B) && (A \subset C)
setI_eq0 (A :&: B == set0) = [disjoint A & B]
imsetP reflect (exists2 x, in_mem x D & y = f x) (y \in imset f D)
card_imset injective f -> #|f @: D| = #|D|

```

#### Section Partitions

cover P  $\bigcup_{B \in P} B$   
trivIset P  $\sum_{B \in P} |B| = |\text{cover}(P)|$   
see also bigop\_doc.pdf





## Appendix B

# Rocq and MathComp Installation

In case of emergency, you can use ROCQ in a web browser, just search for jsCoq (“JavaScript Coq”) on the web. Maybe try <https://coq.vercel.app/>.

You can also install the ROCQ platform. It is a set of compatible packages for ROCQ that is easy to install.

It is however much more convenient to install ROCQ through opam on your computer and to have the source code at hand. You can find installation instructions online, e.g., installation on Linux and Windows. (Please, PR on github if you find errors in these installation notes or if you want to propose an improvement.)

ROCQ is typically used through a customizable text editor. The most popular choice is Emacs with the Proof General extension (and possibly also the Company-coq extension). It is arguably the best solution in terms of speed of edition and integration with other tools, in particular in a Unix-like environment such as Linux or MacOS.

CoqIDE is a text editor specific ROCQ. It is a popular choice for beginners, some advanced users also manage to be productive with it. It comes with the ROCQ platform.

Visual Studio Code is a more recent alternative. It might be a better choice than Emacs on Windows thanks to a good interaction with WSL. It *looks* more modern than Emacs.



# List of Tables

|     |                                                                        |    |
|-----|------------------------------------------------------------------------|----|
| 2.1 | Some files of interest in ROCQ . . . . .                               | 24 |
| 3.1 | Examples of scopes used in MATHCOMP . . . . .                          | 36 |
| 3.2 | A few generic definitions in MATHCOMP . . . . .                        | 37 |
| 3.3 | Naming Convention: Identifiers for operations . . . . .                | 38 |
| 3.4 | Naming Convention: Identifiers for positional notation . . . . .       | 38 |
| 3.5 | Naming Convention: Suffixes for the properties of operations . . . . . | 39 |
| 3.6 | Naming Convention: Identifiers for relations . . . . .                 | 39 |
| 3.7 | Some files of interest in MATHCOMP . . . . .                           | 40 |
| 3.8 | Summary of iterated operations . . . . .                               | 50 |
| 5.1 | Examples of scopes used in MATHCOMP-ANALYSIS . . . . .                 | 73 |
| 5.2 | Comparison of measurable structures . . . . .                          | 76 |



# List of Figures

|     |                                                                  |    |
|-----|------------------------------------------------------------------|----|
| 3.1 | The mathematical structures of MATHCOMP . . . . .                | 41 |
| 3.2 | Some conversions between numeric types . . . . .                 | 53 |
| 4.1 | Mathematical structures of MATHCOMP-ANALYSIS . . . . .           | 59 |
| 4.2 | Hierarchy of mathematical structures to defined real numbers . . | 60 |
| 5.1 | Hierarchy of measure theory structures . . . . .                 | 75 |
| 5.2 | Hierarchy of measure structures . . . . .                        | 82 |
| 6.1 | Hierarchy for non-negative simple functions . . . . .            | 87 |
| 6.2 | Approximation of a measurable function using simple functions .  | 88 |

# Index

- $\sigma$ -algebra, 77
- algebra of sets, 77
- attribute, 42
- backward reasoning, 33
- ceil, 52
- charge, 82
- clear-switch, 20
- coercion, 42
  - is\_true, 42
  - nat\_of\_ord, 46
- command
  - About, 21
  - Check, 21
  - Context, 33
  - Definition, 19
  - End, 33
  - Fixpoint, 26
  - HB.builders, 79
  - HB.factory, 40, 79
  - HB.instance, 40, 80
  - HB.mixin, 40
  - HB.structure, 40
  - Hypothesis, 33
  - Inductive, 23
  - Lemma, 19
  - Let, 33
  - Print, 21
  - Record, 39
  - Search, 22
  - Section, 33
  - Variable, 33
  - Variant, 23
- connector, 64
- constructor, 23
- continuous, 62
- Curry-Howard correspondence, 9
- delimiter
  - E, 73
  - classic, 57
- dependent pair, 30
- diagonal, 64
- differentiation, 97
- dyadic interval, 88
- entourage, 64
- factory, 79, 82
- filter, 61
  - proper filter, 61
- filterbase, 61
- filtered type, 62
- floor, 52
- forward reasoning, 33
- generated  $\sigma$ -algebra, 79
- goal, 19
- identifier
  - interval, 53
  - set\_system, 61
  - trivIset, 58
  - xget, 58
- implicit arguments, 21, 31
- impredicative, 18
- index, 27
- indicator function, 83
- integer, 52
- integrable, 94

- interval, 53
- intro-pattern, 25
- Lebesgue integral, 90
- measurable function, 86
- measure, 81
  - $\sigma$ -finite measure, 82
  - additive measure, 81
  - content, 81
  - Dirac measure, 83
  - Lebesgue measure, 86
  - Lebesgue-Stieltjes measure, 86
  - product measure, 86, 94
  - pushforward measure, 86
  - signed measure, 82
- mixin, 40
- neighborhood, 63
- norm, 52
- normed module, 66
- notation
  - `'D_`, 97
  - `+oo`, 73
  - `-oo`, 73
  - `\_`, 59
  - `\oo`, 61
  - `\o`, 36
  - `^~()`, 52, 97
  - `~`, 18, 36
  - `~&^`, 57
  - `^<=^`, 57
  - `~^`, 57
- occurrence switch, 28
- open set, 63
- ordinal, 46
- parameter, 27
- polymorphism, 13
- poset, 57
- positional notation, 38
- predicative, 18
- pseudometric space, 64
- real number, 59
- ring of sets, 76
- scope, 23, 35
  - `classical_set_scope`, 57
  - `ereal_scope`, 73
  - `seq_scope`, 47
  - Table 5.1, 73
  - Table 3.1, 36
- script, 14, 19
- section
  - x-section, 64
- semi- $\sigma$ -additive, 81
- semiadditive, 81
- semiring of sets, 76
- sigma-type, 30, 42
- simple function, 87, 88
- simplification operation, 28
- strict implicit argument, 32
- supremum, 58
- tactic, 14, 20
  - `apply`, 21
  - `case`, 24
  - `elim`, 26
  - `exact`, 21
  - `have`, 33
  - `left`, 29
  - `move`, 20
  - `near:`, 67
  - `near=>`, 67
  - `pose`, 33
  - `rewrite`, 28
  - `right`, 29
  - `set`, 33
  - `split`, 29
  - terminating tactic, 21
- tactical, 20
  - `:`, 20
  - `;`, 24
  - `=>`, 20
  - `by`, 32
- topological space, 63
- type family, 27
- uniform space, 64